



中国科学院大学

University of Chinese Academy of Sciences

硕士学位论文

基于多元时序的分布式集群异常检测研究

作者姓名: 杨婧雯

指导教师: 裴昶华 副研究员

中国科学院计算机网络信息中心

学位类别: 电子信息硕士

学科专业: 计算机技术

培养单位: 中国科学院计算机网络信息中心

2026 年 6 月

Research on Anomaly Detection for Distributed Clusters Based
on Multivariate Time Series

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Electronic Information
in Computer Technology

By

YANG Jingwen

Supervisor: Prof. PEI Changhua

Computer Network Information Center, Chinese Academy of Sciences

June, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。承诺除文中已经注明引用的内容外，本论文不包含任何其他个人或集体享有著作权的研究成果，未在以往任何学位申请中全部或部分提交。对本论文所涉及的研究工作做出贡献的其他个人或集体，均已在文中以明确方式标明或致谢。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院大学有关收集、保存和使用学位论文的规定，即中国科学院大学有权按照学术研究公开原则和保护知识产权的原则，保留并向国家指定或中国科学院指定机构送交学位论文的电子版和印刷版文件，且电子版与印刷版内容应完全相同，允许该论文被检索、查阅和借阅，公布本学位论文的全部或部分内容，可以采用扫描、影印、缩印等复制手段以及其他法律许可的方式保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘 要

分布式集群是支撑云计算与大数据处理的核心基础设施，其稳定运行直接关系到上层业务的连续性。现有的时序异常检测方法大多聚焦于时间维度和指标维度的二维分析，在面对负载敏感型指标时容易产生误报。针对这一问题，本文提出了基于多元时序曲线相似性的分布式集群异常检测方法，将分布式系统中的节点维度系统性地引入检测模型，通过度量节点间监控曲线的偏离程度来识别异常节点。

针对单指标多节点场景，本文基于联通大数据生产环境的 YARN 集群监控数据构建了高质量的标注数据集，分别采用欧氏距离、自编码器嵌入和密度聚类三种方法从不同视角度量节点曲线间的相似性。针对单一方法阈值敏感的问题，提出了 CSAD-AT 算法，通过分数归一化与加权聚合融合多视角评分，并使用改进的包容性 SPOT 算法实现阈值的动态自适应调整。实验结果表明，该方法在 F1 分数上显著优于多种主流的多指标时序异常检测算法。

针对多指标多节点场景，本文提出了 NSFusion 数值与形状双视角融合的异常检测框架。面向多节点多指标下指标数量增长引起的效率挑战，框架采用轻量化的数值距离计算与无需训练的符号化形状度量相结合的方案替代单指标场景中计算开销较大的自编码器和密度偏离方法。面向多节点多指标下变化模式复杂对算法检测稳定性的挑战，本文在数值视角采用优化的基于欧氏距离的多指标加权融合方法量化节点间的数值差异；在形状视角提出全分辨率符号聚合近似算法，通过保留原始时间分辨率来提升对曲线形状变化的敏感性，弥补纯数值视角的检测盲区。在真实 GPU 集群故障数据集上的实验表明，双视角融合方法的异常节点检测准确率达到 83.8%，显著优于单一视角方法和传统基线方法。

在工程实现方面，本文设计并开发了基于曲线相似性的集群异常节点检测系统，采用前后端分离的四层架构，涵盖数据接入、检测引擎、常态化监控、告警管理和可视化看板五个功能模块。系统以模块化方式封装了全部检测算法，支持定时自动巡检和多级告警通知，具备对接生产环境监控体系的接口能力，单次巡检端到端耗时约 20 秒，满足分钟级巡检间隔的性能要求。

关键词：分布式集群，异常检测，多元时序，曲线相似性，自适应阈值

Abstract

Distributed clusters serve as the core infrastructure for cloud computing and big data processing, where operational stability is critical to business continuity. Existing time series anomaly detection methods primarily focus on temporal and feature dimensions, failing to exploit the strong behavioral similarity among cluster nodes, which leads to high false positive rates for load-sensitive metrics. To address this problem, this thesis proposes a distributed cluster anomaly detection approach based on multivariate time series curve similarity, systematically incorporating the node dimension into the detection model and identifying anomalous nodes by measuring the deviation of their monitoring curves from the cluster-wide behavioral pattern.

For the single-metric multi-node scenario, this thesis constructs a high-quality labeled dataset from the YARN cluster monitoring data in a production environment, and employs three complementary methods, namely Euclidean distance, autoencoder embedding, and density-based clustering, to measure inter-node curve similarity from different perspectives. To overcome the threshold sensitivity of individual methods, the CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) framework is proposed, which fuses multi-perspective scores through normalization and weighted aggregation and introduces an improved Inclusive SPOT algorithm for dynamic adaptive thresholding. Experimental results demonstrate that the proposed method significantly outperforms several state-of-the-art multivariate time series anomaly detection algorithms in terms of F1 score.

For the multi-metric multi-node scenario, this thesis proposes the NSFusion dual-perspective fusion framework combining numerical and shape analysis. To address the efficiency constraints imposed by the substantial data volume growth in multi-node multi-metric settings, the framework replaces the computationally expensive autoencoder and density-based methods from the single-metric scenario with a lightweight combination of numerical distance computation and training-free symbolic shape measurement. To address the challenge that complex variation patterns in multi-node multi-metric settings pose to detection stability, the numerical perspective quantifies inter-node value differences through an optimized Euclidean distance-based multi-metric weighted fusion, while the shape perspective introduces a Full-Resolution Symbolic Aggregate Approximation algorithm that preserves the original temporal resolution to enhance sensitivity to curve shape variations, compensating for the detection blind spots of the purely numerical perspective. Experiments on a real-world GPU cluster fault dataset show that the fusion method achieves an anomalous node detection accuracy of 83.8%, significantly outperforming single-perspective methods and traditional base-

lines.

On the engineering side, this thesis designs and implements a curve similarity-based cluster anomalous node detection system with a four-layer front-back separated architecture, encompassing five functional modules: data ingestion, detection engine, routine monitoring, alert management, and visualization dashboard. The system encapsulates all detection algorithms in a modular manner, supports scheduled automatic patrol inspection and multi-level alert notification, and provides interfaces for integration with production monitoring systems. The end-to-end latency of a single patrol inspection is approximately 20 seconds, meeting the performance requirements for minute-level inspection intervals.

Key Words: Distributed Cluster, Anomaly Detection, Multivariate Time Series, Curve Similarity, Adaptive Threshold

目 录

第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 论文研究内容和贡献	5
1.3.1 主要研究内容	5
1.3.2 主要贡献	5
1.4 论文组织结构	6
第 2 章 相关背景知识与理论基础	9
2.1 时间序列异常概述	9
2.1.1 时间序列异常数据简介	9
2.1.2 时间序列异常定义与分类	9
2.2 时序异常检测算法	12
2.2.1 单变量时序异常检测	12
2.2.2 多元时序异常检测	14
2.3 分布式集群监控数据与异常检测	16
2.3.1 分布式集群概述	16
2.3.2 集群监控数据特点	17
2.3.3 分布式异常检测系统架构	18
2.3.4 异常检测方法在集群数据中的应用与挑战	18
2.4 本章小结	19
第 3 章 CSAD-AT: 阈值鲁棒的单指标多节点异常检测算法	21
3.1 多节点单指标数据构建	21
3.1.1 数据来源与采集	21
3.1.2 数据集预处理与标注	22
3.1.3 数据集统计与可视化	23
3.2 基于时间和空间的异常检测算法评估	24
3.2.1 基线算法选择	24
3.2.2 评估结果分析	25
3.3 基于节点相似性的异常检测算法	25
3.3.1 基于欧氏距离的数值相似性度量	26
3.3.2 基于自编码器嵌入的形态相似性度量	26

3.3.3 基于 DBSCAN 聚类的密度相似性度量	27
3.3.4 参数敏感性实验及分析	28
3.4 CSAD-AT: 阈值自适应的异常检测算法	31
3.4.1 算法框架概述	31
3.4.2 多视角异常评分	32
3.4.3 分数归一化与聚合	33
3.4.4 I-SPOT: 包容性自适应阈值检测算法	33
3.5 实验验证与分析	36
3.5.1 实验设置	36
3.5.2 整体性能分析	36
3.5.3 分数聚合策略对比实验	37
3.5.4 消融实验: 各评分方法贡献分析	37
3.5.5 I-SPOT 与经典 SPOT 对比实验	38
3.6 本章小结	40
第 4 章 NSFusion: 数值-形状融合的多指标多节点异常检测	41
4.1 问题定义及 NSFusion 算法框架	41
4.1.1 真实场景数据概述	41
4.1.2 问题分析与挑战	43
4.1.3 NSFusion 算法框架	44
4.2 基于数值特征的异常检测算法	45
4.2.1 基于欧氏距离的单指标异常度量	45
4.2.2 多指标加权融合策略	46
4.2.3 算法实现与效率优化	47
4.3 基于形状特征的异常检测算法	48
4.3.1 经典 SAX 算法原理	48
4.3.2 面向集群故障检测的 FR-SAX 算法	49
4.3.3 算法效率优化	51
4.4 多算法融合与故障定位	51
4.4.1 双视角加权融合设计	51
4.4.2 异常节点检测	52
4.5 实验验证与对比分析	52
4.5.1 实验设置	52
4.5.2 方法结果对比分析	53
4.5.3 融合方法优势分析	53
4.5.4 算法效率实验对比	55
4.6 本章小结	56

第 5 章 集群节点异常检测与常态化监控系统	59
5.1 系统设计	59
5.1.1 功能模块设计	59
5.1.2 系统架构设计	60
5.1.3 数据库设计	61
5.2 系统实现	62
5.2.1 数据接入与预处理	62
5.2.2 检测引擎	62
5.2.3 常态化监控	63
5.2.4 告警管理	63
5.2.5 可视化与数据看板	64
5.2.6 RESTful API 接口设计	64
5.3 系统功能验证	64
5.3.1 系统性能评估	65
5.3.2 常态化监控功能验证	66
5.3.3 告警功能验证	66
5.4 本章小结	67
第 6 章 总结与展望	69
6.1 本文工作总结	69
6.2 未来研究展望	70
参考文献	71
致谢	77
作者简历及攻读学位期间发表的学术论文与其他相关学术成果	79

图目录

图 1-1	论文研究内容框架图	6
图 2-1	点异常示意图	10
图 2-2	片段异常示意图	11
图 2-3	变量间关联异常示意图	11
图 2-4	节点级异常示意图	12
图 2-5	Hadoop 分布式集群架构示意图	17
图 3-1	YARN 分布式集群架构与 RPC Router 连接数指标示意	22
图 3-2	数据集预处理与半人工标注流程	22
图 3-3	测试集 A 典型异常片段时序可视化	24
图 3-4	自编码器网络结构示意图	27
图 3-5	三种检测方法在不同窗口大小下 F1 分数随阈值变化的折线图	30
图 3-6	CSAD-AT 算法整体架构	32
图 3-7	融合分数上经典 SPOT 与 I-SPOT 检测结果对比	39
图 4-1	GPU 集群典型故障案例的多指标多节点时序可视化	43
图 4-2	NSFusion 算法框架流程	45
图 4-3	传统 SAX 与 FR-SAX 的符号化过程对比	50
图 4-4	传统 SAX 与 FR-SAX 的符号距离矩阵对比	50
图 4-5	FR-SAX 方法优势案例	54
图 4-6	欧氏距离方法优势案例	55
图 5-1	异常检测系统检测结果可视化界面	65

表目录

表 3-1	数据集统计数据	23
表 3-2	基线算法异常检测结果	25
表 3-3	基于曲线相似性方法在最优参数下的异常检测结果	28
表 3-4	欧氏距离方法的参数敏感性实验结果	29
表 3-5	自编码器嵌入方法的参数敏感性实验结果	29
表 3-6	密度偏离方法的参数敏感性实验结果	30
表 3-7	各算法异常检测性能对比	36
表 3-8	不同分数聚合策略的性能对比	37
表 3-9	消融实验：各评分方法与 I-SPOT 结合的性能	38
表 3-10	经典 SPOT 与 I-SPOT 性能对比	38
表 4-1	故障类别及数量分布	41
表 4-2	监控指标列表及说明	42
表 4-3	多指标多节点异常检测方法对比	53
表 4-4	算法优化前后效率对比	56
表 4-5	优化后不同节点规模下的平均检测耗时统计	56
表 5-1	系统算法模块组织结构	61
表 5-2	系统核心 API 接口设计	64
表 5-3	系统检测性能（单位：秒）	66
表 5-4	常态化巡检端到端耗时分布（30 节点 × 500 步）	66

第1章 绪论

1.1 研究背景及意义

分布式集群（如 Hadoop、Kubernetes）是结合了分布式系统和集群系统优点的一种架构，在分布式集群中，不同的服务器负责不同的任务，同时每个任务可以由多个服务器组成的集群来处理。分布式集群是支撑现代云计算与大数据处理的核心基础设施，其稳定性直接关系到上层业务的连续性。此类集群在运行中会产生海量的多维度监控时序数据（如 CPU 负载、内存使用等），这些数据是感知系统健康状态、诊断异常的关键。

然而，随着集群规模不断扩大、架构动态性增强以及业务负载波动加剧，传统的基于阈值的异常检测方法因依赖先验知识、适应性差，导致误报和漏报率高，难以满足生产环境需求。尽管基于深度学习的方法能自动学习复杂模式，但在实际生产环境中，一个核心挑战在于对于如 RPC 连接数这类与用户任务相关的指标，其“正常”的绝对范围和模式难以统一定义，会随系统整体负载发生无规律波动，若使用基于历史数据训练的模型，会在整体高负载时产生大量误报，严重干扰运维效率。在分布式集群中，由于负载均衡机制的作用，在系统正常运行时，同一时刻各节点所承受的负载是相近的，各个节点指标曲线往往表现出极大的相似性，在运维实践中，运维人员常通过对比分布式集群中的同一指标曲线来判断异常：与其他变化模式相差很大的“离群曲线”更可能是异常。本研究正是基于这一洞察，旨在将“分布式集群相似性”这一先验知识系统性地引入多元时序异常检测模型，通过对比集群内多个节点在同一监控指标上的行为曲线，来识别其中显著偏离“群体一致性”模式的个别节点或子集，以提升异常检测的效果。

具体而言，本研究针对的是多节点时序数据集，即对同一指标（如 CPU 使用率）采集自集群中多个节点的、具有时间对齐关系的多条时序曲线。该数据集不仅包含每个节点自身的时序变化模式，更隐含了节点之间的运行关联与协同规律。如何有效捕捉并利用这种分布式关联，构建能够区分节点个体异常与集群整体波动的检测模型，是本研究要解决的关键问题。

本研究的意义在于，从方法和实践应用两个层面为分布式集群异常检测提供了新的思路与解决方案。在方法层面，本研究跳出了传统异常检测聚焦于“时间”与“指标”二维分析的框架，创新性地将“集群节点”这一空间维度系统性地引入模型，构建了“时间 $\times$ 指标 $\times$ 节点”的三维分析范式。这一范式将集群内在的、由负载均衡机制保证的节点行为相似性，从一种隐性的运维经验转化为一种显式的、可计算的先验知识，为处理动态负载场景下的异常检测问题提供了新的视角。在实践应用层面，所提出的方法直接针对生产环境中负载敏感型指标检测的痛点，通过识别“离群曲线”而非定义“绝对正常”，能够有效降低因集群整体负载波动引发的误报，提升检测结果的可靠性。基于真实运维数据构建数据集并

验证算法,既保证了方法在实际业务场景下的有效性,也为后续研究提供了数据基础。本研究结果不仅有助于实现更精准的故障定位与预警,缩短平均故障恢复时间,提升业务连续性,也为提升真实集群运维体系智能化提供了核心的检测能力支撑,具有重要的工程应用价值。

1.2 国内外研究现状

时间序列异常检测是数据挖掘和机器学习领域的重要研究方向,旨在识别时间序列数据中偏离正常行为模式的异常点或异常序列。随着物联网、工业 4.0 和金融科技的发展,时间序列数据规模呈指数级增长,对高效、准确的异常检测方法提出了更高要求。纵观该领域的发展历程,研究方法呈现出清晰的演进脉络:从基于严格统计假设的传统方法,到依赖特征工程的经典机器学习方法,再到能够端到端自动学习复杂模式的深度学习方法,体现了从模型驱动向数据驱动的智能学习范式转变。

传统统计方法奠定了异常检测的理论基石,其核心思想是基于概率分布假设进行统计推断,将显著偏离统计特性的观测点判定为异常。最具代表性的是 3-sigma 准则,该方法假设数据服从正态分布,根据切比雪夫不等式的推论,正常数据落在均值 μ 加减三倍标准差 σ 区间内的概率约为 99.7%,因此将落在该区间之外的点视为异常。然而,该方法对分布假设敏感,且难以处理存在多个离群值的情况。针对单个离群值的检测,Grubbs 检验^[1]提供了更严谨的统计框架,其检验统计量定义为 $G = \frac{\max_s |x_i - \bar{x}|}{s}$,通过与基于样本量和显著性水平确定的临界值比较来判断异常,该方法适用于样本量较小且仅含单个离群值的场景。Dixon 检验^[2](又称 Q 检验)则通过计算可疑值与相邻值的差异相对于数据全距的比例来判断异常,其计算简便且对极端值检测具有较好的鲁棒性。对于多变量场景,上述单变量方法存在明显局限,因为异常点可能在各单变量上并不极端,但其变量组合模式偏离正常关联结构。Mahalanobis 距离^[3]通过引入协方差矩阵来度量样本点偏离分布中心的程度,其定义为 $D_M = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})}$,能够有效识别这类多变量关联异常。此外,基于时序模型的方法如 ARIMA^[4]通过差分运算将非平稳序列转化为平稳序列,再结合自回归和移动平均成分建模时间序列的自相关结构,将预测残差作为异常判据;分解方法如 STL^[5]采用局部加权回归将时间序列分解为趋势、季节和残差三个成分,通过分析残差成分中的极端值来识别异常。此外,在降维与变换方法方面,主成分分析(PCA)作为经典降维技术也被应用于异常检测,Defreitas 等人^[6]将 PCA 用于高维实验数据中的异常识别,通过提取主成分在降维的同时保留关键变异信息。在频域分析方面,Sharpnack 等人^[7]提出基于图傅里叶扫描统计量的网络异常活动检测方法,将信号处理的思想引入异常检测领域;Grané 和 Veiga^[8]则将小波变换应用于金融时间序列中的异常值检测,通过多尺度分解有效捕捉不同时间尺度上的异常波动。传统统计方法原理简单、可解释性强,但严重依赖分布假设,在面对复杂非线性模式和高维数据时能力有限。

经典机器学习方法拓展了异常检测的实用边界，其核心转变是从基于分布假设转向基于数据特征的相似性或密度度量。距离基方法直接利用距离度量分析样本间的相似性，其基本假设是异常点与正常点之间的距离较大。K 近邻方法^[9]通过计算每个样本到其第 K 个最近邻居的距离作为异常分数，距离越大则越可能是异常。密度基方法则从局部密度的角度刻画异常，其中局部离群因子 (LOF)^[10]是最具代表性的算法，它通过比较样本点的局部可达密度与其邻居的局部可达密度来计算离群因子，LOF 值显著大于 1 表明该点相对于其邻域更加稀疏，因而更可能是异常。与距离基方法相比，LOF 能够更好地处理密度分布不均匀的数据。隔离基方法以孤立森林^[11]为代表，其核心思想源于一个直观的观察：异常点由于“少且不同”的特性，在随机分割构成的树结构中更容易被隔离出来。具体而言，算法通过随机选择特征和分割点递归划分数据空间构建隔离树，异常点由于偏离正常数据密集区域，通常只需较少的分割次数即可被隔离，因此其在树中的平均路径长度较短。孤立森林的时间复杂度为 $O(n \log n)$ ，相比需要计算成对距离的方法具有显著的效率优势，因此在工业界得到广泛应用。在聚类分析方面，MacQueen^[12]最早提出 K-means 聚类算法，为后续基于聚类的异常检测方法奠定了算法基础；Celebi^[13]系统比较了 K-means 不同初始化策略对聚类质量和收敛速度的影响，为聚类方法的合理应用提供了实践指导。Li 等人^[14]将聚类方法应用于多变量时间序列异常检测，通过对时间序列进行聚类分析将不属于主要簇的数据段识别为异常，验证了聚类方法在不同类型时序数据中的适用性。在高维场景中，Angiulli 和 Pizzuti^[15]提出了一种高维空间中的快速离群点检测算法，通过优化距离计算策略显著降低了传统离群点检测在高维数据上的计算复杂度。此外，Chen 和 Guestrin^[16]提出的 XGBoost 梯度提升决策树框架凭借其内置正则化和高效计算能力，被广泛应用于包括时序异常检测在内的多种数据分析任务。这些经典方法的共同优势在于无需大量标注数据，但效果依赖于精心设计的特征工程，且由于未显式建模时间依赖关系，难以有效捕捉时间序列中复杂的非线性关系和长期依赖。

深度学习方法实现了异常检测性能的显著突破^[17,18]，其核心创新是利用神经网络自动学习数据的内在表示，摆脱了对人工特征工程的依赖。重构型方法以自编码器及其变体为代表，其核心假设是：模型在正常数据上训练后学习到了正常模式的低维表示，当输入异常数据时由于其模式未被学习，解码器难以准确重构，从而产生较大的重构误差。OmniAnomaly^[19]在此基础上引入随机变分推断和归一化流，通过建模正常数据的复杂潜在分布来提升检测能力；USAD^[20]则采用两个共享编码器的解码器进行对抗训练，增强了模型对异常的敏感性。预测型方法利用序列模型的时序建模能力，通过预测未来时间步的取值并将预测误差作为异常分数。长短期记忆网络 (LSTM)^[21]通过门控机制有效缓解了梯度消失问题，能够捕捉序列中的长期依赖关系。近年来，Transformer 架构凭借其强大的自注意力机制在时序建模中展现出优势，TranAD^[22]采用两阶段对抗训练机制，通过“自调节”策略放大异常样本的重建误差；Anomaly Transformer^[23]则提出关联差异性概念，利用异常点在时序关联模式上的特殊性进行检测。在

多变量时序场景中，如何建模变量间的复杂依赖关系是核心挑战。图神经网络因能够显式建模变量间的拓扑关系而被广泛应用，GDN^[24]通过注意力机制自动学习传感器之间的图结构，并基于图卷积网络进行预测。此外，基于因果分析的方法将异常视为偏离正常因果机制的实例，CAROTS^[25]创新性地将因果关系融入对比学习框架，通过设计保持因果结构和破坏因果结构的两类数据增强策略，训练模型在潜在空间中区分正常与异常样本，同时支持异常根因定位。在基础网络架构方面，He 等人^[26]提出的深度残差网络通过引入残差连接解决了深层网络训练退化问题，其残差学习思想被广泛借鉴至时序异常检测模型的构建中。在实际应用方面，Hundman 等人^[27]将 LSTM 网络与非参数动态阈值机制相结合用于航天器遥测数据的异常检测，展示了深度学习方法在高可靠性要求场景中的实践价值。Munir 等人^[28]提出的 DeepAnT 采用卷积神经网络提取时间序列的局部模式特征进行无监督异常检测，其无需标注数据的特性使其在实际应用中具有广泛的适用性。深度学习的主要优势在于强大的表征学习能力，但也面临训练不稳定、计算资源需求高、可解释性差等局限。

在研究趋势方面，当前领域呈现出多个重要发展方向。频域分析与时域分析的结合成为提升检测能力的新途径^[29]，通过傅里叶变换等方法从频率视角捕捉异常模式。对比学习与自监督学习的兴起为解决标签稀缺问题提供了新思路^[30]，通过设计有效的数据增强策略从无标签数据中学习鲁棒表征。可解释性与评估标准化受到越来越多的重视，因果推断方法被引入以增强模型透明度，专门化评估指标如 VUS-PR^[31]和综合性基准框架如 TAB^[32]不断涌现，为公平比较不同方法提供了平台。随着数据规模的增长，分布式系统架构的发展也成为必然趋势，基于 Spark^[33]、Flink^[34]等框架的系统实现了大规模时序数据的高效处理。预训练模型与大语言模型的兴起为时序异常检测开辟了新范式。Devlin 等人^[35]提出的 BERT 模型和 Radford 等人^[36]提出的 GPT 模型分别通过双向编码和生成式预训练策略在自然语言处理领域取得了突破性进展，其预训练思想被逐步引入时间序列分析领域。Dang 等人^[37]率先探索了将语言模型应用于时序异常检测的可行性，并进一步提出 TS-Bert 方法^[38]将 BERT 的预训练与微调范式系统性地迁移至时序建模。Alnegheimish 等人^[39]提出的 SigLLM 框架则创新性地探索了大语言模型作为零样本时序异常检测器的可能性，无需额外训练即可部署，展示了大语言模型在跨平台通用性方面的优势，但其推理开销也限制了在实时场景中的应用。在国内，智能运维（AIOps）作为将人工智能技术应用于 IT 运维的新兴方向受到广泛关注，裴丹等人^[40]系统梳理了基于机器学习的智能运维技术体系，涵盖异常检测、故障定位等核心环节。

总体来看，现有研究虽然取得了显著进展，但绝大多数工作仍聚焦于时间与指标这两个维度构成的二维数据分析框架内。一个普遍的局限性在于，未能系统性地挖掘和利用分布式集群生产环境中广泛存在的多节点这一宝贵维度信息，忽视了同一集群内各节点指标行为模式的内在相似性及其对异常判别的辅助价值。因此，本研究旨在填补这一空白，探索一种基于时间、指标与集群节点的三

维数据分析新范式，以期提升异常检测在真实动态运维场景中的鲁棒性。

1.3 论文研究内容和贡献

1.3.1 主要研究内容

本文围绕分布式集群多节点场景下的时序异常检测问题，开展了以下三个方面的工作。

第一，面向单指标多节点场景的异常检测算法研究。本文以 YARN 集群的 RPC Router 连接数指标为研究对象，构建了基于联通大数据生产环境的单指标多节点时序数据集。在此基础上，分别采用欧氏距离、自编码器嵌入距离和密度偏离三种方法度量节点曲线间的相似性，验证了基于群体一致性思想的异常检测方法的有效性。针对单一方法阈值高度敏感的问题，提出了 I-SPOT (Inclusive SPOT) 包容性自适应阈值算法，通过将所有数据点纳入模型更新池解决经典 SPOT 在概念漂移场景下的阈值衰减问题。在此基础上，将多视角相似性评分、分数归一化聚合与 I-SPOT 自适应阈值检测整合为统一的 CSAD-AT 算法，实现端到端的自适应异常检测。

第二，面向多指标多节点场景的异常检测算法研究。本文使用真实 GPU 集群故障数据集，针对多指标场景下数据规模增长带来的算法效率约束以及异常表现异质性导致的检测盲区问题，设计了 NSFusion 数值与形状双视角融合的异常检测框架。该框架放弃了单指标场景中计算开销较大的自编码器和密度偏离方法，在数值视角方面采用优化的基于欧氏距离的多指标加权融合方法量化节点间的数值差异，在形状视角方面提出 FR-SAX (Full-Resolution SAX) 符号化表示算法作为自编码器的轻量化替代方案，通过取消分段聚合保留原始分辨率并采用严格距离计算规则来捕捉曲线形态差异。最终通过加权融合两种视角的检测结果，实现对不同类型异常的全面覆盖。

第三，集群节点异常检测与监控系统的设计与实现。本文设计了集群节点异常检测与监控系统，采用前后端分离的四层架构，实现了从数据采集、预处理、异常检测到结果展示的完整流程。系统支持文件上传和 Prometheus API 两种数据接入方式，通过 APScheduler 定时任务框架实现对多个集群的常态化自动巡检，并构建了三级告警机制支持邮件和企业微信等多渠道通知。系统已具备对接联通大数据生产底座平台 Prometheus 监控体系的接口能力，并通过历史数据回放验证了功能完整性。

1.3.2 主要贡献

本文的主要贡献包括以下三个方面。

第一，本文提出了一种基于多节点曲线相似性的分布式集群异常检测方法。该方法突破了传统异常检测聚焦于时间与指标二维分析的框架，创新性地将集群节点这一空间维度系统性地引入模型，构建了时间、指标与节点的三维分析范

式。通过利用分布式集群负载均衡机制所保证的节点行为相似性，将隐性的运维经验转化为显式的可计算先验知识。

第二，本文设计了兼顾计算效率与检测覆盖面的数值与形状双视角融合多指标异常检测框架。针对多指标场景下数据规模成倍增长的效率约束，框架采用轻量化的数值距离计算与无需训练的符号化形状度量相结合的方案；针对不同指标间异常表现异质性导致的单一视角检测盲区，框架从数值幅度和曲线形态两个互补角度刻画节点异常行为，其中 FR-SAX (Full-Resolution SAX) 算法通过保留原始分辨率和采用严格距离计算规则增强了对形状变化的敏感性。通过加权融合两种视角的检测结果，该框架能够有效识别不同类型的节点异常。

第三，本文基于联通大数据真实生产环境构建了单指标多节点时序数据集，并使用某国内大模型公司万卡 GPU 集群故障数据集进行多指标多节点场景的验证。在这些数据集上的实验结果表明，本文所提方法在检测准确率方面显著优于现有的多元时序异常检测算法。此外，本文设计并实现了完整的异常检测系统，支持常态化自动巡检和多级告警，具备对接实际生产环境的接口能力，为分布式集群异常检测提供了从算法到系统的完整解决方案。

1.4 论文组织结构

本文共分为六章，研究内容框架如图1-1所示。各章内容安排如下。

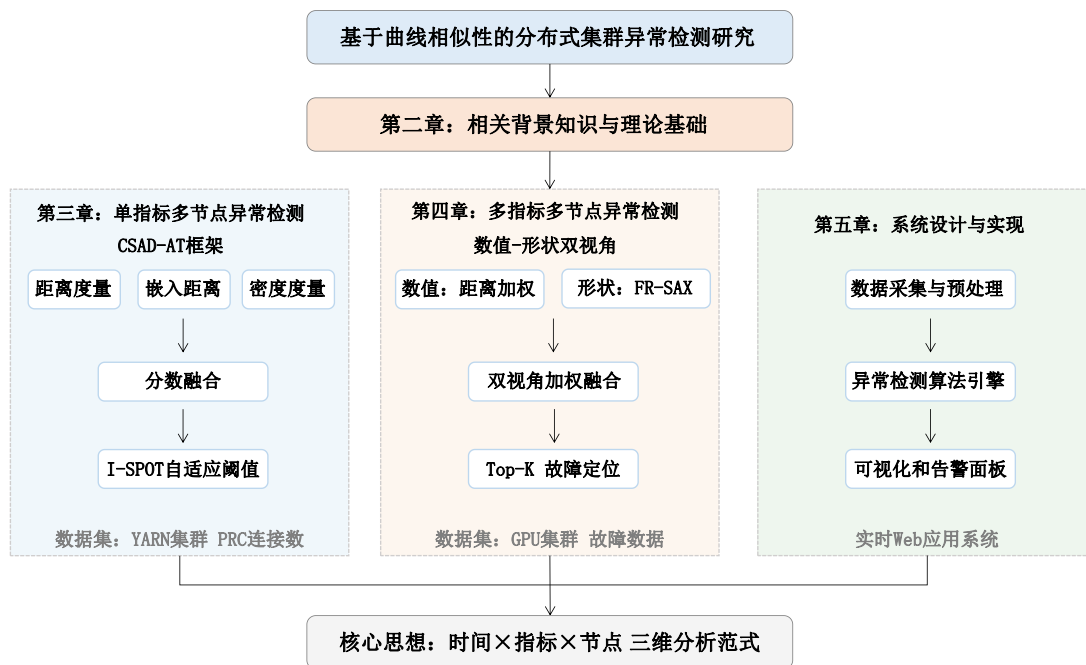


图 1-1 论文研究内容框架图

Figure 1-1 Research Content Framework of This Thesis

第一章绪论部分阐述了本研究的背景与意义，回顾了多元时间序列异常检测领域的国内外研究现状，概述了本文的主要研究内容与贡献。

第二章介绍了相关背景知识与理论基础，包括时间序列异常的定义与分类、主流的时序异常检测算法，以及分布式集群监控数据的特点与挑战。各章实验所采用的评估指标在对应的实验设置部分进行介绍。

第三章针对单指标多节点场景，详细介绍了数据集的构建过程、基于曲线相似性的三种异常检测方法和参数敏感性分析，以及 I-SPOT 包容性自适应阈值算法和 CSAD-AT 算法的设计。

第四章针对多指标多节点场景，以真实 GPU 集群故障数据集为基础，提出了数值与形状双视角融合的异常检测框架，详细阐述了基于欧氏距离的数值异常检测算法和基于 FR-SAX 的形状异常检测算法，并介绍了多算法融合与故障定位方法。

第五章介绍了集群节点异常检测与监控系统，包括系统的总体设计、架构与关键模块、实现细节及功能验证。

第六章对全文工作进行总结，并对未来研究方向进行展望。

第2章 相关背景知识与理论基础

本章系统介绍时间序列异常检测的相关背景知识与理论基础，涵盖时间序列异常的基本概念、数据特征与分类体系，从传统统计方法到深度学习方法的算法演进，以及分布式集群的架构特点、监控数据特性与异常检测面临的挑战，为后续章节的算法设计奠定理论基础。

2.1 时间序列异常概述

时间序列数据作为记录系统、过程或现象随时间演变信息的重要载体，广泛存在于工业监控、金融交易、网络安全及物联网等众多领域。对时间序列中的异常进行准确、及时的检测，是感知系统状态、预测潜在故障、保障业务稳定的关键技术。本节将系统阐述时间序列异常检测的定义、分类等核心概念。

2.1.1 时间序列异常数据简介

时间序列数据是按照时间顺序排列的一系列观测值的集合，其数学形式可表示为 $X = \{x_1, x_2, \dots, x_T\}$ ，其中 x_t 表示在时刻 t 的观测值， T 为序列长度。在单变量时间序列中，每个时刻的观测值为标量；在多元时间序列中，每个时刻的观测值为向量 $\mathbf{x}_t \in \mathbb{R}^D$ ，其中 D 表示变量维度（对于单变量序列， $D = 1$ ）。时间序列异常检测的目标是学习一个异常评分函数 $f(\cdot)$ ，并基于分数 $f(x_t)$ 或 $f(X_{(i,j)})$ （对于子序列）是否超过阈值 δ 来判断异常。

时间序列数据具有若干典型特征。**时序相关性**是其最基本的属性，即相邻时刻的观测值之间往往存在较强的相关性，这种相关性可以是短期的自相关，也可以是长期的依赖关系。此外，许多时间序列数据呈现出**周期性和季节性**波动的规律，例如网络流量的日周期和周周期变化。在更长的时间尺度上，时间序列可能呈现**趋势性**，即长期的上升或下降走势。与此同时，时间序列还普遍具有**非平稳性**，其统计特性可能随时间发生变化，表现为均值、方差或相关结构的漂移。

多变量时间序列具有多维观察、时间依赖性、交叉依赖性、非平稳性以及可能包含不规则取样和缺失数据等关键特性。其异常检测需要同时分析时间维度和变量维度，建模变量间复杂的时空依赖关系，这比单变量场景更具挑战性。

在分布式集群监控场景中，时间序列数据通常以固定采样间隔连续采集，涵盖 CPU 使用率、内存占用、网络流量、磁盘读写等多个维度。这些监控指标共同反映了系统的运行状态，为故障诊断和性能优化提供了重要的数据基础。

2.1.2 时间序列异常定义与分类

时间序列中的异常是指与数据的正常行为模式存在显著偏离的观测点或观测片段，通常定义为明显偏离大多数样本的数据点。根据异常的时空范围和表现

形式，可将时间序列异常划分为以下几类。

第一类是点异常 (Point Anomaly)，指时间序列中单个数据点的突变，表现为某个数据点显著偏离全局或局部正常数据点的分布。点异常可进一步分为全局点异常和局部点异常两个子类。全局点异常不依赖于局部上下文，其数值显著偏离整体数据分布，例如某项监控指标突然飙升至远超历史最高水平的极端值。局部点异常则是在相邻时间点的局部上下文中，某个观测值偏离其周围数据所呈现的局部模式，例如在夜间低峰期出现的中等流量峰值，虽然其绝对数值并未超出全局范围，但在局部时间窗口内构成了显著的偏离。在实际系统监控中，点异常往往与瞬时故障、网络抖动或传感器失灵等事件相关联。如图2-1所示，红色阴影区域标注了曲线中的突变尖峰，即典型的点异常表现。

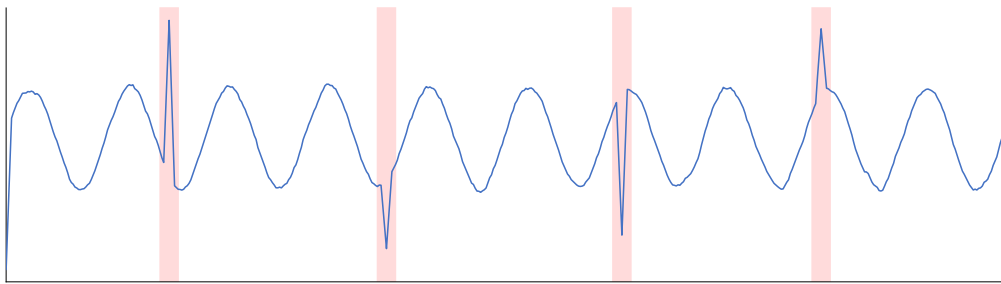


图 2-1 点异常示意图

Figure 2-1 Illustration of Point Anomaly

第二类是片段异常 (Subsequence Anomaly)，也称为模式异常，指时间序列中一段连续数据点的行为显著偏离正常模式。与点异常不同，片段异常中的单个数据点可能并不构成异常，但整个子序列的模式与正常行为存在显著差异。根据异常模式的不同，片段异常可以细分为三种子类型：波形异常表现为振荡频率或幅度的突然改变，例如原本平稳的指标突然出现剧烈的高频震荡；季节性异常表现为周期性规律被打破，原有的周期结构在异常段内消失或发生显著变化；趋势异常则表现为序列的整体水平发生突变式的偏移，且偏移后的新水平在后续时间内持续保持。在分布式系统中，这类异常通常反映了较为持续的系统状态变化，如资源泄漏、配置错误或负载模式的异常转变。如图2-2所示，三个子图从上到下分别展示了波形异常、季节性异常和趋势异常的典型表现。

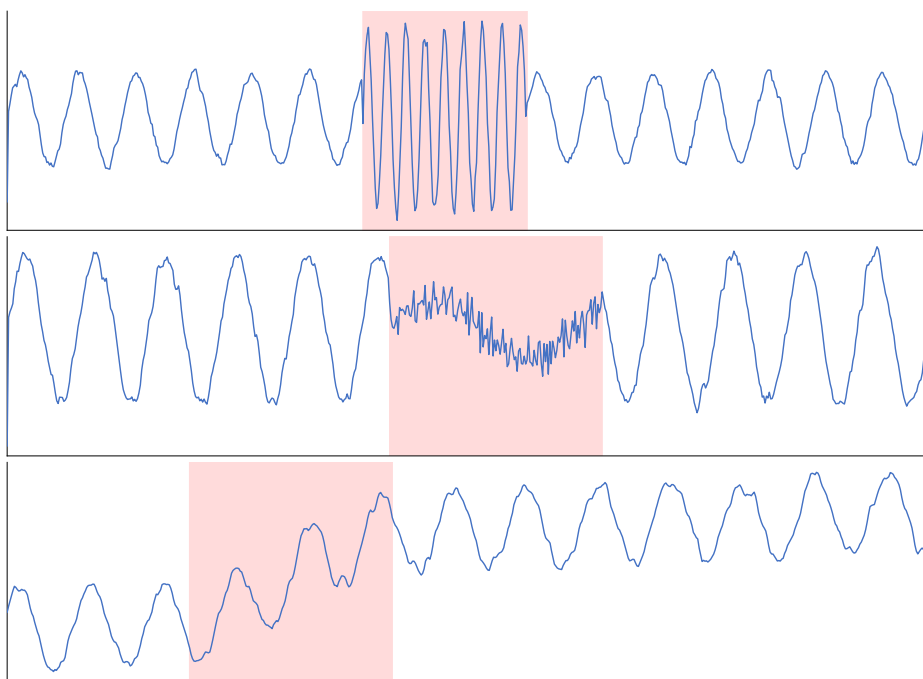


图 2-2 片段异常示意图

Figure 2-2 Illustration of Subsequence Anomaly

第三类是变量间关联异常 (Inter-variable Correlation Anomaly)，这是多变量时间序列特有的异常类型，指变量间的关联关系随时间发生异常变化。在这类异常中，单个变量的数值可能均处于正常范围内，既不表现为点异常也不表现为片段异常，但变量之间原本稳定的关联模式（如正相关性、因果依赖）发生了显著偏离。例如，在正常运行状态下 CPU 使用率与网络吞吐量保持同步变化，而在异常状态下二者的关联关系可能反转或解耦。检测此类异常的核心难点在于需要同时建模时间维度上的演化规律和变量维度上的相互依赖关系，单纯基于单变量分析的方法难以发现这类隐蔽的异常。如图2-3所示，两条曲线在正常区域保持同步波动，而在红色阴影区域内关联关系发生明显反转。

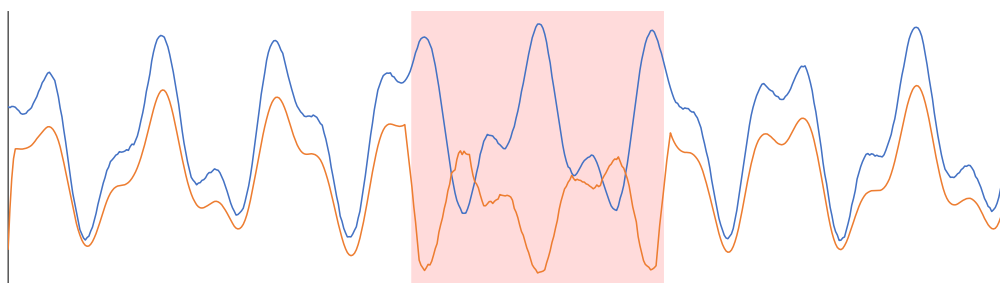


图 2-3 变量间关联异常示意图

Figure 2-3 Illustration of Inter-variable Correlation Anomaly

在分布式集群场景中，本文重点关注的是节点级异常 (Node-level Anomaly)，即在多节点环境下，个别节点的监控曲线偏离集群整体行为模式的情形。在负载

均衡机制的作用下，分布式集群中各节点承担的计算负载相近，其监控指标曲线通常表现出高度的形态相似性。当某个节点出现硬件退化、软件缺陷或配置偏差等问题时，其监控曲线会逐渐偏离正常节点群的行为模式，呈现出与群体不一致的演变趋势。这类异常的识别需要同时考虑时间维度的变化规律和节点维度的群体一致性，其本质是利用多节点之间的行为相似性作为正常基准，通过对比分析识别离群的异常节点。如图2-4所示，多条蓝色曲线表示正常节点群的行为模式，红色曲线为异常节点，其在红色阴影区域内逐渐偏离正常节点群。

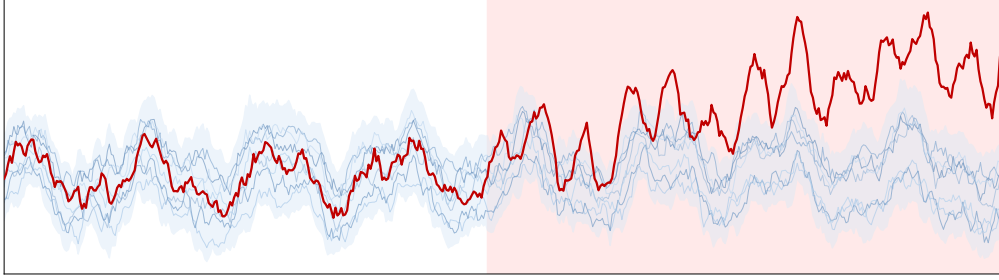


图 2-4 节点级异常示意图

Figure 2-4 Illustration of Node-level Anomaly

2.2 时序异常检测算法

时间序列异常检测方法可以从多个维度进行分类，主要包括学习范式、技术原理和数据维度。从学习范式角度，监督模式的选择取决于是否有标注数据：无监督方法（如孤立森林^[11]）无需任何标注数据，适用于新系统上线初期；半监督方法（如OCSVM、自编码器）仅需正常样本标注，在工业场景中最为常用；有监督方法则需要正常与异常样本均有标注，适用于已知特定异常模式的场景。

2.2.1 单变量时序异常检测

单变量时序异常检测旨在识别单条时间序列中的异常点或异常片段，其方法体系经历了从传统统计方法到经典机器学习方法再到深度学习方法的演进过程^[41-44]。早期方法建立在概率分布假设和时序模型之上，随着数据规模和模式复杂度的增长，学习型方法逐渐成为主流。

传统统计方法在数据量适中、模式相对稳定时具有良好的理论基础和可解释性。基于分布假设的检验方法中，3-sigma 准则假设数据服从正态分布，将落在 $(\mu - 3\sigma, \mu + 3\sigma)$ 区间之外的点视为异常；Grubbs 检验和 ESD (Extreme Studentized Deviate test) 则是专门为单变量数据设计的异常值统计检验方法^[44]。基于时序模型的预测方法以 ARIMA（自回归综合移动平均）模型^[4] 为代表，该方法通过差分将非平稳序列转化为平稳序列，再结合自回归（AR）和移动平均（MA）成分进行建模，将预测值与实际观测值的偏差超过设定阈值的点判定为异常。此外，分解方法如 STL (Seasonal-Trend decomposition using Loess)^[5] 将时间序列分解为季节性、趋势性和残差三个部分，异常通常表现为残差分量中的极端值。这

类传统方法原理简单、计算效率高、可解释性强，但其依赖于时序数据服从特定统计分布的假设，在面对复杂非线性模式时效果有限。

经典机器学习方法通过设计或选择特征度量，以无监督方式识别偏离正常模式的样本，不依赖于严格的分布假设。基于距离的方法如 KNN (K 近邻) 通过计算数据点与其 K 个最近邻居的距离来识别异常。Breunig 等人^[10]提出的 LOF (局部离群因子) 算法从局部密度角度出发，计算每个点相对于邻居的稀疏程度，值远大于 1 通常表示异常，该方法善于处理局部密度变化大的场景。Liu 等人^[11]提出的孤立森林 (Isolation Forest) 因其高效性在工业界应用广泛，其核心思想是异常点“少且不同”，更容易在随机分割构成的树结构中被快速孤立出来，算法通过构建多棵隔离树并基于样本的平均路径长度计算异常分数。Schölkopf 等人^[45]提出的单类支持向量机 (One-Class SVM) 通过在特征空间中找到一个超平面来将正常数据与原点分离，偏离该边界的样本被判定为异常。此外，基于聚类的方法 (如 DBSCAN^[46]、高斯混合模型等) 将数据点分组，远离聚类中心或属于极小簇的点被视为异常。在保形预测框架方面，Ishimtsev 等人^[47]提出了基于保形预测的 k-NN 异常检测器，通过保形理论为异常判断提供统计显著性保证，并支持在线增量处理，适用于流式数据的实时异常检测场景。Jin 等人^[48]提出了一类支持向量机的时间序列变点检测校准方法，在标注样本稀缺的场景下展现了良好的检测效果。这些经典方法的共同优势在于无需大量标注数据且计算效率相对较高，但通常难以捕捉时间序列中复杂的非线性关系和长期时序依赖^[49]。

近年来，深度学习方法凭借其强大的表征学习能力，在单变量时序异常检测领域取得了显著进展^[17,18,50]。Hochreiter 等人^[21]提出的 LSTM (长短期记忆网络) 通过门控机制有效缓解了传统 RNN 的梯度消失问题，能够捕捉序列中的长期依赖关系；Chung 等人^[51]提出的 GRU (门控循环单元) 则通过简化门控结构在保持相近性能的同时降低了计算开销，两者均是时序建模中广泛使用的循环神经网络架构。Malhotra 等人^[52]将 LSTM 应用于时序异常检测，使用历史正常数据训练模型进行多步预测，将显著的预测误差作为异常判据。自编码器 (AutoEncoder) 类方法通过编码器将输入压缩为低维潜在表示，再由解码器进行重构，当异常数据输入时会产生较大的重构误差，从而被识别。Zong 等人^[53]提出的 DAGMM (深度自编码高斯混合模型) 将自编码器的压缩表示与重构误差特征联合输入高斯混合模型进行端到端训练，有效避免了两阶段方法中特征提取与密度估计脱节的问题。Kingma 等人^[54]提出的变分自编码器 (VAE) 进一步引入了对数据潜在分布的概率建模，Xu 等人^[55]在此基础上提出的 Donut 模型将 VAE 应用于周期性 KPI 的无监督异常检测，取得了良好效果。此外，基于 GAN (生成对抗网络) 的方法也被引入单变量时序异常检测，如 BeatGAN^[56]通过对抗训练学习正常时序模式的生成模型，TadGAN^[57]结合了重构误差和评论者分数进行异常评分。Zhang 等人^[58]提出的 TFMAE (时频掩码自编码器) 是应对训练数据含异常导致模型学习偏差这一挑战的创新方法，其核心是采用基于窗口的时间掩码和基于幅值的频率掩码两种策略，有选择地预消除输入中的潜在异

常,通过最小化时间掩码表示与频率掩码表示之间的对比差异实现异常检测,对分布偏移更具鲁棒性。Wu 等人^[59]提出的 TimesNet 将一维时间序列变换为二维张量以捕捉周期内和周期间的二维变化模式,通过自适应发现多个周期并利用二维卷积提取复杂的时序特征,在包括异常检测在内的多项时间序列分析任务中表现优异。在实际部署方面,Yu 等人^[60]提出的 AutoKAD 针对 KPI 异常检测中标签缺失导致的模型选择困难问题,设计了基于局部离群因子的无标签通用目标函数,实现了无需人工标注即可自动选择最优检测模型的能力。在模型架构方面,Zhou 等人^[61]提出的 KAN-AD 将 Kolmogorov-Arnold 网络引入时序异常检测,以截断傅里叶展开替代传统 B 样条基函数,结合轻量化学习机制强调全局模式,在多个基准数据集上取得了优异表现。深度学习方法的主要优势在于能自动捕获非线性与复杂依赖,但其局限性包括对数据量和计算资源需求高、超参数调优复杂以及模型决策可解释性不足^[44]。

2.2.2 多元时序异常检测

多元时序异常检测 (Multivariate Time Series Anomaly Detection, MTSAD) 旨在识别多个相关变量构成的时序数据中的异常模式。相较于单变量时间序列,多变量数据不仅包含每个变量自身随时间变化的模式,还蕴含了复杂的变量间关联关系,这既是其检测能力提升的潜力所在,也构成了核心挑战^[44]。当前多元时序异常检测方法主要可从重构、预测、图结构建模、因果分析和生成对抗等技术路线展开。

基于重构的方法是多元时序异常检测中最具代表性的范式之一。其核心思想是在正常数据上训练模型学习数据的低维表示,当异常数据输入时,由于其模式未被充分学习,会产生较大的重构误差。Su 等人^[19]提出的 OmniAnomaly 采用随机变分自编码器和平面归一化流来学习多元时间序列的正常模式分布,通过重构概率识别异常,能够捕捉时间依赖性并建模正常数据的复杂分布。Audibert 等人^[20]提出的 USAD 通过引入对抗性训练来增强自编码器的鲁棒性,从而提升重构误差对异常的敏感性。Malhotra 等人^[62]则提出了基于 LSTM 的编码器-解码器框架,利用 LSTM 的序列建模能力对多传感器数据进行重构,在多个工业数据集上展现了良好的异常检测性能。Chen 等人^[63]将自编码器应用于网络流量异常检测,通过学习正常流量的低维表示并利用重构误差识别异常网络行为,验证了自编码器在网络安全领域的有效性。Park 等人^[64]提出了基于 LSTM 变分自编码器的多模态异常检测器,结合 LSTM 的时序建模能力和 VAE 的生成建模能力,通过联合建模多种传感器信号的时间依赖关系和概率分布实现异常检测。

基于预测的方法通过学习时间序列的时序演化规律,预测未来时刻的数值,将预测误差作为异常评分的依据。Tuli 等人^[22]提出的 TranAD 是一种基于深度 Transformer 的异常检测与诊断模型,它采用两阶段对抗训练机制:第一阶段生成输入窗口的近似重构,其重建损失作为第二阶段的关注分数;第二阶段利用此分数重新运行推理以放大重建误差,这种自调节机制有助于捕获鲁棒的多模态特征。Xu 等人^[23]提出的 Anomaly Transformer 通过关联差异性机制捕捉异常的

时序特征, 利用 Transformer 架构的强大自注意力机制捕捉时间序列长期依赖关系。Chen 等人^[65]提出的 DCdetector 采用双重注意力对比表示学习, 通过构建频域和时域的双通道注意力机制, 增强了对细粒度异常模式的捕捉能力。Zhao 等人^[66]提出的 MTAD-GAT 结合了图注意力网络^[67]和时间卷积网络, 同时建模变量关联和时间依赖, 是较早将图注意力引入多元时序异常检测的工作之一。

图神经网络 (GNN)^[68,69]天然适合对多变量时间序列中变量间的复杂关系进行显式建模。通过将变量视为图中的节点、关系视为边, GNN 可以学习节点和边的表示, 以识别异常节点或子图。Deng 等人^[24]提出的 GDN 通过学习传感器之间的图结构来检测异常, 利用图神经网络传播信息并预测未来值, 能够捕捉变量之间的复杂依赖关系。Dai 等人^[70]提出将图结构与归一化流相结合, 通过图增强的归一化流模型对多变量时间序列的联合分布进行建模, 为异常检测提供了概率论视角的解决方案。Zhang 等人^[71]提出的 GST-Pro 针对现实中常存在缺失值的非规则采样多变量时间序列数据, 基于神经控制微分方程对图时空过程进行建模, 能够从空间和时间两个视角有效处理含缺失值的数据, 并采用基于分布的异常评分机制。Liu 等人^[72]提出的 MGCL 专注于捕捉变量间的高阶交互关系, 结合多尺度时序建模和自适应超图结构, 以有效学习复杂的变量间和时序依赖关系。

从因果视角审视异常检测问题, 将异常视为偏离了数据生成常规因果机制的实例, 为提升检测的鲁棒性和可解释性提供了新思路。这类方法的核心创新在于将因果结构学习融入检测框架, 利用因果系统的模块化特性, 将复杂的多变量异常检测问题分解为一系列更简单的低维子问题, 从而直接定位异常根源。Chen 等人^[25]提出的 CAROTS 创新性地将对因果关系概念融入对比学习框架, 设计了两种数据增强器: 一种保留原始数据中的因果结构以生成代表正常波动的正样本, 另一种则有意扰动因果关系以生成模拟异常行为的负样本, 通过基于因果关系的对比学习训练编码器在潜在空间中有效区分正常与异常样本。Chen 等人^[73]提出的 CGAD 利用转移熵构建变量间的因果图结构, 并结合加权图卷积网络和因果卷积来同时捕捉因果与时间模式。

基于生成对抗网络 (GAN)^[74]的方法利用生成器和判别器的对抗训练来学习正常数据分布。Li 等人^[75]提出的 MAD-GAN 将 LSTM 与 GAN 相结合, 通过判别器得分和重构误差共同识别异常, 是较早将 GAN 应用于多元时序异常检测的代表性工作。Ren 等人^[76]在微软提出了面向大规模服务的时序异常检测方案, 将谱残差方法与深度神经网络相结合, 在实际生产环境中取得了良好效果。此外, Cañizo 等人^[77]提出了多头 CNN-RNN 架构, 通过为每个变量设置独立的卷积头提取局部特征, 再经由共享的循环网络融合时序信息, 在工业多时间序列异常检测中展现了良好的可扩展性。

近年来, 将频域分析与时域分析相结合成为提升检测能力的重要方向。Wang 等人^[78]提出了频率增强条件变分自编码器 (FCVAE), 从频域视角重新审视 VAE 在无监督时序异常检测中的应用, 通过引入频域增强机制有效捕捉不同频率成

分中的异常特征。Zhang 等人^[79]提出的 TFAD 架构将时频分析与时间序列分解相结合, 同时进行时域分解和频域分析, 通过融合机制整合多视角的检测结果, 在捕捉趋势性异常和周期性异常方面展现了优越的性能。

在面向运维场景的应用方面, He 等人^[80]从系统运维角度出发, 提出了一种基于日志分析的异常检测方法, 通过解析和结构化系统日志数据并利用机器学习模型识别日志中的异常模式, 为分布式系统的故障诊断提供了有效手段。Ma 等人^[81]针对在线服务系统中频繁变更导致异常检测算法需要快速初始化的问题, 提出了 JumpStarter 算法, 结合形状聚类策略和抗异常值采样显著缩短了冷启动时间, 有效应对了概念漂移场景下的检测挑战。

上述方法虽然在多元时序异常检测方面取得了显著进展, 但主要关注时间维度和指标维度, 未能充分利用分布式集群中多节点之间的行为相似性。本文的研究正是旨在弥补这一不足, 将节点维度纳入异常检测的分析框架。

2.3 分布式集群监控数据与异常检测

2.3.1 分布式集群概述

分布式集群是融合了分布式系统与集群系统优势的计算架构, 通过将任务分配到多个互相协作的服务器节点上, 实现大规模数据的并行存储与处理。典型的分布式集群架构包括大数据领域的 Hadoop 集群和容器编排领域的 Kubernetes 集群等。作为支撑现代云计算与大数据处理的核心基础设施, 分布式集群广泛应用于数据仓库、实时计算、机器学习训练等场景。此类集群的稳定运行直接关系到上层业务的连续性, 因此对集群的监控和异常检测具有重要的实际意义。

以 Hadoop 分布式集群为例, 其采用典型的主从 (Master/Slave) 架构, 如图2-5所示。在存储层, Hadoop 分布式文件系统 (HDFS) 由一个 NameNode 主节点和多个 DataNode 工作节点组成。NameNode 负责维护文件系统的元数据, 如文件目录的布局和数据块的分布情况; DataNode 处理数据块的物理存储, 并通过多副本复制机制保证数据的可靠性。Secondary NameNode 定期对 NameNode 的元数据进行检查点备份, 以防止单点故障导致的数据丢失。在资源管理层, YARN (Yet Another Resource Negotiator) 由 ResourceManager 和多个 NodeManager 组成。ResourceManager 负责整个集群的资源调度和管理, NodeManager 运行在各个计算节点上, 负责管理节点的资源并执行容器化的计算任务。当用户通过客户端提交计算任务时, ResourceManager 根据资源需求和调度策略将 MapTask 或 ReduceTask 分配到合适的节点上执行。为保证系统的可扩展性和容错性, Hadoop 集群通常采用负载均衡机制, 使得计算任务能够均匀分布到各个节点上。

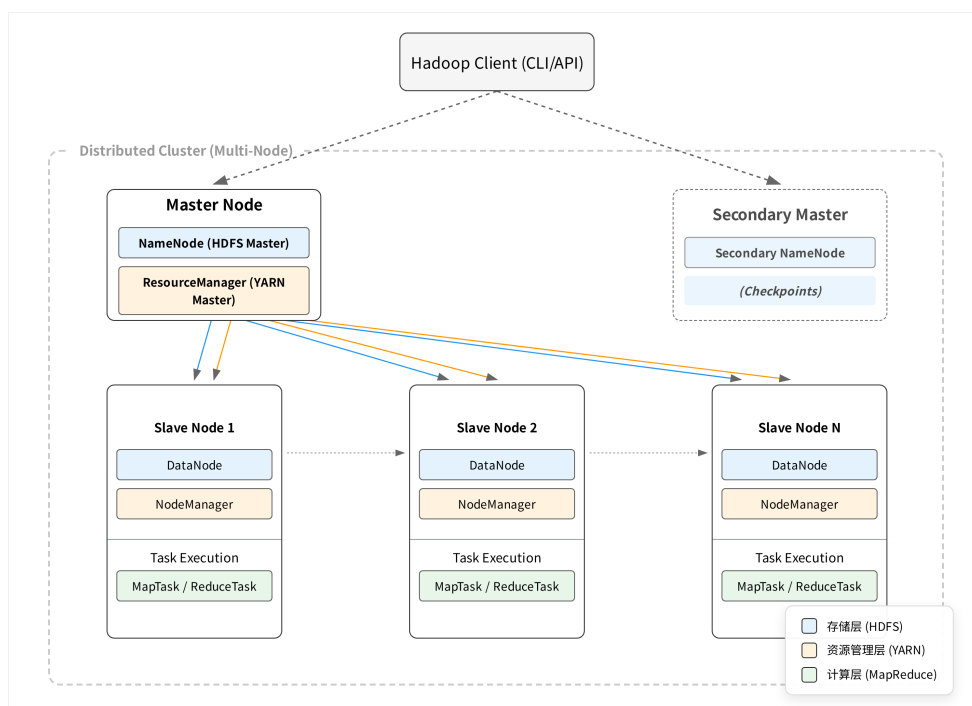


图 2-5 Hadoop 分布式集群架构示意图

Figure 2-5 Architecture of Hadoop Distributed Cluster

Hadoop 集群的上述架构特征体现了分布式集群的核心设计理念：通过将存储与计算分布到多个节点上实现横向扩展能力，同时依靠副本机制和主从协调保障系统的可靠性。这些特征也决定了集群监控数据所呈现的独特模式，为后续异常检测方法的设计提供了重要背景。

2.3.2 集群监控数据特点

分布式集群在运行过程中会产生海量的多维度监控时序数据，这些数据具有若干显著特点。

多维度性是集群监控数据最突出的特点。集群监控涵盖多个层次和维度，包括系统级指标如 CPU 使用率和内存占用，应用级指标如请求响应时间和连接数，以及集群级指标如任务队列长度和资源使用率等，这些指标从不同角度反映系统的健康状态。

高频采样性则体现在数据的时间粒度上。为及时发现异常，监控系统通常以较高频率采集数据，常见的采样间隔为数秒到数分钟，由此产生的数据量巨大，对存储和处理提出了较高要求。

在空间维度上，集群数据具有显著的多节点相关性。由于负载均衡机制的作用，各节点在正常运行时承担的负载相近，其监控指标曲线往往表现出高度的形态相似性，这种相似性为基于群体一致性的异常检测提供了天然的基础。

另一方面，集群数据的动态波动性也不容忽视。与用户负载相关的指标具有正常模式难以统一定义的特点，其数值会随系统整体负载发生无规律波动，这种

动态性增加了异常检测的难度，使得基于固定阈值或历史模式学习的方法容易产生误报。

2.3.3 分布式异常检测系统架构

随着数据规模的爆炸式增长，分布式集群环境下的时间序列异常检测成为必然趋势。传统的单机异常检测方法在处理大规模、高维度、实时性要求严格的场景时面临严重挑战，包括内存限制、计算瓶颈和可扩展性问题。分布式集群架构通过将计算任务和数据分布到多个节点上，实现了大规模时间序列异常检测的高效处理。

分布式多变量时间序列异常检测系统的架构设计主要遵循三种核心范式：

基于 Spark 的批处理系统架构采用弹性分布式数据集（RDD）作为核心数据抽象，通过内存计算优化大规模时间序列数据的离线分析。Spark 框架的优势在于其强大的内存计算能力和容错机制，通过 DAG 调度器优化计算流程，支持迭代算法的高效执行^[33]。

基于流处理的系统架构（如 Apache Flink）专为实时时间序列异常检测设计，采用数据流作为核心抽象，支持有状态计算和事件时间处理。与基于批处理的 Spark 相比，流处理系统在低延迟流处理方面具有优势，能够满足对实时性要求极高的应用场景。

专门设计的分布式系统则针对特定算法或问题进行深度优化。例如，DADS（分布式异常检测系统）^[82] 基于 actor 编程模型，将原始 Series2Graph 算法重构为反应式协议，通过异步消息传递和分布式计算实现了对单变量时间序列中序列异常的检测。

在数据分区策略方面，时间序列数据分区主要分为时间维度分区和变量维度分区两种模式。时间维度分区将长序列切分为等长的子序列片段，分配给不同计算节点处理。多变量数据的分区策略更为复杂，需要考虑变量间的相关性，基于变量聚类的分区策略有助于在并行计算时保持变量间的语义关联。

2.3.4 异常检测方法在集群数据中的应用与挑战

将现有时间序列异常检测方法应用于分布式集群监控数据时，面临来自数据特性、算法适配和应用需求等多个层面的挑战^[44]。本节从与本文研究密切相关的四个方面进行分析。

第一，正常基线缺失与概念漂移。传统异常检测方法通常假设存在可明确刻画的正常模式基线，然而分布式集群中与用户负载相关的监控指标（如 CPU 使用率、网络流量）其正常范围随集群整体负载动态变化，难以预先定义固定的正常区间。更重要的是，集群的运行模式会随业务迭代、版本升级、扩缩容等运维操作持续演变，即数据的生成分布随时间发生漂移^[44]。基于历史数据训练的检测模型容易将负载波动或正常的模式演变误判为异常，导致大量误报；同时，采样抖动和网络延迟引入的噪声进一步模糊了正常行为与异常行为之间的边界。

这一挑战的本质在于：缺乏一种不依赖于绝对正常基线定义、且能适应数据分布动态变化的检测范式。

第二，节点维度信息利用不足。如 2.2 节所述，现有多元时序异常检测方法主要从时间维度和指标维度对数据进行建模，而未能有效利用分布式集群中多节点之间的行为相似性。在负载均衡机制的作用下，集群中各节点承担的计算负载相近，其同类监控指标曲线通常表现出高度的形态一致性。这种节点间的行为相似性蕴含了丰富的正常模式先验知识，为构建“群体行为基准”提供了天然的参照系，但目前尚缺乏系统化的方法来挖掘和利用这一维度的信息。

第三，标签稀缺与阈值自适应。在集群运维实践中，异常事件本身发生频率低且标注成本高昂，可用于模型训练和评估的标注数据极为稀缺^[44]。这一现状限制了有监督方法的适用性，要求检测算法具备在无标注或弱标注条件下有效工作的能力。此外，不同集群、不同指标、不同时段下的最优检测阈值差异显著，静态阈值难以适应动态变化的运行环境，亟需自适应的阈值确定机制。

第四，大规模数据的实时处理需求。分布式集群持续产生海量高频采样的监控数据，单机处理能力已无法满足大规模时序数据的实时分析需求^[82]。这要求检测系统在算法层面具备低计算复杂度，在架构层面支持分布式并行处理，同时满足故障及时发现的实时性约束。

针对上述挑战，本文的核心思路是利用分布式集群多节点曲线的行为相似性构建异常检测方法。在负载均衡机制下，正常节点的监控曲线应保持较高的形态相似性，而异常节点的曲线会显著偏离群体行为模式。基于这一观察，通过对比节点间的曲线差异识别离群的异常节点，能够绕过正常基线定义困难的问题；同时，设计自适应阈值机制应对标签稀缺与阈值选择的挑战，并基于分布式计算框架实现大规模数据的高效处理。

2.4 本章小结

本章系统介绍了时间序列异常检测的相关背景知识与理论基础。在异常概念层面，阐述了时间序列异常数据的基本特征，并对异常类型进行了系统分类，包括点异常、片段异常、变量间关联异常和节点级异常。在算法层面，回顾了单变量和多元时序异常检测的主流方法，涵盖从传统统计方法、经典机器学习方法到深度学习方法的发展脉络，重点介绍了基于重构、预测、图神经网络和因果分析的多元时序异常检测方法。在应用层面，介绍了分布式集群的基本概念、监控数据特点和分布式异常检测系统架构，并从正常基线缺失与概念漂移、节点维度信息利用不足、标签稀缺与阈值自适应、大规模数据实时处理等方面分析了现有方法在集群场景中面临的关键挑战，明确了本文的研究切入点。本章内容为后续章节的算法设计奠定了理论基础。

第3章 CSAD-AT: 阈值鲁棒的单指标多节点异常检测算法

基于第二章揭示的多节点曲线相似性规律, 本章开展面向单指标多节点场景的异常检测方法研究。本章以联通大数据生产环境为背景构建高质量的单指标多节点时序数据集, 在此基础上系统评估基于曲线相似性的多种检测方法的性能, 并通过参数敏感性实验揭示阈值依赖性这一核心问题。针对该问题, 本章提出 I-SPOT 自适应阈值算法, 通过包容性数据池更新与周期性 GPD 重估机制消除人工调参需求, 进而将上述组件整合为统一的 CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) 算法框架, 融合多视角相似性评分、分数归一化聚合与自适应阈值检测, 实现端到端的自适应异常检测。

3.1 多节点单指标数据构建

本章研究的数据基础来源于真实的分布式集群生产环境, 旨在构建能够反映集群节点协同行为、并包含明确异常标注的数据集, 以支撑后续算法的训练、验证与对比分析。

3.1.1 数据来源与采集

本研究的数据来源于中国联通大数据生产底座平台。该平台采用业界标准的 YARN (Yet Another Resource Negotiator) 架构进行分布式计算资源的统一调度与管理, 其核心架构如图3-1所示。ResourceManager 作为集群的中央调度器, 通过 RPC Router 接收各 NodeManager 的心跳与资源请求, 并下发调度指令; 各 NodeManager 负责本地容器的生命周期管理, 通过 RPC Client 与 ResourceManager 保持持续通信; Prometheus 监控系统实时采集各节点的运行指标并持久化存储。

在长期运维实践中, 我们观察到与用户负载强相关的监控指标 (如 RPC 连接数) 呈现出独特的统计特性: 其正常波动模式难以通过先验知识统一定义, 且随全局负载呈现不可预测的剧烈变化, 导致传统基于绝对阈值或历史模式学习的方法面临高误报率。然而, 得益于负载均衡机制的作用, 正常运行状态下各节点同一指标的时序曲线呈现出高度的形态相似性 (如图3-1底部所示), 这一规律性为基于“群体行为一致性”的异常检测提供了理论依据。

基于上述洞察, 本研究选取 RPC Router 连接数作为核心研究指标。该指标直接反映各 NodeManager 与 ResourceManager 之间的通信负载与连接健康状态, 是衡量节点服务能力的关键信号。本研究通过 Prometheus API 接口, 采集了多个生产集群在连续运行周期内所有节点的该指标时序数据, 形成训练集 (覆盖 30 天连续运行数据) 和测试集 (覆盖 8 天连续运行数据), 原始数据采样间隔为 60 秒。

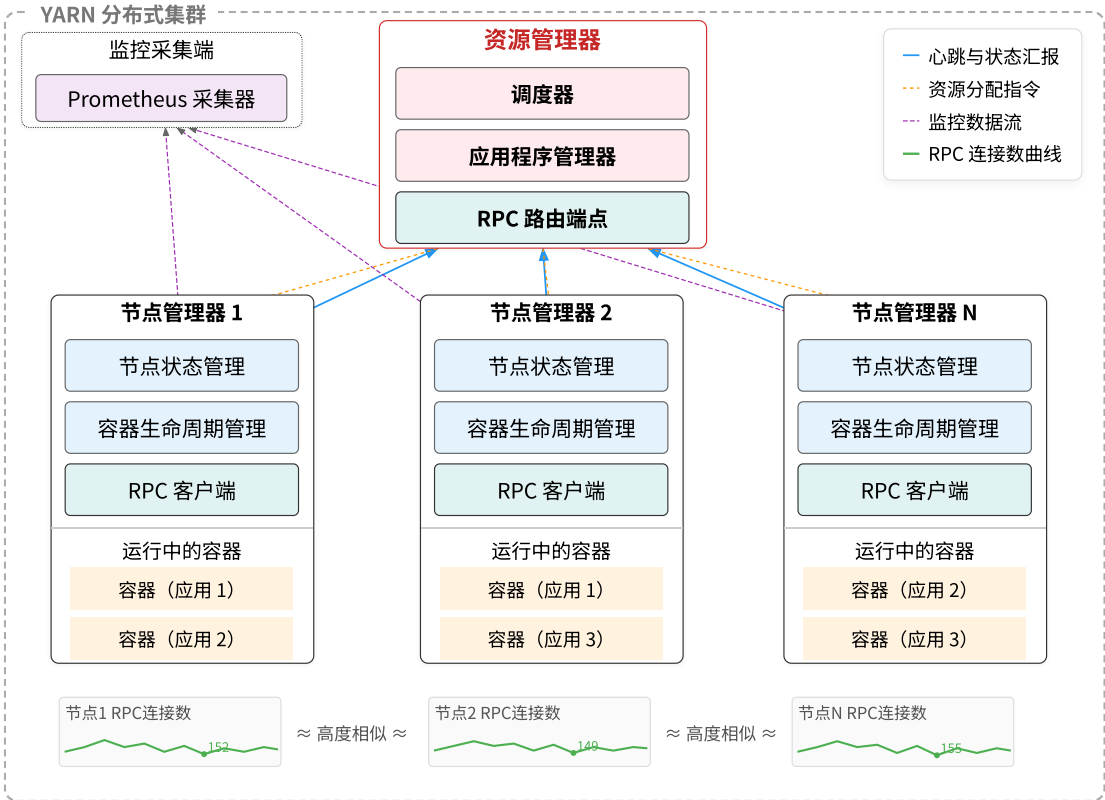


图 3-1 YARN 分布式集群架构与 RPC Router 连接数指标示意

Figure 3-1 YARN Cluster Architecture and RPC Router Connection Count

3.1.2 数据集预处理与标注

为构建适用于算法研究的高质量数据集，并对后续模型性能进行可靠评估，本研究对从生产环境采集的原始监控时序数据实施了系统化的预处理与半人工标注流程，整体流程如图3-2所示。

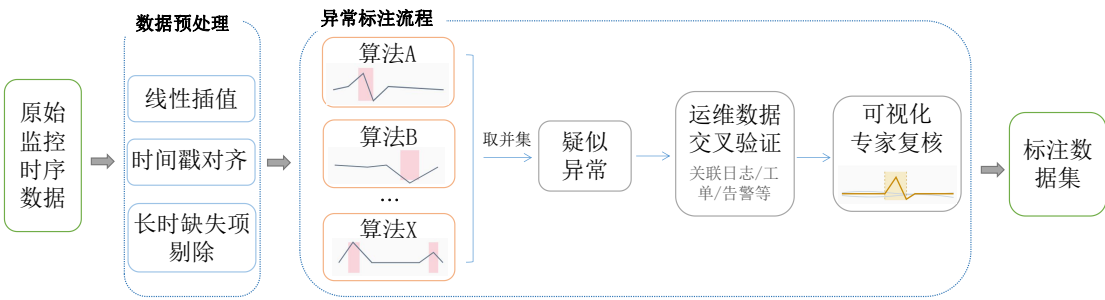


图 3-2 数据集预处理与半人工标注流程

Figure 3-2 Dataset Preprocessing and Semi-manual Annotation Workflow

在数据预处理阶段，本研究对原始监控时序数据执行三项操作：（1）线性插值：针对因网络延迟、采集器瞬时故障或系统抖动产生的短暂缺失值，采用线性插值法进行填充，确保时序数据的连续性；（2）时间戳对齐：将所有节点的数据点对齐至统一的 60 秒间隔时间戳，确保跨节点相似性比较在时间维度上严格对

齐；(3) 长时缺失项剔除：对于因监控代理长期离线、节点重启或下线导致的长时间段数据完全缺失，直接剔除对应的整条节点记录，避免插值引入较大误差。

在异常标注阶段，由于生产环境中的异常样本缺乏系统记录且缺乏自动化标注工具，本研究采用多算法初筛与专家判读相结合的半人工标注策略。

在多算法初筛阶段，利用多种基础的无监督离群点检测算法（如基于统计偏差、距离度量等）分别对全体时序数据进行扫描，将各算法的检测结果取并集，筛选出偏离自身历史基线或群体平均模式的“疑似异常”时间片段。这种多算法取并集的策略以宽松阈值最大化覆盖潜在异常，有效缩小了人工审查范围。

在运维数据交叉验证阶段，对初筛出的疑似异常片段关联查询该时段内对应节点的多源运维数据，包括系统日志、运维工单记录、告警信息及相关指标时序数据等，从多维度辅助判断其是否为真实的节点级故障或性能异常。

在可视化和专家复核阶段，由具备丰富分布式集群运维经验的专家，结合上一步的关联数据，通过可视化工具审查疑似片段及其前后上下文的多节点曲线对比图，做出最终的异常判定。通过以上三阶段流程，对所有的八个 YARN 集群两周的数据进行了筛选和标注，最终根据异常分布情况选取了两个集群 8 天的数据，构建了包含精确起止时间戳的异常片段集合作为测试集。

3.1.3 数据集统计与可视化

经过预处理与标注，本研究构建了一个适用于分布式集群节点级异常检测研究的单指标多节点时间序列数据集。该数据集的详细统计信息如表3-1所示。

表 3-1 数据集统计数据
Table 3-1 Dataset Statistics

	节点数	数据点数量	异常片段数	异常点数量	异常比例
训练集	30	86400	/	/	/
测试集 A	15	11520	72	901	7.82%
测试集 B	15	11520	33	907	7.87%
测试集	30	23040	105	1808	7.85%

本数据集的核心特征在于其直观揭示了分布式集群在负载均衡机制下的运行规律，如图3-3所示，其中红色曲线为异常节点，粉色阴影为标注的异常时段。在绝大部分时间内，集群内多数节点的监控指标曲线呈现出高度的形态同步性与数值聚集性，它们紧密缠绕，形成清晰的“正常行为簇”。这反映了在系统负载均衡策略下，各节点承担相近的工作负载，行为模式高度一致。然而，当个别节点发生资源竞争、进程僵死、网络隔离或本地硬件故障等问题时，其监控曲线会显著偏离这个共同的“行为簇”，表现为形态各异、幅度不一的“离群曲线”。这种“正常聚集，异常离群”的鲜明数据特征，是本研究所提出的、基于群体相似性进行异常检测方法的依据与直观体现。

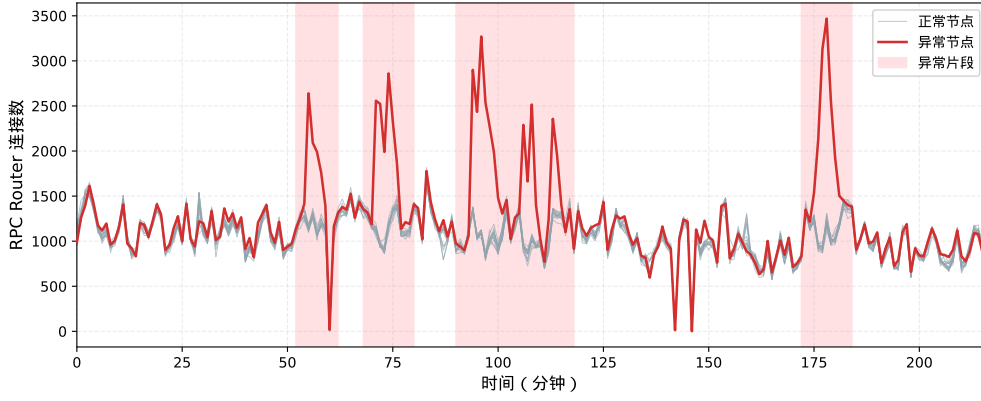


图 3-3 测试集 A 典型异常片段时序可视化

Figure 3-3 Visualization of Typical Anomaly Segments in Test Set A

3.2 基于时间和空间的异常检测算法评估

在提出本文的异常检测方法之前，本节首先系统评估现有的多元时序异常检测算法在本研究构建的分布式集群数据集上的适用性，以明确其局限性并为后续方法设计提供依据。

3.2.1 基线算法选择

为全面评估现有方法在分布式集群多节点场景下的适用性，本研究选取了六种具有代表性的先进异常检测算法作为基线，这些算法涵盖了基于重构、基于预测以及基于生成对抗网络等多种主流范式。

在基于重构的方法中，OmniAnomaly^[19] 采用随机变分自编码器和平面归一化流来学习多元时间序列的正常模式分布，通过重构概率来识别异常样本。USAD^[20] 则在自编码器架构的基础上引入对抗性训练机制，通过增强模型的鲁棒性来提升重构误差对异常的敏感性。

在基于预测的方法中，GDN^[24] 通过图神经网络显式建模变量间的拓扑关系，利用对未来值的预测偏差来检测异常。TranAD^[22] 基于 Transformer 架构构建深度异常检测模型，借助注意力机制捕捉序列中的长期依赖关系，并通过对抗训练增强对异常的敏感度。MTAD-GAT^[66] 结合了图注意力网络和时间卷积网络，能够同时建模变量间的关联关系和时间维度的依赖模式。

在基于生成对抗网络的方法中，MAD-GAN^[75] 采用 LSTM 构建生成器和判别器，通过对抗训练学习正常数据的分布特征，综合利用判别器得分和重构误差来识别异常。

实验设置遵循公平对比原则，所有基线模型均使用其作者开源的原版代码和默认推荐参数进行训练与测试。训练阶段使用3.1.1节所述的同构集群 30 天正常数据，测试阶段在本研究标注的两个测试集上进行评估并报告合并测试集的整体性能。评估指标采用异常检测领域通用的精确率、召回率和 F1 分数。

3.2.2 评估结果分析

本节在构建的单指标多节点 RPC 连接数数据集上对上述六种基线算法进行系统评估, 实验结果如表3-2所示。

表 3-2 基线算法异常检测结果

Table 3-2 Anomaly Detection Results of Baseline Algorithms

算法	Precision	Recall	F1
TranAD ^[22]	0.8889	0.3932	0.5452
USAD ^[20]	0.6892	0.5567	<u>0.6159</u>
OmniAnomaly ^[19]	0.6054	0.4593	0.5224
GDN ^[24]	0.3101	0.5218	0.3890
MAD_GAN ^[75]	0.6309	0.6309	0.6891
MTAD_GAT ^[66]	0.3492	0.1573	0.2169

从实验结果可以观察到, 现有的多元时序异常检测算法在本数据集上的表现普遍欠佳。具体而言, TranAD 虽然取得了较高的精确率 (0.8889), 但召回率仅为 0.3932, 表明该方法遗漏了大量真实异常。USAD 和 OmniAnomaly 作为基于重构的方法, 在精确率和召回率之间取得了相对均衡的结果, 但 F1 分数均未超过 0.65。GDN 和 MTAD-GAT 作为基于图神经网络的方法, 在该数据集上的表现最差, 这可能是因为这些方法主要针对多变量间的关联异常设计, 而本数据集的异常主要表现为单节点的行为偏离。MAD-GAN 在基线方法中表现相对最好, 但 F1 分数也仅为 0.6891。

分析基线算法表现不佳的原因, 主要在于负载均衡类指标的固有特性。此类指标的正常波动具有随机性和不可预测性, 难以通过历史数据学习出稳定的正常模式。基于重构或预测的深度学习方法往往只能学习到数据的主要趋势, 对于负载突变和大幅波动容易产生误报, 而对于相对于其他节点偏离但绝对值在正常范围内的异常则容易漏报。

上述结果表明, 以学习单节点历史模式为核心思路的现有方法难以有效应对负载均衡类指标波动性强的挑战。

3.3 基于节点相似性的异常检测算法

基于上一节的评估结果, 现有多元时序异常检测算法在分布式集群多节点场景下表现欠佳。但第二章揭示的多节点曲线相似性规律为异常检测提供了新的思路: 通过度量节点间曲线的差异程度来识别偏离群体行为模式的异常节点。本节详细介绍三种基于曲线相似性的异常检测方法, 分别从数值距离、深度特征嵌入和局部密度三个不同角度刻画节点间的相似性关系, 并通过实验评估其检测性能与参数敏感性。

3.3.1 基于欧氏距离的数值相似性度量

欧氏距离作为度量空间中最基本的距离度量方式，能够直接量化节点间曲线在数值空间中的差异程度。该方法具有计算简单、物理意义直观的优点，适用于对曲线数值偏离敏感的异常检测场景。

给定一个包含 N 个节点的分布式集群，每个节点 i 在时刻 t 的监控指标观测值记为 x_i^t 。为了捕捉时间序列的局部变化模式并降低瞬时噪声的影响，本方法采用滑动窗口机制对原始序列进行分段处理。对于长度为 w 的时间窗口，节点 i 在时刻 t 的窗口序列可表示为向量形式 $\mathbf{x}_i^t = [x_i^{t-w+1}, x_i^{t-w+2}, \dots, x_i^t]^T \in \mathbb{R}^w$ 。基于此，节点 i 与节点 j 在时刻 t 的欧氏距离定义为两个窗口向量之间的 L2 范数：

$$d_{ij}^t = \|\mathbf{x}_i^t - \mathbf{x}_j^t\|_2 = \sqrt{\sum_{k=t-w+1}^t (x_i^k - x_j^k)^2} \quad (3-1)$$

为了从多节点距离信息中识别异常节点，本方法设计了基于统计标准化的异常分数计算机制。首先，在时刻 t 的滑动窗口内计算所有节点两两之间的欧氏距离，构成距离矩阵 $\mathbf{D}^t \in \mathbb{R}^{N \times N}$ 。然后，计算该距离矩阵中所有非对角元素的均值 μ_t 和标准差 σ_t ，以此表征当前时刻整个集群的曲线整体相似性水平。对于每个节点 i ，计算其与其他所有节点的平均距离：

$$\bar{d}_i^t = \frac{1}{N-1} \sum_{j=1, j \neq i}^N d_{ij}^t \quad (3-2)$$

节点 i 在时刻 t 的异常分数采用 Z-score 标准化形式计算：

$$s_i^t = \frac{\bar{d}_i^t - \mu_t}{\sigma_t} \quad (3-3)$$

该标准化过程将异常分数转换为相对于群体平均水平的偏离程度，消除了不同时刻集群整体负载水平差异带来的影响。在异常判定阶段，设定阈值 T ，若 $s_i^t > T$ ，则判定节点 i 在时刻 t 发生异常。

欧氏距离方法的理论依据在于分布式集群的负载均衡特性。在系统正常运行时，负载均衡机制确保各节点承担相近的工作负载，因此正常节点的监控曲线应保持较高的相似性，其异常分数将聚集在较低水平。当某个节点发生故障或性能异常时，其曲线会显著偏离其他节点的共同模式，导致该节点与其他节点的平均距离增大，异常分数相应升高。

3.3.2 基于自编码器嵌入的形态相似性度量

欧氏距离方法对曲线的绝对数值高度敏感，可能忽略形态层面的相似性。例如，两条变化趋势相同但振幅不同的曲线，在欧氏距离度量下差异较大，但形态

上实际高度一致。为克服这一局限,本研究引入自编码器 (Autoencoder, AE) 进行深度特征提取,通过学习曲线形态的抽象表示来度量节点间相似性。

自编码器是一种无监督表示学习模型,其网络结构如图3-4所示,由编码器 f_{enc} 和解码器 f_{dec} 两部分组成。编码器通过多层全连接网络将输入序列 $\mathbf{x}_t^i$ 逐层压缩为低维嵌入向量 $\mathbf{z}_t^i = f_{\text{enc}}(\mathbf{x}_t^i)$,解码器则以对称的网络结构从嵌入向量逐层恢复,重构原始输入 $\hat{\mathbf{x}}_t^i = f_{\text{dec}}(\mathbf{z}_t^i)$ 。通过最小化输入与重构之间的均方误差进行训练,自编码器迫使嵌入向量在有限的维度中保留曲线最本质的形态特征。相比原始数值表示,嵌入向量对曲线的整体趋势、周期性和变化模式等形态特征更加敏感,而对绝对数值差异相对不敏感。

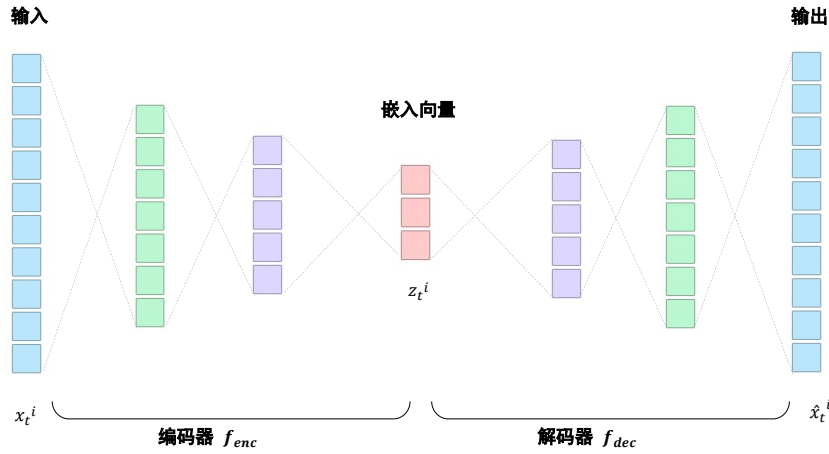


图 3-4 自编码器网络结构示意图

Figure 3-4 Architecture of the Autoencoder Network

本研究采用的自编码器网络包含三层全连接编码器 (维度变化为 $w \rightarrow 128 \rightarrow 128 \rightarrow 5$) 和对称的三层全连接解码器 ($5 \rightarrow 128 \rightarrow 128 \rightarrow w$)，其中 w 为滑动窗口大小。各隐藏层采用 ReLU 激活函数，优化器采用 Adam，学习率设置为 0.001，使用训练集数据训练 50 个 epoch。在应用阶段，仅使用编码器部分为各节点生成嵌入向量，后续基于嵌入向量计算节点间距离 $d_{ij}^t = \|\mathbf{z}_i^t - \mathbf{z}_j^t\|_2$ ，异常分数计算流程与欧氏距离方法一致。

3.3.3 基于 DBSCAN 聚类的密度相似性度量

除了基于距离度量的方法外,基于密度的聚类算法也能够有效识别偏离主流模式的异常点。DBSCAN^[46] 算法是密度聚类的经典代表,其核心思想是将数据空间划分为高密度区域形成的簇和低密度区域中的离群点,具有无需预先指定簇数量、能够发现任意形状簇以及自动识别噪声点等优点。

DBSCAN 算法基于两个关键参数进行聚类:邻域半径 ϵ 和最小样本数 $MinPts$ 。对于数据点 p ,其 ϵ 邻域定义为 $N_\epsilon(p) = \{q \in D \mid dist(p, q) \leq \epsilon\}$ 。若 $|N_\epsilon(p)| \geq MinPts$,则 p 为核心点;若 p 不是核心点但位于某个核心点的 ϵ 邻域内,则 p 为边界点;否则 p 为噪声点。算法通过密度可达关系将核心点及其密度相连的点聚合为簇,而噪声点则不属于任何簇。

在多节点异常检测场景中，本方法对每个时间窗口内的节点曲线进行 DBSCAN 聚类分析。具体而言，将节点 i 在时刻 t 的窗口序列 $\mathbf{x}_i^t$ 作为特征向量，对所有 N 个节点的特征向量集合执行 DBSCAN 聚类。被算法标记为噪声点的节点即被判定为在该时刻发生异常。

该方法的检测逻辑与分布式集群的运行特性高度契合。在负载均衡机制的作用下，正常运行的节点其监控曲线形态相似，在特征空间中应当聚集形成高密度簇。而发生故障或性能异常的节点，其曲线会显著偏离主流模式，在特征空间中表现为远离高密度区域的孤立点，从而被 DBSCAN 算法自动识别为噪声。

DBSCAN 方法的主要优势在于无需人工设定异常分数阈值，算法能够自适应地根据数据分布特征划分正常簇和异常点。然而，该方法对邻域半径 ϵ 参数较为敏感，需要根据具体数据特征进行调优。

3.3.4 参数敏感性实验及分析

3.3.4.1 最优参数检测结果

为验证上述三种基于曲线相似性方法的有效性，本节在构建的单指标多节点 RPC 连接数数据集上进行实验评估。三种方法在各自最优参数配置下的检测结果如表3-3所示。

表 3-3 基于曲线相似性方法在最优参数下的异常检测结果

Table 3-3 Anomaly Detection Results of Curve Similarity Methods under Optimal Parameters

算法	Precision	Recall	F1
欧氏距离	0.9826	0.9657	0.9741
自编码器嵌入	0.9792	0.9592	<u>0.9691</u>
密度偏离	0.9605	0.9659	0.9632

从实验结果可以看到，三种方法的 F1 分数均超过 0.96，显著优于表3-2中所有基线算法的性能。其中，欧氏距离方法在最优参数配置（threshold = 3.6、window size = 20）下达到了 0.9741 的 F1 分数，精确率和召回率分别为 0.9826 和 0.9657。自编码器嵌入方法取得了 0.9691 的 F1 分数，密度偏离方法取得了 0.9632 的 F1 分数。

这一结果充分验证了基于多节点曲线相似性进行异常检测的核心思路的有效性。通过对比节点间的曲线差异而非学习绝对的正常模式，本方法能够有效规避负载敏感型指标波动性强带来的挑战。然而，上述优异性能建立在精确的参数调优基础之上，三种方法均对阈值或关键参数的选择表现出较高的敏感性。下一小节将通过系统的参数敏感性实验揭示这一问题。

3.3.4.2 参数敏感性分析

为深入理解参数选择对算法表现的影响规律, 本节在构建的单指标多节点数据集上开展系统的参数敏感性实验, 分别考察时间窗口大小和检测阈值两个维度对三种方法检测效果的影响, 实验结果如表3-4、表3-5和表3-6所示。

表 3-4 欧氏距离方法的参数敏感性实验结果

Table 3-4 Parameter Sensitivity Results of Euclidean Distance Method

阈值	窗口大小 =10			窗口大小 =20			窗口大小 =30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.5485	1.0000	0.7084	0.7251	1.0000	0.8406	0.8214	1.0000	0.9020
3.2	0.6797	1.0000	0.8093	0.7892	1.0000	0.8822	0.8593	1.0000	0.9243
3.4	0.8116	1.0000	<u>0.8960</u>	0.8974	1.0000	<u>0.9459</u>	0.9256	1.0000	0.9614
3.6	0.9532	0.9655	0.9593	0.9826	0.9657	0.9741	0.9727	0.9469	<u>0.9596</u>
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

其中, 欧氏距离方法的最优参数为 threshold = 3.6、window size = 20, 此时 F1 分数达到 0.9741。当阈值偏低 (如 3.0 或 3.2) 时, 召回率维持在 1.0 但精确率显著下降, 说明产生了大量误报; 当阈值升至 3.8 时检测完全失效, 表明该方法对阈值选择极为敏感。窗口大小的影响相对温和, 较大的窗口有助于平滑噪声, 但各窗口下的整体趋势一致。

表 3-5 自编码器嵌入方法的参数敏感性实验结果

Table 3-5 Parameter Sensitivity Results of Autoencoder Embedding Method

阈值	窗口大小 =10			窗口大小 =20			窗口大小 =30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.4370	0.9833	0.6051	0.6028	1.0000	0.7522	0.7206	1.0000	0.8376
3.2	0.6070	1.0000	0.7554	0.7860	1.0000	0.8802	0.8391	1.0000	0.9125
3.4	0.7814	0.9732	<u>0.8668</u>	0.9106	0.9819	<u>0.9449</u>	0.9333	0.9859	<u>0.9589</u>
3.6	0.9507	0.8977	0.9234	0.9793	0.9404	0.9595	0.9792	0.9592	0.9691
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

自编码器嵌入方法(AE)呈现出类似的敏感性特征。其最优参数为 threshold = 3.6、window size = 30, F1 分数为 0.9691。与欧氏距离方法相比, 自编码器在低阈值时的精确率更低 (如 threshold = 3.0 时仅 0.4370), 说明编码器的重构误差分布较分散, 需要更严格的阈值设定才能有效区分正常与异常样本。

密度偏离方法 (DBSCAN) 的控制参数为邻域半径 eps。当 eps 设置偏小 (如 200) 时, 算法将多数样本视为离群点, 精确率极低; 当 eps 过大 (如 1000) 时, 真实异常点也被纳入主簇而被漏检。最优参数 eps = 600、window size = 10 下

表 3-6 密度偏离方法的参数敏感性实验结果

Table 3-6 Parameter Sensitivity Results of Density Deviation Method

eps	窗口大小 =10			窗口大小 =20			窗口大小 =30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
200	0.2296	1.0000	0.3735	0.0301	1.0000	0.0585	0.0157	1.0000	0.0309
400	0.9160	1.0000	<u>0.9562</u>	0.8208	1.0000	0.9016	0.5698	1.0000	0.7260
600	0.9813	0.9677	0.9745	0.9766	0.9709	0.9738	0.9726	0.9595	0.9660
800	1.0000	0.8238	0.9034	1.0000	0.8988	<u>0.9467</u>	0.9923	0.9281	<u>0.9591</u>
1000	1.0000	0.7685	0.8691	1.0000	0.8313	0.9079	1.0000	0.8207	0.9015

F1 分数为 0.9745，但该参数在不同窗口大小下的最优取值差异较大，进一步说明跨场景参数复用的困难。

综合三张表的实验结果，可以观察到两方面共性规律。在时间窗口维度上，三种方法均表现出窗口越大、检测效果越稳定的趋势，这是因为较大的时间窗口能够包含更丰富的时序模式信息，有效平滑瞬时噪声的干扰。然而窗口过大也会增加检测延迟，在实际部署中需要在精度和响应速度之间进行权衡。在阈值维度上，三种方法均呈现出精确率与召回率此消彼长的典型特征，最优参数均处于极窄的区间内，细微的参数偏移便可导致性能骤变。

为更直观地展示上述敏感性实验的规律，图3-5将三种方法在不同窗口大小下的 F1 分数随阈值变化的趋势以折线图形式呈现。

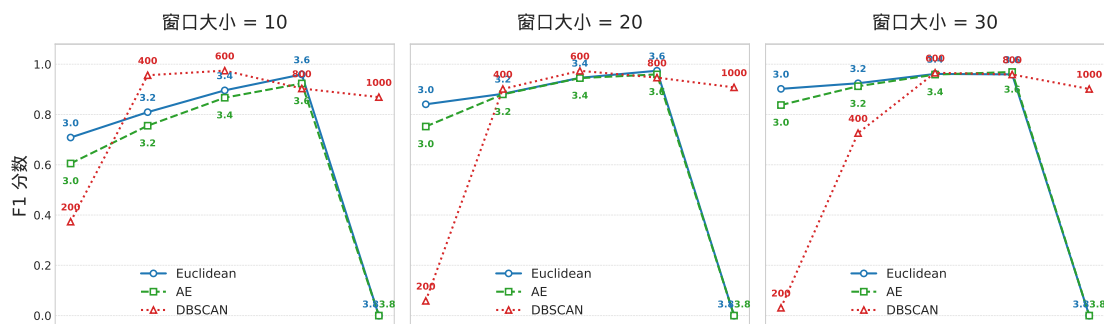


图 3-5 三种检测方法在不同窗口大小下 F1 分数随阈值变化的折线图

Figure 3-5 F1 Score vs. Threshold for Three Detection Methods under Different Window Sizes

从图中可以清晰地观察到，三种方法均呈现出典型的“先升后降”趋势，且在最优阈值附近存在陡峭的性能衰减。尤其是欧氏距离方法和自编码器嵌入方法，当阈值超过 3.6 后 F1 分数急剧跌落至零，呈现显著的“悬崖效应”。DBSCAN 方法虽然在较宽的参数区间内保持较高性能，但不同窗口大小下的最优 eps 值差异显著，进一步说明参数难以跨场景复用。

上述实验结果揭示了基于曲线相似性的异常检测方法在实际部署中面临的核心挑战。尽管三种方法在最优参数配置下均能取得优异的检测效果，但这种高

性能严重依赖于精确的参数调优，而参数调优过程本身又依赖于充足的标注数据，形成了方法实用化的主要障碍。

从参数依赖性的角度分析，如图3-5所示，三种方法的最优参数均处于一个狭窄的有效区间内，超出该区间后性能急剧下降。这种“悬崖效应”的根源在于异常检测任务本身的特殊性：异常样本在总体样本中占比极低，导致精确率和召回率对决策边界的位置高度敏感。以欧氏距离方法为例，阈值从 3.6 调整到 3.7 仅变化约 2.8%，但 F1 分数却下降了约 4%，这种非线性的性能衰减特性给参数选择带来了极大的困难。

从泛化能力的角度分析，最优参数的确定严重依赖于特定数据集的统计特性。不同集群的节点数量、负载模式、监控指标量纲等因素均会影响曲线相似性的分布特征，进而改变最优阈值的位置。在一个集群上调优得到的参数，直接迁移到另一个集群往往难以取得理想效果。这种泛化性不足的问题限制了方法的可扩展性，难以满足大规模分布式系统中多集群统一监控的实际需求。

从工程实践的角度分析，生产环境中获取高质量标注数据的成本极高。异常事件本身具有稀疏性和多样性的特点，完整覆盖各类异常模式需要长期积累。同时，异常标注需要具备领域专业知识的运维人员进行判断，人力成本高昂。更为关键的是，系统运行环境会随时间演化，历史数据上调优的参数可能因概念漂移而逐渐失效，需要持续的人工干预和重新标定。

综合以上分析，如何降低算法对预设阈值的依赖性，实现阈值的动态自适应调整，成为将基于曲线相似性的异常检测方法推向实际生产应用的关键技术挑战。针对这一问题，下一节将提出 CSAD-AT 算法，通过多视角分数融合与基于极值理论的自适应阈值机制，实现端到端的无人工调参异常检测。

3.4 CSAD-AT: 阈值自适应的异常检测算法

上一节的实验表明，基于曲线相似性的异常检测方法在最优参数下能取得优异性能，但对阈值选择高度敏感，难以直接应用于生产环境。为解决这一问题，本节提出 CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) 算法，将多视角相似性评分、分数归一化聚合与自适应阈值检测整合为统一的端到端检测流程，消除对人工阈值调优的依赖。

3.4.1 算法框架概述

CSAD-AT 算法的整体架构如图3-6所示，包含三个核心模块：(1) 多视角相似性评分模块，融合数值距离、嵌入空间距离和局部密度三种互补的评分方法，提升对不同类型异常的覆盖能力；(2) 分数归一化与聚合模块，将不同量纲的评分统一归一化后加权融合，保证各方法贡献的可比性；(3) I-SPOT 自适应阈值检测模块，将所有节点的聚合分数按时间窗口交织为全局序列，基于极值理论动态确定异常判定边界，无需人工设定阈值。

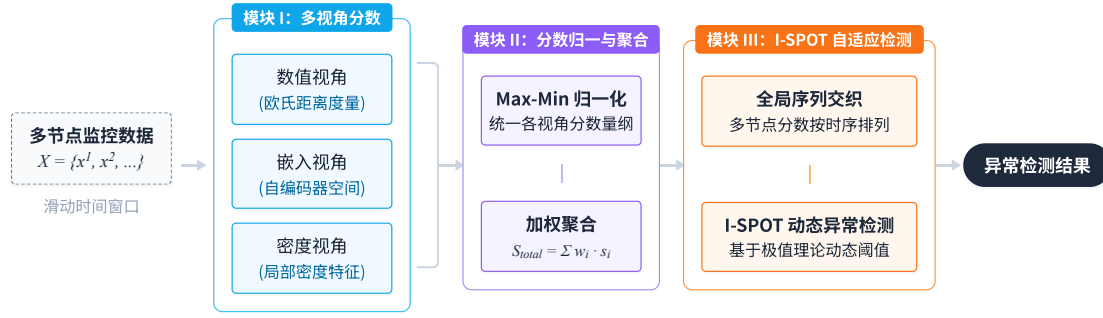


图 3-6 CSAD-AT 算法整体架构

Figure 3-6 Overall Architecture of the CSAD-AT Framework

3.4.2 多视角异常评分

CSAD-AT 算法采用三种基于曲线相似性的异常评分方法，分别从数值距离、嵌入空间距离和局部密度三个互补视角对节点的偏离程度进行量化。其中，欧氏距离评分和自编码器嵌入评分的计算流程与 3.3.1 节所述一致，分别在原始空间和潜在空间中度量节点间的平均距离并通过 Z-score 标准化得到异常分数。

密度偏离评分方面，与 3.3.3 节介绍的 DBSCAN 聚类方法不同，CSAD-AT 算法借鉴局部离群因子 (Local Outlier Factor, LOF)^[10] 的思想，设计了基于 k 近邻可达密度的连续异常评分方法。该方法不进行显式的簇划分，不依赖 ϵ 邻域半径和 $MinPts$ 等聚类参数，而是输出连续的异常分数，更适合与后续的归一化聚合及 I-SPOT 自适应阈值算法衔接。

对于每个时间窗口内的每个节点 i ，首先基于距离矩阵找到其 k_{nn} 个最近邻节点，记第 k_{nn} 近邻距离为 $d_i^{k_{nn}}$ 。以 $d_i^{k_{nn}}$ 作为自适应邻域半径，计算节点 i 的邻域集合 $\mathcal{N}_i = \{j : d_{ij} \leq d_i^{k_{nn}}, j \neq i\}$ 。基于邻域信息，计算节点 i 的局部可达密度 (Local Reachability Density, LRD)：

$$LRD_i = \frac{1}{\frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} d_{ij} + \epsilon} \quad (3-4)$$

其中 ϵ 为防止除零的小常数。LRD 越高，表明节点 i 周围的邻居越密集，处于高密度区域。

为综合刻画节点的局部密度特征，本方法计算三个子分数的 Z-score 标准化值：(1) LRD 的 Z-score z_{LRD} ；(2) 第 k_{nn} 近邻距离的 Z-score z_{kth} ；(3) 邻域内邻居数量的 Z-score z_{nbr} 。最终的密度偏离分数通过加权组合得到：

$$s_i^l = w_1 \cdot z_{LRD} + w_2 \cdot z_{kth} + w_3 \cdot z_{nbr} \quad (3-5)$$

其中权重的设计逻辑是：异常节点通常具有较低的局部密度 (LRD 偏低，故 w_1 取负值)、较大的 k 近邻距离 (z_{kth} 偏高， w_2 取正值) 以及较少的邻居 (z_{nbr} 偏低， w_3 取负值)。加权组合后的分数再经过整体 Z-score 标准化，确保不同窗口间的分数具有可比性。

3.4.3 分数归一化与聚合

三种评分方法输出的原始异常分数在量纲和数值范围上存在显著差异，直接聚合将导致某些方法的贡献被掩盖。为此，CSAD-AT 算法对每种方法的分数序列分别执行两步归一化处理。

第一步，负值截断：由于异常分数的正值表示偏离群体（潜在异常），负值表示与群体高度一致（正常），将所有负值截断为零：

$$s'^{(m)} = \max(s^{(m)}, 0) \quad (3-6)$$

第二步，MinMax 归一化：将截断后的分数映射到 $[0, 1]$ 区间：

$$\tilde{s}^{(m)} = \frac{s'^{(m)} - s'_{\min}^{(m)}}{s'_{\max}^{(m)} - s'_{\min}^{(m)}} \quad (3-7)$$

其中 $s'_{\min}^{(m)}$ 和 $s'_{\max}^{(m)}$ 分别为第 m 种方法截断后分数的最小值和最大值。经过归一化处理，三种方法的分数均位于 $[0, 1]$ 区间内，具有统一的可比性。

归一化后的三种分数通过加权平均进行聚合，得到最终的综合异常分数：

$$s_{\text{fused}} = \sum_{m=1}^M w_m \cdot \tilde{s}^{(m)}, \quad \sum_{m=1}^M w_m = 1 \quad (3-8)$$

其中 $M = 3$ 为评分方法数量， w_m 为各方法的权重。默认采用等权设置 $w_1 = w_2 = w_3 = 1/3$ ，即假设三种方法具有同等重要性。

选择加权平均作为聚合策略的理由在于：（1）相比最大值融合，加权平均保留了连续的分数信息，不会因单个方法的偶然高分而过度放大噪声；（2）相比投票机制，加权平均输出连续值，有利于后续 I-SPOT 的 GPD 尾部建模；（3）加权平均的计算开销极低，适合在线流式处理场景。实验中还对比其他聚合策略，详见 3.5.3 节。

3.4.4 I-SPOT: 包容性自适应阈值检测算法

3.3.4 节的实验揭示了基于曲线相似性的方法对阈值高度敏感的问题。为消除对人工阈值调优的依赖，CSAD-AT 算法引入基于极值理论（Extreme Value Theory, EVT）^[83] 的自适应阈值机制。然而，经典的 SPOT 算法在分布式集群监控场景中存在局限性：其数据池更新策略在概念漂移条件下易导致阈值逐渐失效。为此，本小节在介绍 SPOT 算法原理的基础上，提出改进的 I-SPOT（Inclusive SPOT）算法，通过包容性数据池更新与周期性 GPD 重估机制增强阈值的长期自适应能力。

3.4.4.1 SPOT 算法原理

SPOT（Streaming Peaks Over Threshold）^[83] 算法的核心思想是利用广义帕累托分布（Generalized Pareto Distribution, GPD）对数据分布的尾部进行建模，从而在无需预先假设数据整体分布形式的情况下，自动确定异常判定的阈值边界。

给定异常分数序列 $\{s_1, s_2, \dots, s_n\}$, SPOT 算法首先选取一个初始阈值 t_0 (通常取序列的高分位数), 将超过该阈值的观测值称为“超额” (exceedance)。根据极值理论中的 Pickands-Balkema-de Haan 定理, 当初始阈值 t_0 足够高时, 超额值 $Y = S - t_0$ (其中 $S > t_0$) 的条件分布可以用 GPD 来近似:

$$G_{\xi, \sigma}(y) = 1 - \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi}, \quad y > 0 \quad (3-9)$$

其中 ξ 为形状参数, $\sigma > 0$ 为尺度参数。通过 Grimshaw 方法对超额值序列进行极大似然估计, 可以得到参数 $\hat{\xi}$ 和 $\hat{\sigma}$ 的估计值。进而, 对于给定的风险水平 q , 异常检测阈值可计算为:

$$z_q = t_0 + \frac{\hat{\sigma}}{\hat{\xi}} \left[\left(\frac{n}{N_t} \cdot q \right)^{-\hat{\xi}} - 1 \right] \quad (3-10)$$

其中 n 为总观测数, N_t 为超过初始阈值 t_0 的观测数。

经典 SPOT 算法在流数据场景下持续更新 GPD 模型参数以适应数据分布的动态变化。然而, 该算法在更新模型时仅将未超过阈值 z_q 的正常数据点纳入更新池, 将判定为异常的数据点排除在外。这一设计在概念漂移场景下会引发阈值衰减问题: 当正常数据的整体分布随时间缓慢上移时, 越来越多的正常点被误判为异常而排除在模型更新之外, 导致训练数据池被“截断”, 基于截断样本估计的 GPD 参数使新阈值进一步降低, 形成自我强化的误报螺旋。

3.4.4.2 I-SPOT: 包容性自适应阈值算法

针对经典 SPOT 算法的阈值衰减问题, 本研究提出 I-SPOT (Inclusive SPOT) 算法。其核心改进在于采用包容性更新策略: 所有数据点 (无论是否超过当前阈值) 均纳入数据池, 从而使模型能够动态跟踪数据分布的演化趋势。

I-SPOT 算法维护一个最大容量为 $W_{\max}$ 的先进先出 (FIFO) 数据池 D_t 。对于每个新到达的异常分数 s_t , 更新规则定义为:

$$D_{t+1} = \begin{cases} D_t \cup \{s_t\}, & \text{if } |D_t| < W_{\max} \\ (D_t \setminus \{s_{\text{oldest}}\}) \cup \{s_t\}, & \text{otherwise} \end{cases} \quad (3-11)$$

与经典 SPOT 不同, I-SPOT 不对 s_t 是否超过阈值进行区分, 所有观测值均被纳入后续的模型拟合过程。

在阈值更新方面, I-SPOT 采用周期性重估策略: 每隔 T_{update} 个时间步, 基于当前数据池 D_t 重新计算初始阈值 $t_0^{(t)} = \text{Percentile}(D_t, \text{level})$, 提取超额量 $Y = \{s - t_0^{(t)} \mid s \in D_t, s > t_0^{(t)}\}$, 通过 Grimshaw 方法重新拟合 GPD 参数 $(\hat{\xi}^{(t)}, \hat{\sigma}^{(t)})$, 并按式(3-10)更新自适应阈值 z_q 。

此外, I-SPOT 引入自适应重置机制: 当累计异常数或已处理数据点数达到预设阈值时, 触发数据池的完全重置, 以应对突发性的分布变化。I-SPOT 算法的完整流程如算法1所示。

算法 1 I-SPOT 自适应阈值算法**算法 1** I-SPOT Adaptive Threshold Algorithm

Require: 异常分数流 $\{s_1, s_2, \dots\}$, 风险水平 q , 分位数水平 $level$, 数据池容量 $W_{\max}$, 重估间隔 T_{update} , 初始长度 n_0

Ensure: 异常判定结果, 动态阈值序列 $\{z_q^{(t)}\}$

- 1: $D \leftarrow \{s_1, \dots, s_{n_0}\}; t_0 \leftarrow \text{Percentile}(D, level)$ ▷ 初始化
- 2: $(\hat{\xi}, \hat{\sigma}) \leftarrow \text{GrimshawMLE}(\{s - t_0 \mid s \in D, s > t_0\})$ ▷ 拟合 GPD
- 3: $z_q \leftarrow \text{ComputeThreshold}(t_0, \hat{\xi}, \hat{\sigma}, q); steps \leftarrow 0$
- 4: **for each** incoming score s_t **do**
- 5: **if** $s_t > z_q$ **then** 标记时刻 t 为异常
- 6: **end if**
- 7: **if** $|D| \geq W_{\max}$ **then** $D \leftarrow D \setminus \{s_{\text{oldest}}\}$
- 8: **end if** ▷ 淘汰最旧
- 9: $D \leftarrow D \cup \{s_t\}; steps \leftarrow steps + 1$ ▷ 包容性更新
- 10: **if** $steps \geq T_{\text{update}}$ **then** ▷ 周期性重估
- 11: $t_0 \leftarrow \text{Percentile}(D, level)$
- 12: $(\hat{\xi}, \hat{\sigma}) \leftarrow \text{GrimshawMLE}(\{s - t_0 \mid s \in D, s > t_0\})$
- 13: $z_q \leftarrow \text{ComputeThreshold}(t_0, \hat{\xi}, \hat{\sigma}, q); steps \leftarrow 0$
- 14: **end if**
- 15: **end for**

通过包容性更新策略, I-SPOT 能够在保持对真实异常敏感性的同时, 自适应地跟踪正常数据分布的长期演化趋势。当系统负载整体上升导致异常分数分布右移时, I-SPOT 的阈值会相应上调, 避免产生大量误报; 当系统恢复稳定状态时, 阈值又会逐渐回落至合适水平。

3.4.4.3 全局时间线构建与异常判定

CSAD-AT 算法的一个关键设计是将所有节点的聚合分数按窗口交织为全局时间序列, 而非为每个节点独立运行阈值检测。具体而言, 对于窗口 k 中 N 个节点的聚合分数 $\{s_{\text{fused},0}^{(k)}, s_{\text{fused},1}^{(k)}, \dots, s_{\text{fused},N-1}^{(k)}\}$, 全局序列的排列方式为:

$$\mathbf{S}_{\text{global}} = [s_{\text{fused},0}^{(0)}, s_{\text{fused},1}^{(0)}, \dots, s_{\text{fused},N-1}^{(0)}, s_{\text{fused},0}^{(1)}, \dots] \quad (3-12)$$

即采用窗口优先、节点次序的交织方式。

在全局序列 $\mathbf{S}_{\text{global}}$ 上运行 I-SPOT 算法, 得到每个位置的异常判定结果。随后, 将异常标记映射回 (窗口, 节点) 矩阵, 即可确定具体在哪个时间窗口的哪个节点发生了异常。

这种全局时间线设计的优势在于: (1) 所有节点共享同一个 I-SPOT 实例和阈值, 大幅降低了计算和存储开销; (2) 全局序列的样本量更大, 使 GPD 拟合

更加稳健；(3) 阈值能够同时反映所有节点的分数的分布特征，对整体负载变化更加敏感。

3.5 实验验证与分析

3.5.1 实验设置

本节实验在本文构建的单指标多节点数据集上进行，旨在全面验证 CSAD-AT 算法及其各组件的有效性。实验评估采用异常检测领域通用的精确率 (Precision)、召回率 (Recall) 和 F1 分数三个指标，异常判定采用窗口级别的评估协议：若检测到的异常窗口与真实异常标注窗口的重叠比例超过设定参数值，则视为正确检测 (True Positive)。

I-SPOT 算法的超参数设置为：风险水平 $q = 0.0065$ ，分位数水平 $\text{level} = 0.98$ ，重估间隔 $T_{\text{update}} = 50$ ，初始序列长度 $n_0 = 1500$ 。

实验对比的基线算法包括六种具有代表性的多元时序异常检测算法：TranAD、USAD、OmniAnomaly、GDN、MAD-GAN 和 MTAD-GAT，涵盖基于重构、预测和生成对抗网络等主流范式。所有基线算法均使用作者开源的原版代码和默认推荐参数，训练阶段使用 3.1.1 节所述的 30 天正常数据。

3.5.2 整体性能分析

表 3-7 展示了 CSAD-AT 算法与各基线算法以及三种单独评分方法在测试集上的检测性能对比。

表 3-7 各算法异常检测性能对比

Table 3-7 Anomaly Detection Performance Comparison of All Methods

类别	算法	Precision	Recall	F1
基线算法	TranAD ^[22]	0.8889	0.3932	0.5452
	USAD ^[20]	0.6892	0.5567	0.6159
	OmniAnomaly ^[19]	0.6054	0.4593	0.5224
	GDN ^[24]	0.3101	0.5218	0.3890
	MAD_GAN ^[75]	0.6309	0.6309	0.6891
	MTAD_GAT ^[66]	0.3492	0.1573	0.2169
相似性算法 (最优阈值)	欧氏距离	0.9826	0.9657	0.9741
	自编码器嵌入	0.9792	0.9592	0.9691
	密度偏离	0.9605	0.9659	0.9632
本章算法	CSAD-AT	1.0000	0.9732	0.9864

从实验结果可以观察到以下规律。现有的多元时序异常检测算法在本数据集上的表现普遍欠佳，F1 分数均未超过 0.7，其原因在于此类方法主要针对多变

量间的关联异常设计,而本数据集的异常主要表现为单节点的行为偏离,且负载均衡类指标的正常波动具有随机性和不可预测性,难以通过历史数据学习出稳定的正常模式。相比之下,三种基于曲线相似性的评分方法在最优人工调参条件下均取得了显著更优的检测效果,F1 分数均超过 0.96,充分验证了基于多节点曲线相似性进行异常检测的核心思路的有效性。

本研究提出的 CSAD-AT 算法在无需人工阈值调整的条件下,F1 分数达到了 0.9864,优于各评分方法在最优阈值下的性能水平,证明分数聚合与 I-SPOT 自适应阈值的组合能够有效替代基于标注数据的人工调参过程,具有重要的实际应用价值。

3.5.3 分数聚合策略对比实验

为验证所选分数聚合策略的有效性,本节对比了不同归一化方法与聚合策略的组合。实验结果如表3-8所示。

表 3-8 不同分数聚合策略的性能对比

Table 3-8 Performance Comparison of Different Score Aggregation Strategies

归一化	聚合策略	Precision	Recall	F1
Z-score	加权平均	0.9157	0.9806	0.9470
MinMax	最大值	1.0000	0.8426	0.9146
MinMax	投票	0.9831	0.9587	0.9707
MinMax	加权平均	1.0000	0.9732	0.9864

实验结果表明,MinMax 归一化在所有聚合策略下均优于 Z-score 归一化。分析其原因,Z-score 归一化假设分数服从近似正态分布,但异常分数的分布往往具有显著的右偏特征,Z-score 归一化可能过度压缩正常区域的分数差异,而 MinMax 归一化直接将分数映射到固定区间,保留了原始分布的相对关系。在聚合策略方面,MinMax 加权平均以 0.9864 的 F1 分数取得最优性能,投票策略以 0.9707 居次,最大值策略的 F1 分数最低,仅为 0.9146。最大值策略的召回率仅有 0.8426,说明仅取单方法最高分容易受到个别方法偶然高分的干扰,导致部分真实异常被遗漏。投票策略虽然具有较好的鲁棒性,但其离散化的决策过程损失了连续分数信息,不利于后续 I-SPOT 的 GPD 尾部建模。综合来看,加权平均策略在保留连续分数信息和抑制噪声干扰之间取得了最优平衡。

3.5.4 消融实验: 各评分方法贡献分析

为分析三种评分方法在 CSAD-AT 算法中的各自贡献,本节设计了消融实验,将每种方法单独与 I-SPOT 结合进行检测,并与三种方法融合后的完整算法进行对比。实验结果如表3-9所示。

消融实验结果表明,每种评分方法单独与 I-SPOT 结合均能取得较好的检测

表 3-9 消融实验：各评分方法与 I-SPOT 结合的性能

Table 3-9 Ablation Study: Performance of Each Scoring Method Combined with I-SPOT

检测配置	Precision	Recall	F1
欧氏距离 + I-SPOT	1.0000	0.9187	0.9576
自编码器嵌入 + I-SPOT	0.9891	0.8667	0.9239
密度偏离 + I-SPOT	1.0000	0.8917	0.9427
CSAD-AT	1.0000	0.9732	0.9864

效果，F1 分数均超过 0.92。其中，欧氏距离方法表现最优（F1=0.9576），精确率达到 1.0，说明其产生的异常分数分布最适合 I-SPOT 进行阈值划分。自编码器嵌入方法的召回率相对较低（0.8667），原因在于部分形态异常的分数值与正常样本重叠度较高。密度偏离方法同样实现了零误报（Precision=1.0），但召回率略低于欧氏距离方法。

完整的三方法融合方案在 F1 分数上达到 0.9864，显著优于任何单一方法。特别是召回率从最优单方法的 0.9187 提升至 0.9732，说明三种方法形成了有效的互补：欧氏距离捕捉数值幅度偏离，自编码器嵌入识别形态模式异常，密度偏离发现局部孤立型异常。融合后的框架综合了三种视角的判断，提升了对复杂异常场景的覆盖能力。

3.5.5 I-SPOT 与经典 SPOT 对比实验

为验证 I-SPOT 算法相对于经典 SPOT 的改进效果，本节在相同的融合分数序列上分别运行经典 SPOT 与 I-SPOT，对比两者的检测性能。两种算法采用完全相同的超参数设置，包括风险水平 $q = 0.0065$ 、分位数水平 $\text{level} = 0.98$ 和重估间隔 $T_{\text{update}} = 50$ ，以确保性能差异完全来自于更新策略的不同。

表 3-10 经典 SPOT 与 I-SPOT 性能对比

Table 3-10 Performance Comparison of Classic SPOT and I-SPOT

阈值算法	Precision	Recall	F1
经典 SPOT	0.3218	1.0000	0.4870
I-SPOT	1.0000	0.9732	0.9864

表3-10给出了两种算法的量化检测性能对比。经典 SPOT 虽然实现了 1.0 的召回率，即成功检测到了所有真实异常窗口，但其精确率仅为 0.3218，意味着在全部报警中超过三分之二为误报。这一严重的误报问题使得经典 SPOT 的 F1 分数仅有 0.4870，在实际应用中会导致运维人员频繁收到无效告警，大幅降低告警系统的可信度。相比之下，I-SPOT 在保持 0.9732 高召回率的同时实现了 1.0 的精确率，即所有报警均为真实异常，F1 分数达到 0.9864，较经典 SPOT 提升了

约 50 个百分点。

图3-7从阈值动态变化的角度直观揭示了两种算法产生如此显著性能差异的根本原因。该图分为上下两个子图，分别对应经典 SPOT 和 I-SPOT 在同一融合分数序列上的检测过程。两个子图中，蓝色竖线表示各数据点的融合异常分数值，反映了每个时间窗口内各节点的综合异常程度；橙色曲线为算法动态计算的自适应阈值 z_q ，当某个数据点的融合分数超过该阈值时即被判定为异常；红色实心圆点标记为算法输出的异常检测点。图中左侧的灰色阴影区域对应算法的初始化阶段，该阶段用于积累初始数据以完成 GPD 模型的首次拟合，不进行异常判定。

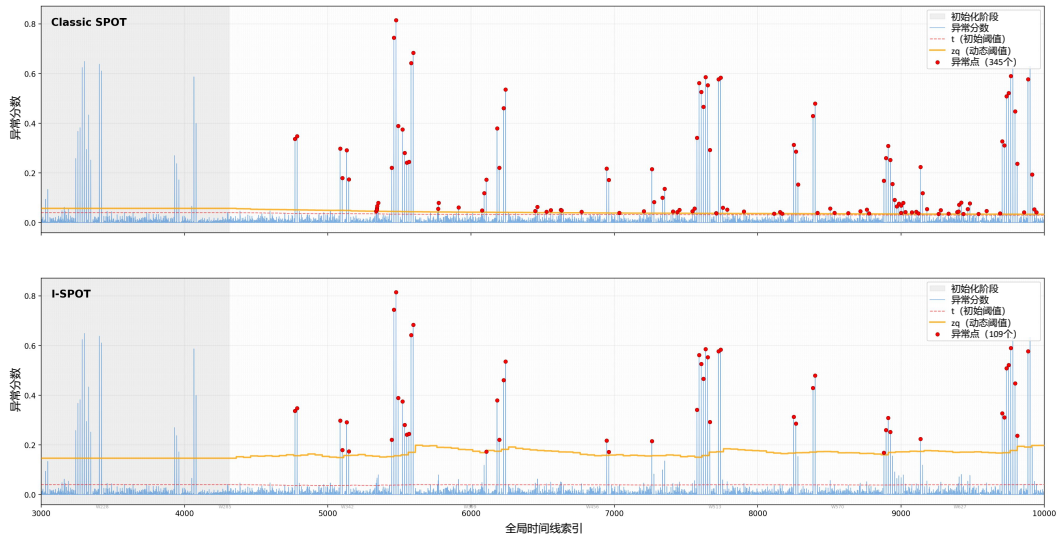


图 3-7 融合分数上经典 SPOT 与 I-SPOT 检测结果对比

Figure 3-7 Comparison of Classic SPOT and I-SPOT Detection Results on Fused Scores

从上子图可以清晰地观察到经典 SPOT 的阈值衰减现象。在初始化阶段结束后的早期，橙色阈值曲线尚处于较高水平，能够有效区分正常数据点和异常数据点。然而随着流式处理的持续推进，阈值呈现出明显的单调下降趋势，到序列后半段已降至极低水平，导致几乎所有数据点都超过了阈值而被标记为异常，图中后半段密集出现的了大量误报。这一现象的根源在于经典 SPOT 采用排他性更新策略，超过阈值 z_q 的数据点被视为异常而排除在数据池之外，数据池中仅保留低分数的正常样本。随着被排除的数据点逐渐增多，数据池的分数分布不断向低值区域收缩，GPD 模型拟合的尾部分布随之变窄，阈值进一步降低，形成了“阈值衰减”的恶性正反馈循环。

下子图展示了 I-SPOT 在相同数据上的表现。可以看到，橙色阈值曲线在整个处理过程中始终保持平稳，没有出现经典 SPOT 中的单调下降现象。I-SPOT 的红色异常点与真实标注高度吻合，几乎不存在误报。这得益于 I-SPOT 采用的包容性更新策略，将所有数据点纳入数据池 D_t ，不论其是否被标记为异常。由于高分数据点始终参与 GPD 的尾部拟合，模型能够持续感知分数分布的真实形态，尾部参数估计更加稳健，阈值也因此能够维持在合理水平。

3.6 本章小结

本章针对单指标多节点场景下的分布式集群异常检测问题,从数据构建、算法评估、方法设计到算法整合进行了系统研究,提出了 CSAD-AT(Curve Similarity-based Anomaly Detection with Adaptive Threshold) 算法。主要工作和贡献如下:

(1) 基于联通大数据生产环境构建了高质量的单指标多节点时序数据集,采用多算法初筛与专家判读相结合的半人工标注策略,确保异常标签的准确性,为后续算法研究提供了可靠的数据支撑。

(2) 系统评估了六种主流多元时序异常检测算法在分布式集群场景下的适用性,实验表明此类基于时间和空间建模的方法在本场景下表现欠佳,验证了基于节点曲线相似性进行异常检测的必要性。

(3) 提出了基于欧氏距离、自编码器嵌入和 DBSCAN 聚类的三种曲线相似性异常评分方法,分别从数值幅度、形态模式和密度分布三个视角度量节点间的行为偏离程度,并通过参数敏感性实验揭示了阈值依赖性这一核心挑战。

(4) 设计了 CSAD-AT 算法,融合多视角相似性评分、分数归一化与加权聚合、全局时间线构建等模块,并提出 I-SPOT 自适应阈值算法,通过包容性数据池更新与周期性 GPD 重估机制解决了经典 SPOT 算法的阈值衰减问题。实验结果表明,CSAD-AT 算法在无需人工阈值调整的条件下,F1 分数达到 0.9864,优于各评分方法在最优阈值下的表现,显著降低了对人工参数调优的依赖。

本章的工作验证了基于曲线相似性的异常检测方法在单指标多节点场景下的有效性。然而,实际生产环境中的分布式集群通常需要同时监控多种指标(如 CPU 使用率、内存占用、网络吞吐量等),如何将异常检测方法扩展到多指标场景,充分利用指标间的关联信息,将在下一章进行研究。

第4章 NSFusion：数值-形状融合的多指标多节点异常检测

第三章围绕单指标多节点场景，验证了基于曲线相似性的异常检测思路的有效性，并通过多算法融合与自适应阈值机制解决了参数敏感性问题。然而，真实分布式集群的监控数据通常包含多个节点和多种指标，单指标方法无法充分利用多维信息之间的互补关系。本章将检测范围扩展至多指标多节点场景，以真实 GPU 集群故障数据集为实验基础，从数值特征和形状特征两个互补视角分别设计异常检测算法，并通过加权融合实现视角的结合以提升检测效果。

4.1 问题定义及 NSFusion 算法框架

4.1.1 真实场景数据概述

本章所使用的数据集来源于真实 GPU 计算集群的生产环境监控系统，数据采集时间跨度覆盖 2024 年 7 月至 2025 年 1 月。大规模 GPU 集群是支撑人工智能模型训练和高性能计算任务的核心基础设施，其运行稳定性直接关系到业务连续性和计算资源利用效率。一个典型的集群由数百台 GPU 计算节点组成，各节点通过统一的资源调度系统协同执行深度学习训练任务。在这种高度并行化的架构下，任何单个节点的故障都可能导致整个训练任务的中断，而目前集群运维团队虽已借助自动化工具覆盖了大部分常见故障的处理流程，但仍有少量复杂故障需要运维人员协同排查，耗费大量人力和时间成本。因此，利用多指标多节点的时间序列监控数据开展自动化异常检测研究，对于提升集群运维效率和缩短故障响应时间具有重要的工程价值。

该数据集包含 105 个经过标注的故障案例作为测试集，同时构造了约 100 个无标签的正常运行作业作为训练集。每个故障案例的根因标注来自运维系统中积累的专家经验记录。表4-1统计了测试集中各类故障的分布情况。

表 4-1 故障类别及数量分布

Table 4-1 Fault Categories and Distribution

故障类别	数量	典型根因
GPU 故障	51	Double bit error、XID error、GPU 利用率不均衡
机器故障	13	节点宕机、机器检查错误
网络故障	18	交换机问题、光模块故障、线缆故障、信号质量差
PCIe 故障	9	PCIe 性能退化、PCIe 信号质量差
NCCL 故障	5	NCCL 分段错误、NCCL 超时
其他	9	未明确分类的故障

集群监控系统以固定的采样周期持续采集各节点的运行状态数据，形成具

有时间对齐关系的多节点多指标时间序列。每个节点产生多个维度的监控指标，从硬件状态、计算负载、网络通信等不同层面反映节点的健康状况。

根据指标数值是否随计算任务负载发生变化，可将全部监控指标划分为两类。第一类是负载无关指标，如 PCIe Width、Network Link Down Event、Cable BER 和 XID Errors 等，这些指标的数值主要由硬件自身状态决定，不随计算任务的变化而波动，因此可以直接通过横向对比各节点的数值来识别离群节点。第二类是负载相关指标，如 GPU 利用率、GPU 温度和显存使用率等，这些指标的数值与当前运行的计算任务密切相关，正常工况下就会产生较大的波动幅度，需要采用更为精细的分析方法才能有效区分正常波动与真实异常。表4-2列出了数据集中的主要监控指标及其含义。

表 4-2 监控指标列表及说明

Table 4-2 List of Monitoring Metrics and Descriptions

监控指标	说明
CPU Usage	CPU 繁忙处理任务的时间占比
Memory Usage	系统 RAM 被程序和操作系统占用的比例
GPU PCIe Throughput	GPU 与主板间通过 PCIe 传输数据的速率
NVLink Throughput	通过高速 NVLink 互连传输数据的速率
GPU Temp	GPU 的当前运行温度
Tensor Core Activity	GPU 张量核心用于 AI 计算的利用率
FP32/FP16 Activity	单精度/半精度浮点运算单元的活动百分比
SM Activity	GPU 流式多处理器的利用率
GPU Memory Usage	GPU 专用显存的使用比例
GPU Usage	GPU 整体处理任务的繁忙时间占比
PCIe Width*	PCIe 连接中活动数据通道的数量
Network Link Down Event*	网络接口连接丢失的事件次数
Cable BER*	线缆数据传输过程中的误码率
XID Errors*	GPU 报告的严重错误次数

注：带 * 标记的为负载无关指标。

对于一个包含 N 个节点、 M 个监控指标、时间长度为 T 的数据集，可以形式化表示为一个三维张量：

$$\mathbf{X} \in \mathbb{R}^{N \times M \times T} \quad (4-1)$$

在实际运行环境中，集群节点通常执行相同类型的计算任务，并通过调度系统实现负载均衡。这一机制使得各节点在正常运行状态下，相同指标上的变化趋势通常保持较高的一致性。当某个节点发生异常时，其监控指标曲线会偏离这种集群一致性模式。值得注意的是，这种偏离存在两种不同的表现形式。一种是数值幅度上的异常，例如某节点的 GPU 温度显著高于集群中其他节点，在曲线上

表现为明显的垂直偏移。另一种是变化模式上的异常，例如功耗曲线出现不规则的波动或抖动，而数值幅度本身仍处于正常范围之内。后一种异常更具隐蔽性，仅通过数值比较难以有效捕捉。正是这种由系统调度机制和任务一致性所带来的节点间行为一致性，为基于相似性的异常检测方法奠定了理论基础。

图4-1展示了一个真实故障案例的多节点时序可视化结果，从该案例涉及的众多监控指标中选取了两个具有代表性的指标进行对比分析。图中灰色曲线为正常节点，红色曲线为异常节点。左图展示的 `tcp_prio0_tx_throughput` 是与本次故障密切相关的核心异常指标，异常节点不仅在数值上明显偏离正常节点簇，还在变化趋势上出现了突发性的剧烈波动和持续下降，同时呈现出数值偏移和形态异常两种特征。然而，一次故障中并非所有指标都会表现出明显的异常。右图展示的 `mem_usage` 是与此次故障弱相关的指标，异常节点的曲线与其他正常节点并没有显著偏离，凭该指标难以可靠地识别异常。实际上，在该案例的其他指标中还存在与故障完全无关的情形，异常节点与正常节点的曲线几乎完全重合。这一对比表明，不同指标对同一故障的敏感程度存在显著差异，异常节点并非在所有指标上都表现出差异，这要求检测算法能够从众多指标中有效识别出异常节点及其真正异常的指标。

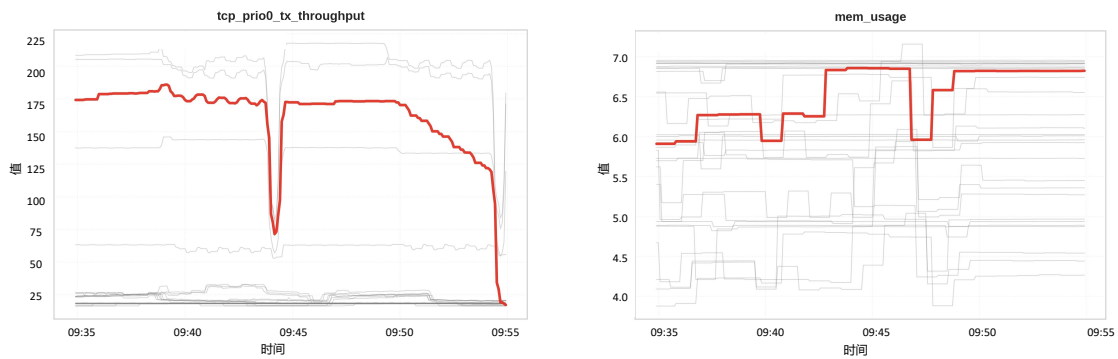


图 4-1 GPU 集群典型故障案例的多指标多节点时序可视化

Figure 4-1 Multi-metric Multi-node Time Series Visualization of a Typical GPU Cluster Fault

该数据集具有四方面显著特点。首先，数据来源于真实生产环境，具有较高的工程真实性和复杂性，不同节点之间存在自然波动和采样噪声。其次，数据具有典型的多指标特征，不同指标对异常的敏感程度存在差异，仅依赖单一指标难以全面刻画节点的异常行为。再次，数据具有显著的多节点关联特性，正常状态下大多数节点表现出一致的运行模式，而异常节点则在统计意义上偏离多数节点的共同行为。最后，数据具有明显的时间序列特征，异常既可能表现为短时突发的瞬态故障，也可能表现为持续存在的长期退化。

4.1.2 问题分析与挑战

在 GPU 集群的日常运维实践中，工程师定位故障的基本思路是寻找与集群中大多数节点表现不一致的节点。这一朴素而有效的经验可以形式化为如下的离群检测问题。给定一个包含 N 台机器的集合 $M = \{m_1, m_2, \dots, m_N\}$ 和一个不

相似度函数 $d(\cdot, \cdot)$ ，目标是找到与其他所有机器的累积不相似度最高的机器 m^* ，即

$$m^* = \arg \max_{m_k \in M} D(m_k) \quad (4-2)$$

其中 $D(m_k)$ 为机器 m_k 与集合中其他所有机器的总不相似度。

尽管这一形式化定义清晰明确，前文的 CSAD-AT 算法也已在单指标场景下验证了基于曲线相似性的异常检测思路的有效性，但从单指标扩展到多指标多节点场景时，实现可靠高效的异常检测与故障定位仍面临两方面新挑战。

第一，多指标场景下数据规模大幅增长，对检测算法的**整体效率**提出了更高要求。第三章的单指标场景仅需处理一个指标的节点间距离计算，而 GPU 集群数据集包含十余个监控指标，每个指标都需要独立完成 $N \times N$ 的节点距离矩阵构建，计算总量随指标数成倍增长。在此效率约束下，第三章 CSAD-AT 算法的三种评分方法面临不同程度的迁移应用困难。欧氏距离方法虽然计算流程相对轻量，但在多指标场景下同样需要优化距离计算和聚合策略以适应数据规模的增长；密度偏离方法涉及 k 近邻搜索和局部密度估计，在节点数高达 500 至 700 的大规模场景下计算代价显著；自编码器需要为每个指标分别训练独立模型，模型数量和训练开销随指标数线性增长，是三者中效率瓶颈最为突出的组件。而在 CSAD-AT 的三种评分方法中，欧氏距离和密度偏离均从数值角度度量差异，自编码器从嵌入向量分析高位形态差异，其无法迁移意味着整个框架缺少形态分析维度。这就要求一方面寻找不依赖训练过程、计算效率更高的替代方案来显式捕捉形状特征，另一方面整体算法框架的设计也需要在保证检测效果的前提下兼顾大规模多指标数据的处理效率。

第二，多指标间的分布差异和异常表现的异质性对单一视角方法的**检测稳定性**构成挑战。GPU 集群的监控指标涵盖硬件状态、计算负载、网络通信等多个层面，由于物理含义和采集方式不同，各指标在量纲、数值范围和分布特征上存在显著差异，波动模式也各不相同。如前文图4-1的案例所示，异常在不同指标上的表现形式高度异质，既包括数值幅度的偏离，也包括变化模式的异常，还有部分指标对当前故障完全不敏感。在这种多指标环境下，仅依赖数值距离的度量方式存在固有的检测盲区：对于数值幅度正常但变化模式异常的隐蔽故障，欧氏距离无法有效识别；同时，与故障无关的指标产生的噪声距离可能干扰真正异常指标的判别信号。因此，需要从数值和形状等多个互补视角对节点的异常行为进行度量，并设计合理的跨指标融合策略来提升检测的覆盖面和鲁棒性。

4.1.3 NSFusion 算法框架

针对上述挑战，本章提出 NSFusion (Numerical-Shape Fusion) 算法框架，即数值与形状双视角融合的异常检测框架。该框架在整体设计上以计算效率为优先约束，放弃了 CSAD-AT 中依赖模型训练的自编码器和计算密集密度偏离方法，转而采用轻量化的数值距离计算与无需训练的符号化形状度量相结合的方案，在保证检测效果的前提下适应多指标大规模数据的处理需求。同时，框

架从数值特征和形状特征两个互补视角分别分析节点的异常行为，通过加权融合弥补单一数值视角的检测盲区。整个框架包含三个处理阶段，其整体流程如图4-2所示。

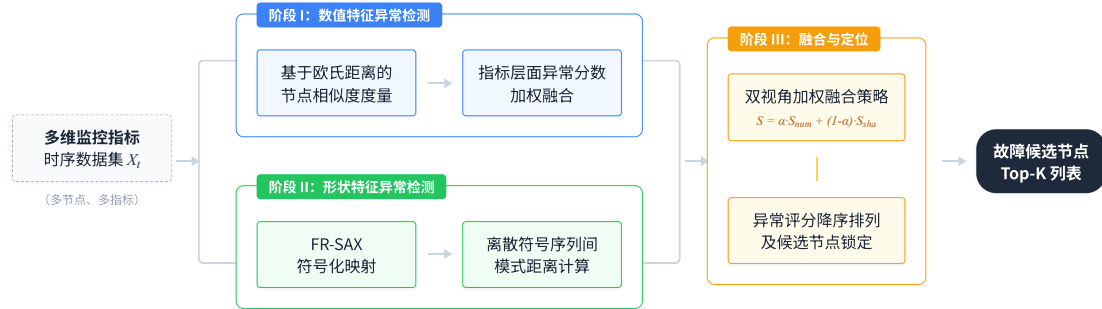


图 4-2 NSFusion 算法框架流程

Figure 4-2 Workflow of the NSFusion Algorithm Framework

第一阶段为数值特征异常检测。该阶段沿用第三章中基于欧氏距离的相似度度量思路，对每个监控指标分别计算各节点与其他节点的平均距离作为异常分数，并通过优化的加权平均策略将多个指标的异常分数融合为数值视角下的综合异常评分，在适应多指标数据规模的同时整合多维信息。

第二阶段为形状特征异常检测。该阶段采用 FR-SAX 符号化算法，通过将时间序列映射为离散符号序列并计算符号序列间的距离来捕捉曲线形态的差异。该方法无需模型训练，计算效率显著优于自编码器，同时能够识别数值幅度正常但变化模式异常的隐蔽故障，与数值视角形成互补。

第三阶段为多视角融合与故障定位。该阶段将前两个阶段输出的异常分数进行加权融合，综合考虑数值偏离和形态变化两方面因素，输出各节点的最终异常评分并进行降序排列，排名靠前的节点即为最可能的故障候选，供运维人员进一步排查确认。

4.2 基于数值特征的异常检测算法

本节介绍数值视角下的异常检测算法。该算法的基本思路与第三章中基于欧氏距离的曲线相似性度量方法一脉相承，但在多指标场景下需要解决一个新的问题，即如何将多个指标各自独立的异常度量结果整合为对节点整体异常状态的综合评估。

4.2.1 基于欧氏距离的单指标异常度量

面对多指标数据，一个直观的处理方式是将所有指标的时序数据拼接为高维向量，直接计算节点间的多指标联合距离。在多元统计分析领域，马氏距离 (Mahalanobis Distance)^[3] 是处理多维联合距离的经典方法，其核心思想是在计算样本间距离时引入协方差矩阵的逆来消除变量间的相关性和尺度差异。对于两

个 p 维向量 $\mathbf{x}$ 和 $\mathbf{y}$ ，其马氏距离定义为：

$$d_M(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \mathbf{y})} \quad (4-3)$$

其中 $\boldsymbol{\Sigma} \in \mathbb{R}^{p \times p}$ 为数据的协方差矩阵。当 $\boldsymbol{\Sigma}$ 为单位矩阵时，马氏距离退化为欧氏距离。马氏距离值越大，表明样本偏离正常分布中心的程度越高，是多元异常检测中被广泛采用的基线方法。

然而，这类多指标联合距离方案在 GPU 集群故障检测场景中存在以下局限性：（1）量纲与尺度差异的干扰，不同监控指标的数值范围和物理单位差异显著（如 CPU 利用率为 0% 至 100%，PCIe 吞吐量可达数十 GB/s），数值尺度较大的指标会主导距离结果，马氏距离虽通过协方差矩阵缓解该问题，但其估计依赖多元正态分布假设，实际监控数据的分布往往更为复杂；（2）异常信号的稀释效应，实际故障往往只影响部分指标，正常指标的距离贡献会稀释异常指标的判别信号；（3）可解释性的缺失，联合距离将多指标信息压缩为单一数值，不利于后续的故障根因定位。在本章的实验部分，马氏距离将作为联合距离方案的代表性基线与本文方法进行对比。

基于上述分析，本文采用先分后合的处理策略：对每个监控指标独立计算节点间的异常分数，再通过加权融合整合为综合评分。这种策略规避了量纲差异的干扰，保留了各指标异常分数的独立可查性，同时通过权重调整可灵活控制不同指标的贡献程度。

对于单个监控指标 m ，沿用第3.3.1节的欧氏距离度量方法，对集群中 N 个节点在时间窗口 $[1, T]$ 内的时序数据 $\mathbf{X}^{(m)} \in \mathbb{R}^{N \times T}$ ，计算节点间的距离矩阵 $\mathbf{D}^{(m)} \in \mathbb{R}^{N \times N}$ ，并得到节点 i 在指标 m 上的平均距离 $\bar{d}_i^{(m)}$ 。与第三章不同的是，多指标场景需要消除不同指标间的尺度差异以确保异常分数具有可比性，因此采用 Z-score 标准化：

$$s_i^{(m)} = \frac{\bar{d}_i^{(m)} - \mu_d^{(m)}}{\sigma_d^{(m)}} \quad (4-4)$$

其中 $\mu_d^{(m)}$ 和 $\sigma_d^{(m)}$ 分别为所有节点在指标 m 上平均距离的均值和标准差。标准化后的异常分数 $s_i^{(m)}$ 反映节点 i 在指标 m 上相对于集群整体的偏离程度，正值越大表明异常程度越高。

4.2.2 多指标加权融合策略

在获得各指标的独立异常分数后，需要设计合理的融合机制将其整合为节点的综合异常评分。本文采用加权求和的融合策略，节点 i 的综合异常分数定义为：

$$S_i = \sum_{m=1}^M w_m \cdot \max(s_i^{(m)}, 0) \quad (4-5)$$

其中 w_m 为指标 m 的权重，满足 $\sum_{m=1}^M w_m = 1$ 。

公式4-5中对异常分数施加了 $\max(\cdot, 0)$ 的截断操作, 其设计动机在于处理正常指标的负面干扰问题。当某个指标的异常分数 $s_i^{(m)}$ 为负值时, 说明节点 i 在该指标上的累积距离低于集群平均水平, 即该节点在该指标上的表现处于正常甚至优于平均的状态。在故障检测的语境下, 这种正常信号不应对最终的异常判定结果产生抵消作用, 否则可能导致真实异常节点因部分指标表现正常而被低估。因此, 将负值截断为 0 能够确保每个指标要么对最终异常评分产生正向贡献, 要么不产生贡献, 避免了正常指标对异常指标判别信号的稀释。

关于各指标权重的确定, 本文采用等权重策略, 即对所有监控指标赋予相等权重 $w_m = 1/M$ 。选择这一策略的主要考虑在于, 本研究面向的是通用的集群异常检测场景, 在缺乏特定集群的故障模式先验知识、且无法依赖运维工程师对各指标重要性进行经验性标注的条件下, 等权重方案是最为稳健和客观的选择。该方案不对任何指标引入人为偏好, 使得检测结果完全由数据本身的统计特征驱动, 避免了因权重设定不当而遗漏某些指标上的异常信号。在实际部署中, 如果运维团队积累了充足的历史故障案例, 可以进一步根据各指标在故障识别中的贡献情况调整权重分配, 以获得更优的检测效果。

4.2.3 算法实现与效率优化

算法2给出了基于数值特征的异常检测算法的完整流程。算法以多节点多指标时序数据张量 $\mathbf{X} \in \mathbb{R}^{N \times M \times T}$ 和指标权重向量 $\mathbf{w}$ 作为输入, 输出各节点的综合异常分数向量 $\mathbf{S}$ 。

算法2 基于数值特征的多指标异常检测算法

算法2 Multi-metric Anomaly Detection Algorithm Based on Numerical Features

Require: 时序数据 $\mathbf{X} \in \mathbb{R}^{N \times M \times T}$, 权重向量 $\mathbf{w} \in \mathbb{R}^M$

Ensure: 异常分数向量 $\mathbf{S} \in \mathbb{R}^N$

- 1: 初始化 $\mathbf{S} \leftarrow \mathbf{0}$
 - 2: **for** $m = 1$ to M **do** ▷ 遍历每个指标
 - 3: $\mathbf{D}^{(m)} \leftarrow$ 计算节点间欧氏距离矩阵 ▷ $\mathbf{D}^{(m)} \in \mathbb{R}^{N \times N}$
 - 4: $\bar{\mathbf{d}}^{(m)} \leftarrow$ 计算各节点累积距离 ▷ $\bar{d}_i^{(m)} = \frac{1}{N-1} \sum_{j \neq i} D_{ij}^{(m)}$
 - 5: $\mathbf{s}^{(m)} \leftarrow$ Z-score 标准化 ($\bar{\mathbf{d}}^{(m)}$) ▷ 转换为异常分数
 - 6: $\mathbf{S} \leftarrow \mathbf{S} + w_m \cdot \max(\mathbf{s}^{(m)}, 0)$ ▷ 加权累加
 - 7: **end for**
 - 8: **return** $\mathbf{S}$
-

从算法复杂度来看, 外层循环遍历 M 个指标, 内层对每个指标需要计算 $N \times N$ 的距离矩阵, 而每次距离计算涉及长度为 T 的时间序列, 因此整体时间复杂度为 $O(M \cdot N^2 \cdot T)$ 。其中距离矩阵的构建是最主要的计算瓶颈。为使算法能够满足大规模集群实时监控场景对响应速度的要求, 本文在工程实现层面进行了三方面的效率优化。

在距离计算环节，本文将传统的双重循环改写为基于 NumPy 和 SciPy 库的向量化矩阵运算。具体而言，利用 `scipy.spatial.distance.cdist` 函数一次性计算整个 $N \times N$ 的距离矩阵，该函数的底层实现调用了经过高度优化的 BLAS 线性代数库，能够充分利用现代 CPU 的 SIMD 指令集和多级缓存结构进行并行计算，相比 Python 层面的显式循环可以获得数量级的性能提升。

在数据预处理环节，本文针对原始监控数据中存在的问题设计了相应的处理流程。对于由采样丢失或传输延迟导致的缺失值，采用线性插值方法进行填充，以保证时序数据在时间维度上的连续性和完整性。对于长时间跨度的监控数据，按照固定大小的时间窗口进行切分，将长时序分解为多个可独立处理的短窗口。这种切分策略一方面降低了单次计算的内存占用，另一方面使得算法能够以滑动窗口的方式对数据流进行在线处理。

在增量计算环节，针对滑动窗口检测的应用场景，本文实现了增量式的距离矩阵更新机制。当新的数据点到达时，算法仅重新计算涉及新增数据点的那部分距离，而非从头构建完整的距离矩阵。与此同时，各节点的历史累积距离统计量被缓存在内存中，用于 Z-score 标准化时的均值和标准差估计，从而避免了对历史数据的重复遍历。

通过上述优化措施的综合应用，欧氏距离算法在处理包含 500 个节点、6 个监控指标的典型数据规模时，单次检测耗时约为 3.7 秒；当节点数量增至 700 个时，耗时约为 6 秒，能够满足生产环境中分钟级检测周期的实时监控需求。

4.3 基于形状特征的异常检测算法

数值特征方法通过欧氏距离度量能够有效捕捉曲线在幅度上的偏离，但对于数值范围正常而变化模式异常的情况存在检测盲区。例如，某个节点的 GPU 功耗虽然始终处于正常范围之内，但出现了与其他节点不同的高频抖动模式，这类形态层面的异常在欧氏距离的度量下可能被低估。为弥补数值方法的这一局限，本节从形状特征的视角出发，基于 SAX 符号化表示方法设计异常检测算法。

4.3.1 经典 SAX 算法原理

SAX^[84] 是时间序列分析领域中一种经典的符号化表示方法，其核心思想是将连续的数值时间序列转换为离散的符号序列，从而在降低数据维度的同时保留序列的主要形态特征。传统 SAX 算法的处理流程包含四个步骤。

第一步是归一化处理。将原始时间序列标准化为均值为 0、标准差为 1 的分布形式，以消除不同序列之间在量纲和基线水平上的差异。对于时间序列 $\{x_1, x_2, \dots, x_T\}$ ，归一化后的序列为：

$$\hat{x}_t = \frac{x_t - \mu}{\sigma} \quad (4-6)$$

其中 μ 和 σ 分别为序列的均值和标准差。

第二步是分段聚合近似。将归一化后的时间序列均匀划分为 w 个等长的片段, 并以每个片段内所有数据点的算术平均值作为该片段的代表值。这一步骤实现了从长度 T 到长度 w 的降维压缩。

第三步是符号化映射。利用标准正态分布的分位数点将数值空间划分为 α 个等概率的区间, 其中 α 为字母表的大小。每个片段的均值根据其所处的区间被映射为相应的离散符号, 由此将连续时间序列转换为长度为 w 的符号序列。

第四步是距离计算。在符号空间中定义符号间的距离函数, 并据此计算符号序列之间的距离。在传统 SAX 的距离定义中, 相邻符号之间的距离被设为 0, 仅非相邻符号之间才计算实际距离, 该距离等于对应分位数点之差。

4.3.2 面向集群故障检测的 FR-SAX 算法

传统 SAX 算法的设计目标是通用的时间序列相似性搜索, 其中分段聚合和宽容性距离设计有利于在降维后仍保持检索精度。然而, 在 GPU 集群故障检测场景中, 这两个设计反而可能成为性能瓶颈。基于对实际数据的分析, 本文提出 FR-SAX (Full-Resolution SAX) 算法, 从全分辨率保留和严格距离度量两方面对传统 SAX 进行有针对性的改进。

第一项改进是取消分段聚合步骤, 保留原始时间分辨率。传统 SAX 通过分段聚合实现降维, 但片段内的均值化操作不可避免地会平滑掉短时间尺度上的细节信息。在 GPU 性能监控场景中, 故障的早期征兆往往表现为持续时间很短的瞬时尖峰或短暂抖动。例如, PCIe 通信出现间歇性故障时, 吞吐量曲线可能在个别采样点上出现骤降, 但片段聚合后这种瞬态变化可能被周围正常数据点的均值所淹没。因此, 本文选择跳过分段聚合步骤, 直接对归一化后的每个数据点逐一进行符号化映射。虽然这意味着符号序列的长度与原始序列相同, 不再具有降维效果, 但在集群场景下各节点的序列长度通常在同一量级, 计算开销仍然可控, 而换来的是对细粒度形态差异的完整保留。

图4-3以一条包含 20 个采样点的归一化时间序列为例, 直观对比了这一改进的效果。在传统 SAX 中, 序列被划分为 4 个等长片段, 每个片段取均值后映射为单个符号, 得到长度为 4 的符号序列, 原始序列中的局部波动细节在均值化过程中被平滑掉。而 FR-SAX 直接对每个数据点逐一进行符号化映射, 得到与原始序列等长的符号序列, 完整保留了时序变化的细粒度信息。

第二项改进是采用严格的符号距离计算规则。传统 SAX 将相邻符号之间的距离设为 0, 这一设计的出发点是认为相邻区间内的数值差异不具有统计显著性。然而在故障检测场景中, 这种宽容性处理可能导致算法对形状变化不够敏感。考虑这样一种情况, 某个节点的符号序列与正常节点的符号序列在多个位置上存在一个符号级别的偏差, 传统距离函数会将这些偏差全部忽略, 而实际上这种系统性的单级偏差可能正是故障的早期信号。本文将距离计算规则修改为: 仅当两个位置的符号完全相同时距离才为 0, 否则一律计算实际的分位数点

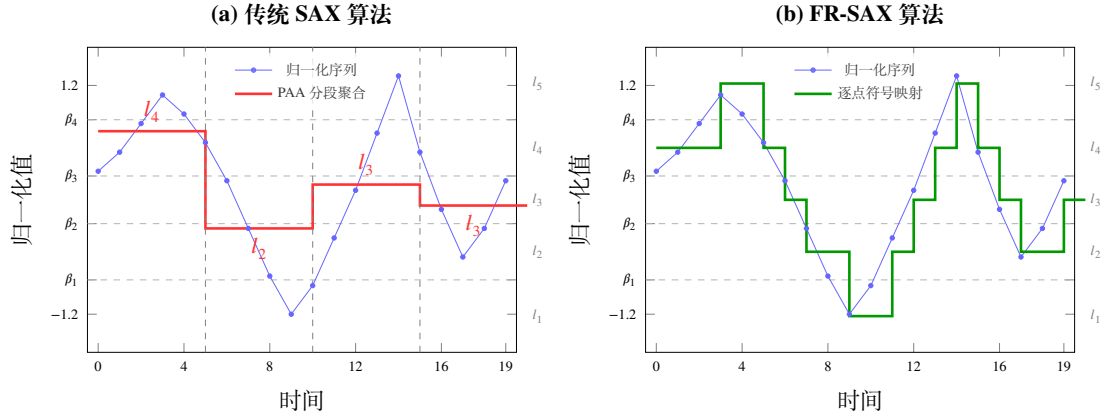


图 4-3 传统 SAX 与 FR-SAX 的符号化过程对比

Figure 4-3 Comparison of Symbolization Process between Traditional SAX and FR-SAX

差距，即

$$d'(l_i, l_j) = \begin{cases} 0, & \text{if } i = j \\ \beta_{\max(i,j)-1} - \beta_{\min(i,j)}, & \text{if } i \neq j \end{cases} \quad (4-7)$$

其中 β_k 为标准正态分布的第 k 个分位数点。图4-4对比了两种距离定义的差异。传统 SAX 中相邻符号距离为 0（如 $d(l_3, l_4) = 0$ ），一个符号级别的偏差被完全忽略；FR-SAX 中只要符号不同就计算实际的分位数点差距（如 $d'(l_3, l_4) = 0.59$ ），从而对细微的形态差异保持敏感。

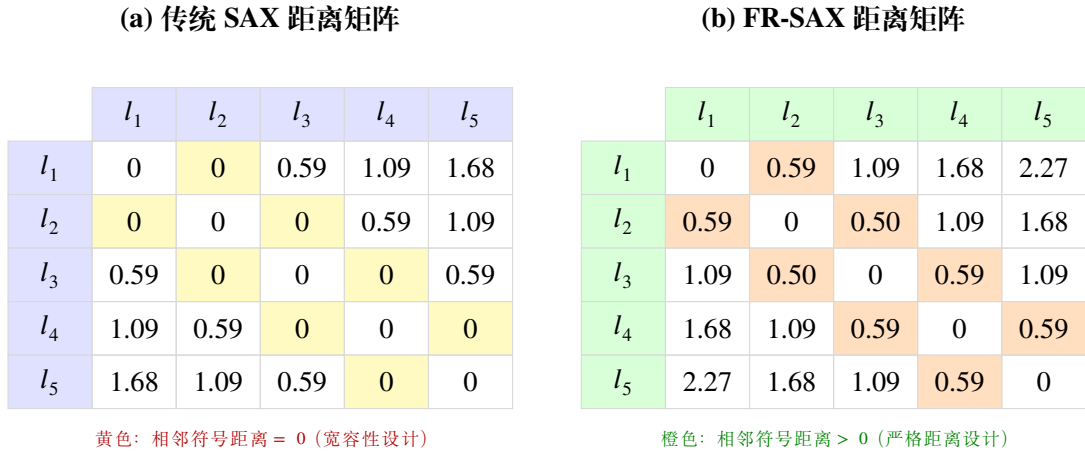


图 4-4 传统 SAX 与 FR-SAX 的符号距离矩阵对比

Figure 4-4 Comparison of Symbol Distance Matrices between Traditional SAX and FR-SAX

基于上述两项改进，多节点形状异常检测的完整流程如下。首先对各节点的时间序列进行归一化处理，然后根据预设的字母表大小计算高斯分位数点，将归一化后的序列逐点映射为符号序列。在此基础上，计算所有节点对之间的符号序列距离并构建节点距离矩阵，进而计算每个节点与其他节点的平均符号距离。最后，采用与数值方法相同的 Z-score 标准化流程将平均距离转换为异常分数，并对各指标的异常分数进行加权聚合得到形状视角下的综合评分。

4.3.3 算法效率优化

FR-SAX 算法由于取消了分段聚合, 符号序列长度与原始序列一致, 在节点数量较多时距离矩阵的计算规模会相应增大。为确保算法在大规模集群环境下仍能满足实时监控的响应要求, 本文在工程实现中采取了针对性的效率优化措施。

在符号距离计算方面, 本文利用 SAX 符号空间的有限性进行预计算。对于字母表大小为 α 的符号化表示, 任意两个符号之间的距离取值仅有 $\alpha \times \alpha$ 种可能。因此, 在算法初始化阶段预先计算并存储完整的 $\alpha \times \alpha$ 符号距离查找表, 运行时通过直接查表获取符号对的距离值, 避免了对分位数点进行重复的浮点运算。在此基础上, 将整个符号序列距离的计算过程改写为 NumPy 的向量化数组操作, 通过索引映射一次性完成全部符号对的距离查询和累加, 取代了逐元素的 Python 循环。

在内存管理方面, 采用按时间窗口分批处理的策略。对于长时间跨度的监控数据, 算法不会一次性将全部数据加载到内存中, 而是以固定大小的时间窗口为单位逐批读取和处理。每个窗口内的距离矩阵在计算完成并提取出异常分数后即可释放, 从而将峰值内存占用控制在与单个时间窗口数据量成正比的水平。对于节点数量特别多的场景, 还可以进一步采用即时计算策略, 仅在需要时计算特定节点对之间的距离, 而非预先构建完整的 $N \times N$ 距离矩阵。

通过上述优化措施的综合应用, FR-SAX 算法在处理包含 500 个节点的多指标数据集时, 单次检测耗时约为 7.3 秒; 当节点规模增至 700 个时, 耗时约为 11.9 秒, 满足实时监控场景的响应要求。

4.4 多算法融合与故障定位

4.4.1 双视角加权融合设计

数值特征方法和形状特征方法分别从不同的分析维度评估节点的异常程度, 二者的输出需要通过合理的融合机制整合为统一的异常判定结果。本文采用加权平均的融合策略来实现这一目标。

设节点 i 通过数值检测算法得到的异常分数为 s_i^{num} , 通过形状检测算法得到的异常分数为 s_i^{shape} 。由于两种算法的评分机制不同, 输出的分数在尺度上可能存在差异, 因此在融合之前需要分别对两组分数进行归一化处理, 使其落入可比较的数值范围。归一化后通过加权平均得到融合异常分数:

$$s_i^{final} = w_{num} \cdot s_i^{num} + w_{shape} \cdot s_i^{shape} \quad (4-8)$$

其中 w_{num} 和 w_{shape} 分别为数值检测和形状检测的融合权重, 满足 $w_{num} + w_{shape} = 1$ 。

融合权重的选择反映了对两种视角相对重要性的判断。在默认设置下, 本文采用等权重方案 $w_{num} = w_{shape} = 0.5$, 表示对数值特征和形状特征同等信任。在

实际部署中，如果运维团队根据历史故障统计发现某类故障更多地表现为数值层面的异常，可以适当提高数值检测的权重；反之，如果形态异常更为常见，则可以向形状检测倾斜。

在单一视角内部的多指标融合过程中，本文遵循两条规则以提高融合的鲁棒性。第一条规则是将异常分数为负的指标按 0 计入融合计算。负的 Z-score 意味着该节点在该指标上的表现处于集群平均水平之下，属于正常甚至优于平均的状态，不对异常判定产生抵消作用。第二条规则是异常分数为 0 的指标同样不产生融合贡献，因为零分数表明该指标在当前时间段内未能提供有区分度的异常判别信息。这两条规则的共同效果是确保融合结果仅由真正提供了异常证据的指标所驱动，避免了无关指标对判别信号的稀释。

4.4.2 异常节点检测

融合异常分数计算完成后，算法按照分数从高到低对所有节点进行排序，从而实现异常节点的自动识别与定位。排名靠前的节点具有最高的异常嫌疑度，在实际运维流程中通常取排名前 K 的节点作为异常候选，由运维工程师进一步排查确认。

双视角融合框架在异常节点检测中的一个重要优势在于，两种视角各自的异常分数能够从不同维度刻画节点的异常特征，为异常类型的初步判断提供有价值的参考信息。当某个节点同时具有较高的数值异常分数和较高的形状异常分数时，说明该节点的监控曲线在幅度和形态上都显著偏离了集群的正常模式，这种情况通常对应较为严重的硬件故障或系统级问题，例如 GPU 核心损坏导致的计算指标全面异常，或散热系统失效引起的温度和功耗同步飙升。当节点的数值异常分数较高但形状异常分数较低时，表明其曲线的整体变化趋势与其他节点一致，但在数值水平上存在系统性偏移，这种模式更可能对应配置层面的问题，例如资源分配参数设置不当或固件版本差异导致的性能基线偏移。当节点的数值异常分数较低但形状异常分数较高时，表明其指标数值始终处于正常范围内，但变化模式呈现出异于其他节点的特征，这类隐藏的异常通常指向间歇性故障或软件层面的问题，例如驱动程序兼容性问题导致的周期性性能抖动，或进程调度异常引起的不规则负载波动。

综上所述，本文提出的异常节点检测方法通过融合异常分数排序实现异常节点的自动识别，同时借助双视角各自的分数分布辅助判别异常类型，从而为运维人员的排查工作提供方向性指引，帮助其更高效地缩小问题范围。

4.5 实验验证与对比分析

4.5.1 实验设置

本节在真实 GPU 计算集群故障数据集上对所提方法进行实验验证。该数据集包含 105 个真实故障案例，每个案例对应一次经过运维团队确认并记录的 GPU 节点故障事件，故障类型涵盖 GPU 性能退化、温度异常升高、功耗异常波

动等多种情况。每个故障案例中同时包含故障节点及其所在集群全部节点在故障发生时间窗口内的多指标时间序列数据，为节点级别的对比分析提供了完整的数据支撑。

实验评估采用两个指标。第一个是 Acc@5 ，定义为真实故障节点出现在算法输出的 Top-5 排名列表中的案例占总案例数的比例，用于衡量算法的故障检测覆盖率。第二个是 Avg Rank ，定义为在算法成功检测到故障节点的案例中，故障节点在排名列表中的平均位次，用于衡量算法的定位精度，数值越小表示故障节点被排列在越靠前的位置。

基线算法选择马氏距离方法。马氏距离在计算样本间距离时考虑了变量之间的协方差结构，能够在一定程度上处理指标间的相关性，是多元统计异常检测领域中广泛使用的经典方法。

4.5.2 方法结果对比分析

表4-3展示了各方法在全部 105 个故障案例上的检测效果。从实验结果可以观察到几个值得关注的现象。首先，基于欧氏距离的数值检测方法在 Acc@5 指标上达到 0.800，相比马氏距离基线方法的 0.453 有大幅提升。这一结果表明，在 GPU 集群场景中，基于节点间曲线逐点比较的相似性度量方式比基于多变量协方差结构的统计方法更能有效地识别异常节点。其主要原因在于，马氏距离方法需要估计高维协方差矩阵，而在节点数量有限的情况下协方差估计的精度难以保证，从而影响了检测效果。

表 4-3 多指标多节点异常检测方法对比

Table 4-3 Comparison of Multi-metric Multi-node Anomaly Detection Methods

算法	Acc@5	Avg Rank
马氏距离	0.453	1.20
数值方法	0.800	1.25
形状方法	0.762	1.30
数值-形状融合	0.838	1.18

其次，基于 FR-SAX 的形状检测方法取得了 0.762 的 Acc@5 。虽然这一数值略低于数值检测方法，但形状方法能够成功检测到一部分数值方法遗漏的故障案例，说明两种方法在异常模式的覆盖范围上确实存在互补性。将两种方法进行加权融合后， Acc@5 进一步提升至 0.838，同时 Avg Rank 降至 1.18，验证了双视角融合策略的有效性。

4.5.3 融合方法优势分析

融合方法相比单一视角方法在两个层面上展现出明显优势。在异常覆盖的全面性方面，数值方法擅长识别幅度显著偏离的异常，而形状方法则能够捕捉那

些数值范围正常但变化模式异常的隐蔽故障。通过对 105 个故障案例的逐一分析，约有 15% 的案例主要表现为形态层面的异常而非数值层面的异常。这些案例中，故障节点的各项指标数值始终处于正常范围之内，但其变化趋势或波动模式与集群中其他节点存在明显差异。如果仅依赖数值方法进行检测，这部分案例将无法被有效覆盖。

在检测性能的稳定性方面，单一方法往往在其擅长的故障类型上表现优异，但在其他类型的故障上效果可能明显下降。融合方法通过综合两个视角各自的判断结果，能够在不同故障类型之间获得更为均衡的检测表现，降低了算法性能对特定故障类型的依赖程度。

为更直观地说明两种视角的互补性，下面通过两个典型故障案例进行具体分析。图4-5展示了一个 FR-SAX 方法检测效果优于欧氏距离方法的案例。该案例中，异常节点（红色曲线）的 CPU 利用率整体处于较低水平（约 2% 至 10%），与集群中大多数节点（15% 至 25%）之间存在明显的数值差距。从数值视角来看，这似乎是一个容易检测的案例。然而，需要注意的是，图中部分被标注的“正常”节点（如 Node_328、Node_057 等）在某些时段也出现了短暂的数值下跌，其瞬时数值甚至接近异常节点的水平。这些正常节点的偶发波动会在欧氏距离度量中产生干扰，缩小它们与异常节点的距离，从而降低异常节点的离群程度排名。相比之下，FR-SAX 方法关注的是曲线的整体变化模式而非瞬时数值，异常节点持续低位运行的形态特征与正常节点的高频波动模式之间存在本质差异，即使个别正常节点偶尔出现数值下跌，其整体形状仍与集群主体保持一致，因此 FR-SAX 能够更稳健地将异常节点识别出来。

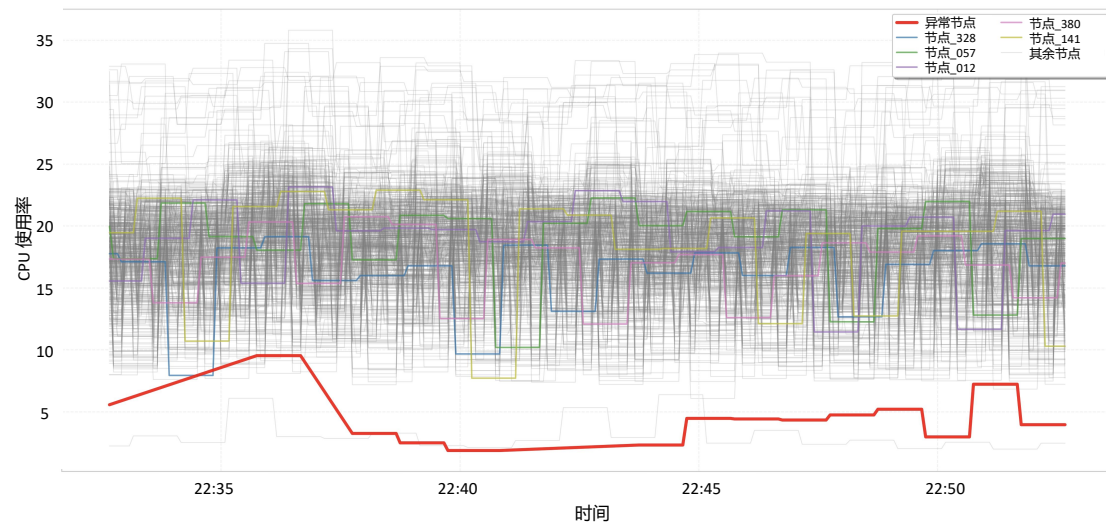


图 4-5 FR-SAX 方法优势案例

Figure 4-5 Case Where FR-SAX Outperforms

图4-6展示了一个欧氏距离方法检测效果优于 FR-SAX 方法的案例。该案例中，异常节点在 NVLink 发送字节数指标上的表现与集群存在显著的幅度差异：当集群中大多数节点的通信量从接近零快速攀升至 4×10^{10} 至 5×10^{10} 量级时，

异常节点虽然也呈现出上升趋势，但峰值仅达到 1×10^{10} 至 2×10^{10} 左右，幅度明显偏低。这种情况下，欧氏距离能够直接捕捉数值幅度上的巨大偏差，给出较高的异常分数。而对于 FR-SAX 方法，由于异常节点的变化趋势（从零到上升再到波动）与正常节点在形状上具有一定的相似性，符号化后的模式差异不如数值差异那么显著。同时值得注意的是，图中部分正常节点（如 Node_114、Node_025 等）的上升时间和波动模式与集群主体存在差异，在形状特征空间中反而可能表现为离群，这种“形状层面的正常节点噪声”会干扰 FR-SAX 对真正异常节点的识别。

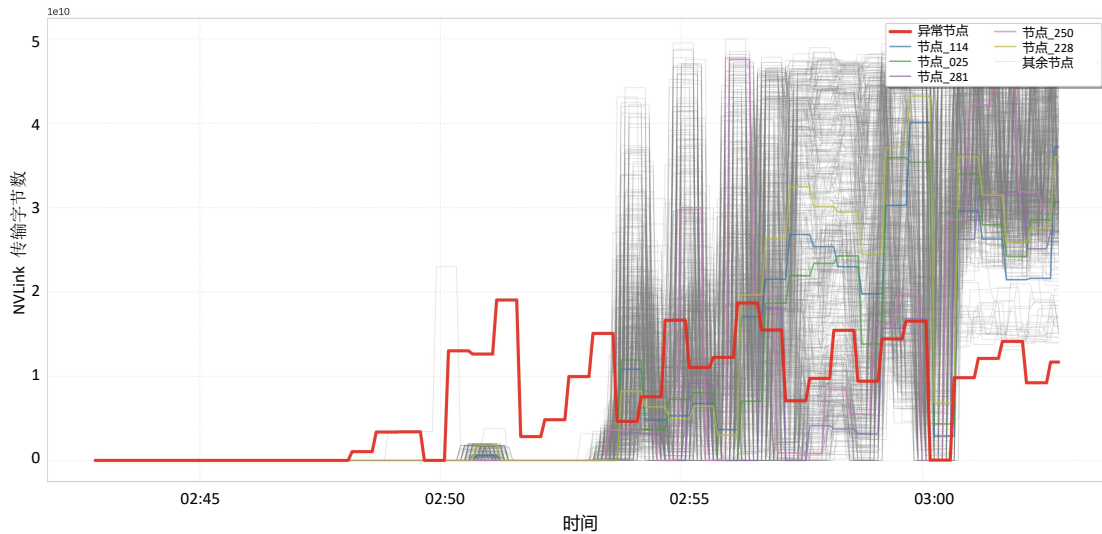


图 4-6 欧氏距离方法优势案例

Figure 4-6 Case Where Euclidean Distance Outperforms

上述两个案例清楚地表明，数值视角和形状视角各有其擅长的故障类型和局限场景。在实际生产环境中，故障的表现形式多种多样，难以事先判断某次故障更适合哪种检测方法。因此，将两种视角进行融合能够综合利用各自的优势，在面对不同类型的异常时都保持较为稳健的检测能力。

4.5.4 算法效率实验对比

为验证前述效率优化措施的实际效果，本节基于真实 GPU 集群 113 个故障检测任务，分别统计了欧氏距离算法和 FR-SAX 算法在优化前后的执行时间。这些任务涵盖了不同规模的检测场景，节点数量从数个到 700 个不等，覆盖了 6 个监控指标。表4-4给出了各算法优化前后的平均执行时间、最大执行时间及加速比。其中 FR-SAX 优化前的数据基于小规模采样测试结果按数据规模比例外推估算。

从表中数据可以看出，向量化计算和预计算查找表的引入带来了极为显著的性能提升。欧氏距离方法的平均执行时间从优化前的 149.97 秒降至 3.78 秒，实现了约 39.7 倍的加速，最大耗时也从 269.58 秒降至 6.59 秒。FR-SAX 方法的优化效果同样显著，优化前由于涉及大量逐元素的符号距离查询操作，Python

表 4-4 算法优化前后效率对比

Table 4-4 Efficiency Comparison Before and After Algorithm Optimization

算法	平均耗时	最大耗时	加速比
欧氏距离（优化前）	149.97s	269.58s	1.0×
欧氏距离（优化后）	3.78s	6.59s	39.7×
FR-SAX（优化前）	217.92s	483.56s	1.0×
FR-SAX（优化后）	7.44s	18.02s	29.3×

层面的循环实现导致平均耗时 217.92 秒，而通过预计算符号距离查找表并结合向量化索引映射，优化后平均耗时降至 7.44 秒，实现了 29.3 倍的加速。优化后 FR-SAX 的平均耗时（7.44 秒）约为欧氏距离（3.78 秒）的 1.97 倍，主要原因在于 FR-SAX 保留了全分辨率的符号序列，距离矩阵的计算规模相应更大。

为进一步分析优化后完整检测流程在不同数据规模下的耗时表现，表4-5按节点规模统计了 113 个检测任务中欧氏距离算法、FR-SAX 算法及双视角融合总耗时的分布情况。

表 4-5 优化后不同节点规模下的平均检测耗时统计

Table 4-5 Average Detection Time Statistics under Different Node Scales after Optimization

节点规模	任务数	欧氏距离	FR-SAX	融合总耗时
~500	33	3.73s	7.33s	11.05s
~600	20	4.50s	8.93s	13.44s
~700	34	5.96s	11.90s	17.86s
全部	113	3.78s	7.44s	11.22s

从表中数据可以看出，优化后两种算法的执行时间均随节点规模近似线性增长。在最常见的 500 节点规模下，欧氏距离算法平均耗时 3.73 秒，FR-SAX 算法平均耗时 7.33 秒，双视角融合的总平均耗时为 11.05 秒。当节点规模增至 700 个时，融合总平均耗时为 17.86 秒，最大耗时为 24.47 秒。全部 113 个检测任务均成功完成，无一超时或失败。考虑到生产环境中故障检测通常以分钟级周期执行，优化后的算法耗时水平完全满足实时监控的响应要求。

4.6 本章小结

本章将异常检测的研究范围从第三章的单指标多节点场景扩展至多指标多节点场景，针对多指标数据规模增长带来的算法效率约束和异常表现异质性导致的检测盲区两方面挑战，提出了 NSFusion 数值与形状双视角融合的异常检测框架。在整体设计上，框架放弃了第三章 CSAD-AT 中计算开销较大的自编码器

和密度偏离方法,采用轻量化的数值距离计算与无需训练的符号化形状度量相结合的方案,在保证检测效果的前提下适应大规模多指标数据的处理需求。在数值视角方面,沿用并优化了基于欧氏距离的曲线相似性度量方法,设计了先分后合的多指标加权融合策略,在各指标独立进行距离计算和标准化的基础上,通过加权聚合得到节点的综合异常评分。在形状视角方面,采用改进的 **FR-SAX** 符号化算法替代自编码器来捕捉曲线形态特征,取消了分段聚合步骤以保留原始时间分辨率,并采用严格的符号距离计算规则以提升对细微形态差异的敏感性,弥补了纯数值视角对变化模式异常的检测盲区。在融合阶段,通过加权平均将两种视角的异常分数整合为统一的故障评估结果。在包含 105 个真实故障案例的 GPU 集群数据集上的实验结果表明,所提融合方法的故障检测准确率 $\text{Acc}@5$ 达到 83.8%,显著优于单一视角方法和马氏距离基线方法,验证了双视角互补融合策略在多指标多节点异常检测任务上的有效性。

第5章 集群节点异常检测与常态化监控系统

前述章节分别针对单指标多节点和多指标多节点两种场景提出了基于曲线相似性的异常检测方法，并通过实验验证了其有效性。然而，算法研究成果要服务于生产环境，还需构建完整的系统平台，将数据采集、预处理、异常检测、告警通知和可视化分析等环节有机串联。本章介绍集群节点异常检测与常态化监控系统的设计与实现，以联通大数据生产底座平台为应用背景，系统具备对接 Prometheus 监控体系的能力，可实现常态化巡检，为运维团队提供自动化的集群健康监控方案。

5.1 系统设计

5.1.1 功能模块设计

系统旨在为分布式集群提供自动化的节点级异常监控与告警能力。根据生产环境运维需求，系统划分为五个核心功能模块：数据接入与管理模块、检测引擎模块、常态化监控模块、告警管理模块以及可视化与数据看板模块。各模块通过标准化接口实现数据流转与交互。

数据接入与管理模块负责对接多种数据源，实现监控数据的统一采集与管理。该模块支持两种输入方式：通过文件上传导入 CSV 格式的历史数据，适用于离线分析和算法验证；通过 Prometheus API 接口实时采集集群监控指标，适用于在线监控。模块同时提供数据集的查询、预览和时序曲线可视化功能。

检测引擎模块是系统的核心计算组件，封装了本文提出的全部异常检测算法。对于单指标多节点场景，集成了第三章提出的 CSAD-AT 算法，包含欧氏距离检测、自编码器嵌入检测和密度偏离检测三种评分方法，以及分数归一化聚合和 I-SPOT 自适应阈值检测的完整流水线。对于多指标多节点场景，集成了第四章提出的数值与形状双视角融合框架。检测引擎根据用户配置的场景类型自动编排检测流程。

常态化监控模块通过定时任务调度机制，按照预设的巡检周期自动从 Prometheus 拉取最新监控数据，执行预处理和异常检测流程，并将结果写入数据库。运维人员无需手动触发，系统即可实现对集群健康状态的持续监控。

告警管理模块实现从告警规则配置、告警触发判定、多渠道通知分发到告警生命周期管理的完整闭环。

在告警规则配置方面，系统支持按集群维度配置差异化的告警规则。每条告警规则包含告警级别、触发阈值和通知方式三个核心要素。系统定义了信息级（记录但不主动推送）、警告级（需要运维人员关注）和严重级（需要立即处置）三个告警级别，每个级别可以独立设置异常分数阈值。

在告警触发方面，告警判定流程在每次检测任务完成后自动执行。系统遍历检测结果中每个节点的异常评分，与该集群关联的所有已启用告警规则进行匹配，当某节点的异常分数超过规则阈值时创建告警记录。通知方式支持邮件和企业微信两种渠道，通知发送采用异步执行以避免阻塞主检测流程。

为避免告警风暴，系统实现了告警抑制机制：同一集群同一节点在可配置的抑制窗口内，相同级别的告警仅触发一次通知。告警记录支持确认操作和处置备注，确保每条告警可追踪到具体的响应人和处理结果，满足企业运维流程的合规要求。系统同时提供告警统计分析功能，按告警级别、集群、时间维度对告警记录进行聚合统计，帮助运维管理者从宏观层面评估集群的稳定性和告警规则的合理性。

可视化与数据看板模块基于 ECharts 图表库构建，通过交互式图表和数据面板为运维人员提供多层次的集群健康状态视图。

实时监控大屏是运维人员日常监控的主入口页面，集中展示所有被监控集群的实时健康状态。页面顶部的状态概览区域以卡片形式显示关键统计指标，包括当前监控的集群数量、最近一小时内检测到的异常节点总数和未处理的告警数量等。页面中部的集群健康状态面板以状态标签区分正常（绿色）、警告（黄色）和严重（红色）三种状态，运维人员可以点击集群卡片快速跳转至详细检测页面。

多节点曲线对比图是系统最核心的可视化视图，用于直观展示异常节点与正常节点群体行为模式之间的偏离情况。图表以折线图绘制所有节点的时序曲线，异常节点以红色加粗线条高亮显示，正常节点以低透明度的蓝色线条作为背景参照，支持鼠标滚轮缩放和拖拽平移操作。异常分数排名以水平柱状图展示各节点评分，阈值线以虚线标记。检测时间线热力图以节点为纵轴、时间步为横轴，通过颜色深浅编码异常分数，展示异常在时间和节点两个维度上的分布规律。历史检测结果页面支持按时间范围和集群筛选，提供检测记录的完整回溯与导出功能。

5.1.2 系统架构设计

系统采用前后端分离的分层架构，从上至下分为用户层、应用层、算法层和数据层四个层次。

用户层提供基于 Web 浏览器的可视化交互界面，利用 ECharts 图表库实现数据可视化效果，通过异步请求与后端 API 进行数据交互。

应用层承接用户请求并协调各功能模块，基于 Flask 框架构建 RESTful API 服务，负责路由分发、业务逻辑编排和响应封装。应用层同时负责定时任务调度，通过 APScheduler 实现常态化监控任务的定时触发，并集成告警通知服务。

算法层封装了本文提出的全部异常检测算法，是系统的核心计算引擎。算法层采用独立的 Python 模块组织，便于算法的迭代更新与扩展。系统的算法模块组织结构如表5-1所示。

表 5-1 系统算法模块组织结构

Table 5-1 Organization of System Algorithm Modules

模块	核心组件	对应章节
euc_detector	欧氏距离偏离度计算	第三章 3.3.1
ae_detector	自编码器嵌入偏离度计算	第三章 3.3.2
dbscan_detector	密度偏离度计算	第三章 3.3.3
ispot	I-SPOT 自适应阈值算法	第三章 3.4.4
pipeline	CSAD-AT 流水线编排	第三章 3.4
fusion	双视角融合检测	第四章 4.2
evaluate	检测结果评估	第三章 3.5
visualize	检测结果可视化	—

数据层采用 SQLite 关系型数据库存储用户信息、集群配置、检测任务、检测结果和告警记录等结构化数据，同时在应用层实现内存缓存机制以提高响应速度。对于 Prometheus 时序数据，系统在每次检测时通过 API 实时拉取，避免数据冗余。

5.1.3 数据库设计

系统的结构化数据采用 SQLite 关系型数据库进行存储，共设计了六张核心数据表，覆盖用户管理、集群配置、检测任务、检测结果、告警规则和告警记录等关键业务实体。各表之间通过主外键关联，保障数据的完整性与一致性。

在用户与集群管理层面，users 表存储用户名、密码哈希值、角色和邮箱等基础信息；cluster_configs 表记录每个被监控集群的 Prometheus 地址、PromQL 查询表达式、巡检间隔和检测场景类型等配置参数，其中 scenario_type 字段区分单指标检测和多指标检测两种场景，系统据此自动选择对应的检测算法流程。

在检测任务与结果层面，detection_tasks 表记录每次检测的任务类型（手动触发或定时巡检）、算法配置参数和执行状态（等待、执行中、已完成、失败）；detection_results 表以节点为粒度存储异常评分和判定结果，其 details 字段以 JSON 格式保存各视角的分项异常分数、阈值判定结果和异常时间窗口等详细信息，便于结果回溯和可视化展示。

在告警管理层面，alert_rules 表支持按集群维度配置差异化的告警策略，每条规则包含告警级别（信息级、警告级、严重级）、触发阈值和通知方式（邮件、企业微信 Webhook、系统内记录）三个核心要素；alert_logs 表保存每条告警事件的触发信息和处理状态，支持告警确认与追踪。

在表间关系上，cluster_configs 通过外键关联 users 表实现集群配置的用户归属追踪，detection_tasks 通过外键关联 cluster_configs 表建立任务与集群的对应关系，detection_results 和 alert_logs 均通过外键关联 detection_tasks 表，确保检测结

果和告警记录可追溯到具体的检测任务实例。

5.2 系统实现

5.2.1 数据接入与预处理

数据接入模块支持文件上传和 Prometheus API 两种数据输入方式，并提供统一的数据集管理功能。

文件上传方式支持 CSV 格式，系统自动检测宽表（列为节点）和长表（包含 timestamp、node、value 列）两种结构并转换为统一的内部格式。文件上传接口限制最大上传大小为 50MB，仅允许指定扩展名的文件类型，防止非法文件注入。Prometheus API 接入通过 HTTP 请求调用 Prometheus 的 query_range 接口，支持标准的 PromQL 查询语法。系统自动解析返回的 JSON 数据，提取节点标识（instance 标签）和时序数值。两种数据接入方式的输出统一为以节点为列、时间戳为索引的标准化数据结构，确保后续预处理和检测模块无需关注数据来源差异。

在数据管理方面，系统提供数据集的查询、预览和删除功能。用户可在数据集管理页面查看已上传数据集的列表信息，包括数据集名称、数据规模（节点数和时间步数）以及上传时间等。模块内置时序数据预览功能，以折线图形式展示各节点的指标变化趋势，支持通过下拉框切换不同指标的预览视图。

预处理流水线对原始监控数据执行系统性清洗与标准化处理，主要包含三个步骤：（1）时间戳对齐，通过重采样方法将不规则采样的数据转换为固定间隔的时序数据，聚合策略采用均值；（2）缺失值处理，采用分段策略，对短时缺失（连续缺失不超过 5 个采样点）采用线性插值方法进行填充，对长时间连续缺失的节点数据直接剔除以避免引入较大误差；（3）数据归一化，支持 Z-score 标准化和 Min-Max 归一化两种方法，用户可根据数据特点选择合适的归一化策略。

5.2.2 检测引擎

检测引擎是系统的核心模块，根据用户配置的场景类型自动选择对应的检测流程。

5.2.2.1 单指标多节点检测流程

检测引擎模块采用模块化设计，将每种算法封装为独立的检测器组件，通过统一的调度接口进行调用，各模块与论文章节的对应关系清晰明确，便于算法的独立迭代和测试验证。

单指标场景的检测流程直接对应第三章的 CSAD-AT 算法。系统接收预处理后的多节点单指标时序数据，执行以下流水线：

（1）多视角异常评分：依次调用欧氏距离检测器、自编码器嵌入检测器和密度偏离检测器，分别从数值距离、形态特征和局部密度三个视角计算各节点在每个滑动窗口的异常分数，产出三个形如（窗口数 $\times$ 节点数）的分数矩阵。

(2) 分数归一化与聚合：对每种方法的分数执行负值截断和 MinMax 归一化，将异常分数映射到 $[0, 1]$ 区间后进行加权平均融合，得到综合异常分数矩阵。

(3) 全局 I-SPOT 自适应阈值检测：将所有节点的融合分数按窗口优先顺序拼接为一维全局时间序列，运行 I-SPOT 算法计算动态阈值并识别异常。全局时间线的设计使所有节点共享同一个 I-SPOT 实例，既降低了计算开销，又使阈值能反映全局分数分布特征。

(4) 异常片段提取：将 I-SPOT 的一维检测结果映射回 (窗口, 节点) 矩阵，提取每个异常节点的连续异常窗口组成异常片段，并映射至对应的时间戳区间。

I-SPOT 算法的核心参数设置为：目标误报率 $q = 0.0065$ ，初始阈值分位数 $\text{level} = 0.98$ ，GPD 重估间隔 $T_{\text{update}} = 50$ 。滑动窗口大小默认为 20，步长默认为 10，三种评分方法采用等权融合。

5.2.2.2 多指标多节点检测流程

多指标场景的检测流程对应第四章的数值与形状双视角融合框架。系统对各指标分别计算数值偏离分数和形状偏离分数，前者基于欧氏距离度量各节点与群体的数值差异，后者通过 FR-SAX 算法提取形状特征并计算差异。两种视角的分数经归一化后以可配置的权重融合，产出节点级的综合异常评分，按分数排序输出异常节点排名。

5.2.3 常态化监控

常态化监控模块通过定时任务调度实现对分布式集群的持续自动巡检。系统为每个已启用的被监控集群创建独立的定时任务，按照配置的巡检间隔（默认 5 分钟）自动执行完整的检测流程：拉取 Prometheus 数据、预处理、异常检测、结果存储和告警规则匹配。

当管理员新增、修改或停用集群监控配置时，系统通过动态更新定时任务列表实现巡检策略的实时生效，无需重启服务。集群管理页面展示所有被监控集群的状态信息，包括集群名称、巡检间隔、最近巡检时间和检测结果摘要。

5.2.4 告警管理

告警管理模块实现从告警规则配置、告警触发判定、多渠道通知分发到告警生命周期管理的完整闭环。

系统支持按集群维度配置差异化的告警规则，每条规则包含告警级别、触发阈值和通知方式三个核心要素。告警触发流程在每次检测任务完成后自动执行，当某节点的异常分数超过规则阈值时创建告警记录并发送通知。通知方式支持邮件和企业微信 Webhook 两种渠道，通知发送采用异步执行以避免阻塞主流程。

为避免告警风暴，系统实现了告警抑制机制：同一集群同一节点在 30 分钟的抑制窗口内，相同级别的告警仅触发一次通知。告警记录支持确认操作和处置备注，确保每条告警可追踪到响应人和处理结果。

5.2.5 可视化与数据看板

可视化模块基于 ECharts 图表库构建，提供多层次的集群健康状态视图。

实时监控大屏集中展示所有集群的健康状态概览，以状态标签区分正常（绿色）、警告（黄色）和严重（红色）三种状态。多节点曲线对比图以折线图绘制所有节点的时序曲线，异常节点以红色高亮显示，支持缩放和拖拽操作。异常分数排名以柱状图展示各节点评分，阈值线以虚线标记。检测时间线以节点为纵轴、时间为横轴，通过颜色编码异常分数，展示异常在时间和节点两个维度上的分布规律。历史检测结果页面支持按时间范围和集群筛选，提供检测记录的完整回溯与导出。

5.2.6 RESTful API 接口设计

系统后端基于 Flask 框架提供 RESTful API 服务，所有前后端交互均通过标准化的 HTTP 接口完成。表5-2列出了系统的核心 API 接口及其功能说明。

表 5-2 系统核心 API 接口设计
Table 5-2 Design of Core System API Endpoints

接口路径	请求方法	功能说明
/api/auth/login	POST	用户登录认证
/api/datasets	GET/POST	数据集列表查询与上传
/api/datasets/{id}/preview	GET	数据集时序曲线预览
/api/clusters	GET/POST/PUT	集群配置管理
/api/detection/run	POST	提交异常检测任务
/api/detection/results/{id}	GET	查询检测结果详情
/api/monitor/status	GET	常态化监控状态查询
/api/alerts/rules	GET/POST/PUT	告警规则配置管理
/api/alerts/logs	GET	告警记录查询与筛选
/api/dashboard/overview	GET	监控大屏数据聚合

所有 API 接口均遵循统一的响应格式，包含状态码、消息和数据三个字段。需要身份认证的接口通过 Flask-Session 的会话令牌机制进行校验，管理员专属接口额外进行角色权限检查。对于耗时较长的检测任务，系统采用异步提交模式，前端通过轮询任务状态接口获取执行进度和最终结果。

5.3 系统功能验证

本节对系统的各项功能进行验证，评估系统在不同数据规模下的检测性能，并通过导入历史数据模拟常态化监控流程，验证系统各模块的正确性和可用性。

系统已在测试集群上完成功能验证，待测试通过后计划部署到生产集群，实现对关键业务指标的常态化自动巡检。

图5-1展示了系统异常检测功能的界面。左侧为检测配置面板，用户可选择数据来源、检测场景和滑动窗口大小，并勾选所需的检测方法（欧氏距离相似性、自编码器嵌入、DBSCAN 聚类）。右侧上方显示本次检测的统计摘要，包括监控节点数、监控指标数、检测框架、异常节点数和异常片段数。右侧下方以多节点曲线对比图的形式展示检测到的异常时间片段，异常节点的时序曲线以红色高亮标注，便于运维人员快速定位异常行为的时间范围和偏离程度。

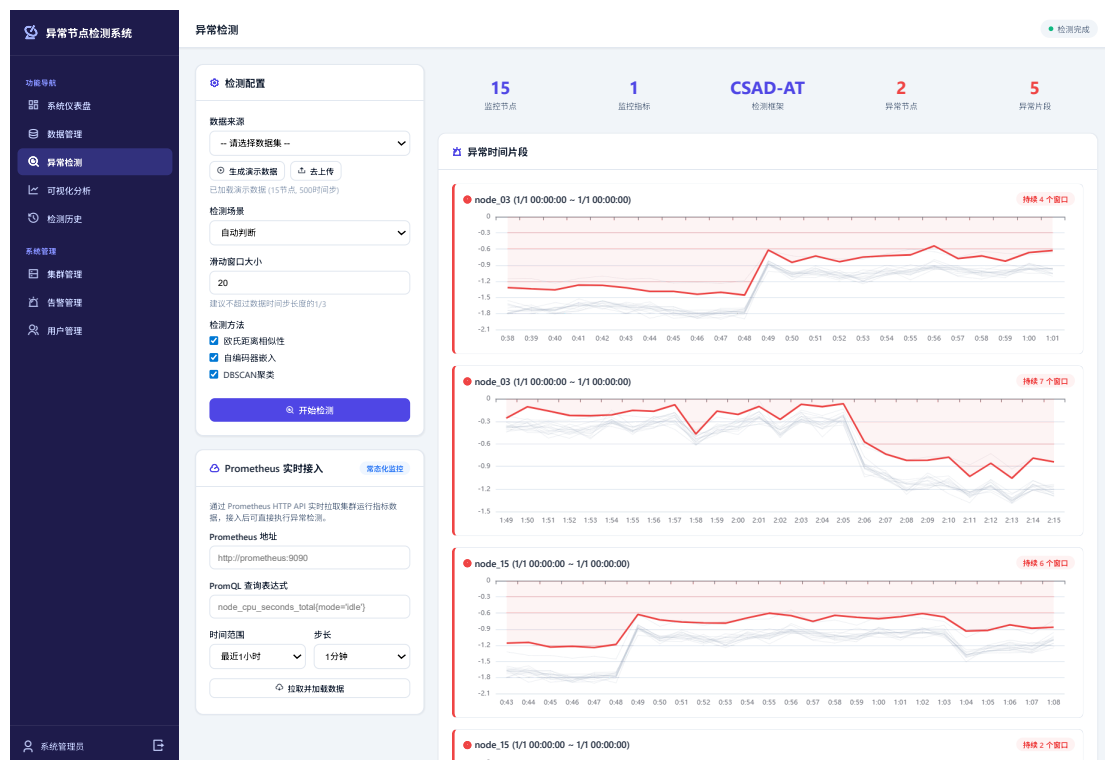


图 5-1 异常检测系统检测结果可视化界面

Figure 5-1 Visualization Interface of Anomaly Detection System Results

5.3.1 系统性能评估

表5-3展示了系统在不同规模数据下的检测耗时。欧氏距离检测器得益于向量化实现，在 50 节点、1000 时间步的场景下也能在 3 秒内完成。自编码器检测器的训练阶段耗时较长，但推理阶段与欧氏距离检测器相当。

在常态化监控场景下，系统需在每个巡检周期内完成数据拉取、预处理和检测的完整流程。表5-4展示了模拟典型 YARN 集群规模（30 个节点，500 个时间步）时各处理阶段的耗时分布，其中 Prometheus 数据拉取阶段使用本地缓存的历史数据模拟。单次巡检的端到端耗时约 20 秒，远低于预设的 5 分钟巡检间隔，表明系统在部署到生产集群后能够稳定支持多个集群的并行巡检。

表 5-3 系统检测性能（单位：秒）

Table 5-3 System Detection Performance (Unit: Seconds)

数据规模	欧氏距离	自编码器	密度偏离	CSAD-AT 流水线
15 节点 ×500 步	0.8	12.5	1.2	14.8
30 节点 ×500 步	1.5	15.3	2.1	19.4
50 节点 ×1000 步	3.2	28.7	4.5	37.1

表 5-4 常态化巡检端到端耗时分布（30 节点 ×500 步）

Table 5-4 End-to-End Latency Distribution of Routine Inspection (30 Nodes × 500 Steps)

处理阶段	平均耗时（秒）	占比
Prometheus 数据拉取	1.2	6.2%
数据预处理	0.8	4.1%
CSAD-AT 检测流水线	16.5	85.1%
结果存储与告警检查	0.9	4.6%
总计	19.4	100%

5.3.2 常态化监控功能验证

为验证常态化监控模块的功能完整性，本研究使用第三章构建的标注数据集模拟在线巡检场景：将历史数据按时间窗口逐段输入系统，模拟 Prometheus 数据拉取过程，系统自动执行预处理、异常检测、结果存储和告警规则匹配的完整流程。验证结果表明，系统能够正确执行定时任务调度、自动编排检测流水线，并在检测到异常时按预设规则生成告警记录。

基于历史标注数据的回放验证表明，系统的检测结果与第三章离线实验一致，验证了工程实现的正确性。系统在设计上具备以下核心能力：

- （1）检测方法上，系统通过多节点曲线对比定义异常，不依赖对“正常范围”的先验定义，对负载敏感型指标具有更强的鲁棒性。
- （2）阈值管理上，I-SPOT 自适应阈值算法自动跟踪数据分布变化，无需人工调整阈值参数，可减轻运维人员的配置负担。
- （3）运维模式上，常态化监控可实现从被动响应到主动发现的转变，在部署到生产集群后有望将异常发现的平均时间从依赖人工巡检的小时级缩短至分钟级。

5.3.3 告警功能验证

告警管理模块的功能通过模拟数据进行了验证。系统能够根据配置的告警规则自动判定告警级别（信息级、警告级、严重级），正确生成告警记录，并通过邮件和企业微信两种渠道发送通知。告警抑制机制在测试中表现正常，能够在

设定的抑制窗口内过滤同一节点的重复告警，避免告警风暴。通知发送采用异步执行机制，不影响主检测流程的运行。

系统已在测试集群上完成全部告警功能的验证，具备对接联通大数据生产底座平台 Prometheus 监控体系的接口能力，待测试集群验证通过后计划部署到生产集群，届时可进一步收集真实业务环境下的告警统计数据 and 检测效果反馈。

5.4 本章小结

本章详细介绍了集群节点异常检测系统的设计与实现过程，将前述章节的算法研究成果转化为可落地的工程系统。

在系统设计方面，系统划分为数据接入与管理、检测引擎、常态化监控、告警管理和可视化与数据看板五个功能模块，采用用户层、应用层、算法层和数据层的四层前后端分离架构。数据库设计了用户表、集群配置表、检测任务表、检测结果表、告警规则表和告警记录表六张核心数据表，完整覆盖了系统的业务数据存储需求。

在系统实现方面，数据接入模块支持文件上传和 Prometheus API 两种数据源，预处理模块实现了包含时间戳对齐、缺失值处理和归一化的完整流水线。检测引擎以模块化设计封装了本文提出的全部检测算法，根据场景类型自动编排单指标或多指标检测流水线。常态化监控通过 APScheduler 定时任务框架实现对多个集群的持续自动巡检，告警管理构建了三级告警机制并支持邮件和企业微信等多渠道通知，同时实现了告警抑制和告警生命周期管理功能。RESTful API 接口设计遵循统一的响应格式规范，为前后端交互提供了标准化的通信协议。

在功能验证方面，系统通过历史标注数据的回放测试验证了各模块的功能完整性和检测结果的正确性。性能测试表明，单次巡检端到端耗时约 20 秒，满足 5 分钟巡检间隔的性能要求。系统已在测试集群上完成全部功能验证，待测试通过后计划部署到生产集群，为运维团队提供自动化的集群健康监控服务。

第6章 总结与展望

6.1 本文工作总结

分布式集群是云计算和大数据处理的底层支撑，一旦出现故障会影响上层业务的正常运行。目前已有的异常检测方法大多只关注时间和指标两个维度，没有充分利用集群中多个节点行为相似这一特点。针对这个问题，本文提出了基于多元时序曲线相似性的异常检测方法，并开发了配套的检测系统。以下对本文的主要工作做简要总结。

首先，提出了一种基于多节点曲线相似性的异常检测思路。与传统方法只分析时间和指标不同，本文把集群节点作为第三个分析维度加入模型，形成了时间、指标、节点三个维度的联合分析方式。算法设计的核心思路是：根据运维经验，分布式集群通过负载均衡使各节点的运行行为保持一致，所以可以用节点之间的行为差异来判断异常，有效减少了因分析单节点变化的绝对正常模式而产生的误报问题。

在单指标多节点的场景下，本文从联通大数据的生产环境中采集数据，构建了标注完整的时序数据集。在此基础上，分别用欧氏距离、自编码器嵌入和 DBSCAN 聚类三种方式来衡量节点曲线之间的相似程度，并提出了 CSAD-AT 算法。该算法框架对多种评分方法的结果做归一化和加权融合，同时使用 I-SPOT 算法自动调整检测阈值，避免了人工设定固定阈值的不稳定性。实验对比了 OmniAnomaly、USAD、TranAD 等方法，结果显示本文方法在 F1 分数上显著优于现有多元时序异常检测基线算法。

针对多指标多节点场景，本文提出了 NSFusion 数值与形状双视角融合的异常检测算法。针对多指标数据规模增长带来的效率约束，框架放弃了单指标场景中的自编码器和密度偏离方法，采用轻量化的计算方案。在数值视角方面，采用优化的基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面，提出 FR-SAX (Full-Resolution SAX) 符号化表示算法作为自编码器的替代方案，通过保留原始时间分辨率和采用严格距离计算规则来捕捉曲线形态差异，弥补纯数值视角的检测盲区。在 GPU 集群故障数据集上的实验表明，融合方法的故障检测准确率达到 83.8%，显著优于单一视角方法和传统基线方法。

在以上算法研究的基础上，本文设计并实现了集群节点异常检测与监控系统。系统采用前后端分离的四层架构，以模块化方式封装全部检测算法，支持常态化自动巡检和多级告警通知，具备对接 Prometheus 监控体系的接口能力。功能验证表明，单次巡检端到端耗时约 20 秒，满足分钟级巡检间隔的性能要求，为生产环境的实际部署奠定了工程基础。

6.2 未来研究展望

本文的研究工作尚存在一些局限性，以下几个方向值得在未来进一步探索。

(1) 节点拓扑感知与异构集群适配。本文假设同构集群内各节点地位平等，以此为前提利用节点间的行为相似性进行异常检测。然而，实际生产环境中的集群往往具有更复杂的结构特征：节点之间可能存在主从、上下游等拓扑依赖关系，不同角色的节点承担差异化的计算任务；此外，异构集群中不同硬件配置或软件版本的节点，其正常行为模式本身就存在系统性差异。未来可以引入图神经网络等方法显式建模节点间的拓扑关系和角色属性，在此基础上自动识别可比较的节点分组，并针对不同分组设计适应性更强的相似性度量策略，从而将本文方法的适用范围从同构集群扩展到更一般的异构分布式系统。

(2) 在线增量学习与概念漂移适应。当前方法主要采用批量处理模式，自编码器深度学习组件需要在离线阶段完成训练后才能部署。然而，分布式集群的运行环境是持续演变的，业务负载模式、集群规模和软件版本的变化都可能导致节点行为的正常基线发生漂移。如果模型不能及时跟踪这些变化，将逐渐产生更多的误报或漏报。未来可以研究在线增量学习机制，使模型在保持对历史异常模式记忆的同时，通过轻量级的参数更新策略持续适应新的正常行为分布，结合概念漂移检测方法自动判断何时需要触发模型重训练，从而在长期运行中维持稳定的检测性能。

(3) 异常根因分析与智能运维闭环。本文的工作聚焦于异常检测和故障节点定位，能够回答“哪个节点在何时出现了异常”的问题，但对于“为什么出现异常”缺乏深入分析。在实际运维场景中，检测到异常只是故障处置的第一步，运维人员还需要进一步定位异常的根本原因才能采取有效的修复措施。未来可以将时序监控数据与系统日志、配置变更记录、部署事件等多源异构数据进行关联分析，结合因果推理方法构建从异常检测、根因定位到修复建议的完整智能运维链路，进一步提升系统的自动化运维能力，推动分布式集群运维从被动响应向主动预防的范式转变。

参考文献

- [1] Grubbs F E. Procedures for detecting outlying observations in samples [J]. *Technometrics*, 1969, 11(1): 1-21.
- [2] Dixon W J. Analysis of extreme values [J]. *The Annals of Mathematical Statistics*, 1950, 21(4): 488-506.
- [3] Mahalanobis P C. On the generalised distance in statistics [J]. *Proceedings of the National Institute of Science of India*, 1936, 2: 49-55.
- [4] Box G E, Jenkins G M. Time series analysis: Forecasting and control [M]. Holden-Day, 1970.
- [5] Cleveland R B, Cleveland W S, McRae J E, et al. Stl: A seasonal-trend decomposition procedure based on loess [J]. *Journal of Official Statistics*, 1990, 6(1): 3-73.
- [6] Defreitas A, Alexander W N, Devenport W, et al. Anomaly detection in wind tunnel experiments by principal component analysis [J]. *AIAA Journal*, 2022, 60(4): 2297-2307.
- [7] Sharpnack J, Rinaldo A, Singh A. Detecting anomalous activity on networks with the graph Fourier scan statistic [J]. *IEEE Transactions on Signal Processing*, 2015, 64(2): 364-379.
- [8] Grané A, Veiga H. Wavelet-based detection of outliers in financial time series [J]. *Computational Statistics & Data Analysis*, 2010, 54(11): 2580-2593.
- [9] Ramaswamy S, Rastogi R, Shim K. Efficient algorithms for mining outliers from large data sets [C]//*Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. 2000: 427-438.
- [10] Breunig M M, Kriegel H P, Ng R T, et al. Lof: identifying density-based local outliers [J]. *ACM SIGMOD Record*, 2000, 29(2): 93-104.
- [11] Liu F T, Ting K M, Zhou Z H. Isolation forest [C]//*Proceedings of the 8th IEEE International Conference on Data Mining*. 2008: 413-422.
- [12] MacQueen J B, et al. Some methods for classification and analysis of multivariate observations [C]//*Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability: volume 1*. 1967: 281-297.
- [13] Celebi M E. A comparative study of efficient initialization methods for the k-means clustering algorithm [J]. *Expert Systems with Applications*, 2013, 40(1): 4708-4718.
- [14] Li J, Izakian H, Pedrycz W, et al. Clustering-based anomaly detection in multivariate time series data [J]. *Applied Soft Computing*, 2021, 100: 106919.
- [15] Angiulli F, Pizzuti C. Fast outlier detection in high dimensional spaces [C]//*Proceedings of the European Conference on Principles of Data Mining and Knowledge Discovery*. 2002: 15-27.
- [16] Chen T, Guestrin C. XGBoost: A scalable tree boosting system [C]//*Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016: 785-794.
- [17] Choi K, Yi J, Park C, et al. Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines [J]. *IEEE Access*, 2021, 9: 120043-120065.
- [18] Pang G, Shen C, Cao L, et al. Deep learning for anomaly detection: A review [J]. *ACM Computing Surveys*, 2021, 54(2): 1-38.
- [19] Su Y, Zhao Y, Niu C, et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network [C]//*Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019: 2828-2837.

- [20] Audibert J, Michiardi P, Guyard F, et al. Usad: Unsupervised anomaly detection on multivariate time series [C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020: 3395-3404.
- [21] Hochreiter S, Schmidhuber J. Long short-term memory [J]. Neural computation, 1997, 9(8): 1735-1780.
- [22] Tuli S, Casale G, Jennings N R. Tranad: Deep transformer networks for anomaly detection in multivariate time series data [C]//Proceedings of the VLDB Endowment: volume 15. 2022: 1201-1214.
- [23] Xu J, Wu H, Wang J, et al. Anomaly transformer: Time series anomaly detection with association discrepancy [C]//International Conference on Learning Representations. 2022.
- [24] Deng A, Hooi B. Graph neural network-based anomaly detection in multivariate time series [C]//Proceedings of the AAAI Conference on Artificial Intelligence: volume 35. 2021: 4027-4035.
- [25] Chen W, Liu Y, Zhang M. Carots: Causal-aware contrastive learning for robust multivariate time series anomaly detection [C]//Proceedings of the 42nd International Conference on Machine Learning. 2025.
- [26] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition [C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.
- [27] Hundman K, Constantinou V, Laporte C, et al. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding [C]//Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2018: 387-395.
- [28] Munir M, Siddiqui S A, Dengel A, et al. DeepAnT: A deep learning approach for unsupervised anomaly detection in time series [J]. IEEE Access, 2018, 7: 1991-2005.
- [29] Ren H, Xu B, Wang Y, et al. Time-series anomaly detection service at Microsoft [C]//Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2019: 3009-3017.
- [30] Darban Z Z, Webb G I, Pan S, et al. Carla: Self-supervised contrastive representation learning for time series anomaly detection [J]. Pattern Recognition, 2025, 157: 110874.
- [31] Paparrizos J, Kang Y, Boniol P, et al. Tsb-uad: An end-to-end benchmark suite for univariate time-series anomaly detection [J]. Proceedings of the VLDB Endowment, 2022, 15(8): 1697-1711.
- [32] Wu Q, Jiang T, Zhang Y. Tab: Unified benchmarking of time series anomaly detection methods [J]. arXiv preprint arXiv:2506.18046, 2024.
- [33] Fan X, Li F, Qi L. Distributed discord discovery: Spark based anomaly detection in time series [C]//Proceedings of the IEEE International Conference on High Performance Computing and Communications. 2015: 154-159.
- [34] Carbone P, Katsifodimos A, Ewen S, et al. Apache Flink: Stream and batch processing in a single engine [J]. IEEE Data Engineering Bulletin, 2015, 36(4): 28-38.
- [35] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding [J]. arXiv preprint arXiv:1810.04805, 2018.
- [36] Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training [R]. OpenAI, 2018.
- [37] Dang W, Zhou B, Zhang W, et al. Time series anomaly detection based on language model [C]//Proceedings of the 11th ACM International Conference on Future Energy Systems. 2020: 544-547.

- [38] Dang W, Zhou B, Wei L, et al. TS-Bert: Time series anomaly detection via pre-training model Bert [C]//Computational Science – ICCS 2021, Lecture Notes in Computer Science. Springer, 2021: 209-223.
- [39] Alnegheimish S, Nguyen L, Berti-Equille L, et al. Large language models can be zero-shot anomaly detectors for time series? [J]. arXiv preprint arXiv:2405.14755, 2024.
- [40] 裴丹, 张圣林. 基于机器学习的智能运维 [J]. 中国计算机学会通讯, 2017, 13(12): 68-73.
- [41] Chandola V, Banerjee A, Kumar V. Anomaly detection: A survey [J]. ACM Computing Surveys, 2009, 41(3): 1-58.
- [42] Braei M, Wagner S. Anomaly detection in univariate time-series: A survey on the state-of-the-art [J]. arXiv preprint arXiv:2004.00433, 2020.
- [43] Blázquez-García A, Conde A, Mori U, et al. A review on outlier/anomaly detection in time series data [J]. ACM Computing Surveys, 2021, 54(3): 1-33.
- [44] 陈福荣, 朱贵富, 李淑琴, 等. 时间序列异常检测综述 [J]. 北京交通大学学报, 2025, 49(1): 1-13.
- [45] Schölkopf B, Platt J C, Shawe-Taylor J, et al. Estimating the support of a high-dimensional distribution [J]. Neural Computation, 2001, 13(7): 1443-1471.
- [46] Ester M, Kriegel H P, Sander J, et al. A density-based algorithm for discovering clusters in large spatial databases with noise [J]. KDD, 1996, 96(34): 226-231.
- [47] Ishimtsev V, Bernstein A, Burnaev E, et al. Conformal k-NN anomaly detector for univariate data streams [C]//Proceedings of Conformal and Probabilistic Prediction and Applications. 2017: 213-227.
- [48] Jin B, Chen Y, Li D, et al. A one-class support vector machine calibration method for time series change point detection [C]//Proceedings of IEEE International Conference on Prognostics and Health Management. 2019: 1-5.
- [49] Schmidl S, Wenig P, Papenbrock T. Anomaly detection in time series: A comprehensive evaluation [J]. Proceedings of the VLDB Endowment, 2022, 15(9): 1779-1797.
- [50] Chalapathy R, Chawla S. Deep learning for anomaly detection: A survey [J]. arXiv preprint arXiv:1901.03407, 2019.
- [51] Chung J, Gulcehre C, Cho K, et al. Empirical evaluation of gated recurrent neural networks on sequence modeling [C]//NIPS 2014 Workshop on Deep Learning. 2014.
- [52] Malhotra P, Vig L, Shroff G, et al. Long short term memory networks for anomaly detection in time series [C]//European Symposium on Artificial Neural Networks. 2015: 89-94.
- [53] Zong B, Song Q, Min M R, et al. Deep autoencoding Gaussian mixture model for unsupervised anomaly detection [C]//International Conference on Learning Representations. 2018.
- [54] Kingma D P, Welling M. Auto-encoding variational bayes [J]. arXiv preprint arXiv:1312.6114, 2014.
- [55] Xu H, Chen W, Zhao N, et al. Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications [C]//Proceedings of the 2018 World Wide Web Conference. 2018: 187-196.
- [56] Zhou B, Liu S, Hooi B, et al. BeatGAN: Anomalous rhythm detection using adversarially generated time series [C]//Proceedings of the 28th International Joint Conference on Artificial Intelligence. 2019: 4433-4439.
- [57] Geiger A, Liu D, Alnegheimish S, et al. TadGAN: Time series anomaly detection using generative adversarial networks [C]//IEEE International Conference on Big Data. 2020: 33-43.

- [58] Zhang H, Wang Y, Huang T. Tfmae: Temporal-frequency masked autoencoder for time series anomaly detection [C]//Proceedings of the 40th IEEE International Conference on Data Engineering. 2024: 1-12.
- [59] Wu H, Hu T, Liu Y, et al. TimesNet: Temporal 2d-variation modeling for general time series analysis [C]//International Conference on Learning Representations. 2023.
- [60] Yu Z, Ma M, Zheng Z, et al. Autokad: Empowering kpi anomaly detection with label-free deployment [C]//Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering (ISSRE). 2023: 364-375.
- [61] Zhou Q, Ma M, Pei D, et al. Kan-ad: Time series anomaly detection with kolmogorov-arnold networks [C]//Proceedings of the 42nd International Conference on Machine Learning (ICML). 2025.
- [62] Malhotra P, Ramakrishnan A, Anand G, et al. LSTM-based encoder-decoder for multi-sensor anomaly detection [C]//ICML Workshop on Anomaly Detection. 2016.
- [63] Chen Z, Yeo C K, Lee B S, et al. Autoencoder-based network anomaly detection [C]//2018 Wireless Telecommunications Symposium. 2018: 1-5.
- [64] Park D, Hoshi Y, Kemp C C. A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder [J]. IEEE Robotics and Automation Letters, 2018, 3 (3): 1544-1551.
- [65] Chen Y, Kang Y, Chen Y, et al. DCdetector: Dual attention contrastive representation learning for time series anomaly detection [C]//Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2023: 218-228.
- [66] Zhao H, Wang Y, Duan J, et al. Multivariate time-series anomaly detection via graph attention network [C]//Proceedings of the 2020 IEEE International Conference on Data Mining. 2020: 841-850.
- [67] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks [C]//International Conference on Learning Representations. 2018.
- [68] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks [J]. arXiv preprint arXiv:1609.02907, 2017.
- [69] Wu Z, Pan S, Chen F, et al. A comprehensive survey on graph neural networks [J]. IEEE Transactions on Neural Networks and Learning Systems, 2021, 32(1): 4-24.
- [70] Dai E, Chen J. Graph-augmented normalizing flows for anomaly detection of multiple time series [C]//International Conference on Learning Representations. 2022.
- [71] Zhang J, Chen H, Li Y. Graph spatiotemporal process for multivariate time series anomaly detection with missing values [J]. arXiv preprint arXiv:2401.05800, 2024.
- [72] Liu Y, Wang H, Zhang Q. Mgcl: Multiorder graph neural network with cross-learning for multivariate time-series anomaly detection [J]. IEEE Transactions on Instrumentation and Measurement, 2025.
- [73] Chen X, Huang W, Wang Z. Entropy causal graphs for multivariate time series anomaly detection [J]. ACM Transactions on Knowledge Discovery from Data, 2025.
- [74] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets [C]//Advances in Neural Information Processing Systems: volume 27. 2014.
- [75] Li D, Chen D, Shi L, et al. Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks [C]//International Conference on Artificial Neural Networks. 2019: 703-716.

- [76] Ren H, Xu B, Wang Y, et al. Time-series anomaly detection service at Microsoft [C]// Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2019: 3009-3017.
- [77] Cañizo M, Triguero I, Conde A, et al. Multi-head CNN-RNN for multi-time series anomaly detection: An industrial case study [C]//Neurocomputing: volume 363. 2019: 246-260.
- [78] Wang Z, Ma M, et al. Revisiting VAE for unsupervised time series anomaly detection: A frequency perspective [C]//Proceedings of the ACM on Web Conference 2024. 2024: 3096-3105.
- [79] Zhang C, Zhou T, Wen Q, et al. TFAD: A decomposition time series anomaly detection architecture with time-frequency analysis [C]//Proceedings of the 31st ACM International Conference on Information & Knowledge Management. 2022: 2497-2507.
- [80] He S, Zhu J, He P, et al. Experience report: System log analysis for anomaly detection [C]// Proceedings of the 27th International Symposium on Software Reliability Engineering. 2016: 207-218.
- [81] Ma M, Zhang S, Chen J, et al. Jump-starting multivariate time series anomaly detection for online service systems [C]//Proceedings of the 2021 USENIX Annual Technical Conference. 2021: 413-426.
- [82] Boniol P, Palpanas T. Distributed detection of sequential anomalies in univariate time series [J]. The VLDB Journal, 2022, 31: 233-256.
- [83] Siffer A, Fouque P A, Termier A, et al. Anomaly detection in streams with extreme value theory [C]//Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2017: 1067-1075.
- [84] Lin J, Keogh E, Wei L, et al. Experiencing sax: a novel symbolic representation of time series [J]. Data Mining and Knowledge Discovery, 2007, 15(2): 107-144.

致 谢

时光荏苒，研究生阶段的学习与科研生涯即将画上句号。回首这段充实而难忘的旅程，心中满怀感恩。

首先，我要衷心感谢我的导师裴昶华老师。裴老师不仅在学术研究上给予了我细致入微的指导，从选题方向的确立到研究思路的梳理都耐心引导、悉心点拨，更在日常生活中给予了许多关怀与帮助。裴老师严谨求实的治学态度和温和耐心的为人风范使我受益终身。

感谢我的企业导师陈斌老师，在校企联合培养期间为我提供了宝贵的实践平台和工程指导，让我能够将所学知识与真实的生产环境相结合。感谢联通的前辈蔡总和浪哥，在项目实施过程中给予的专业指导和耐心帮助，让我对工业界的实际需求有了更深刻的理解。

感谢师兄周全、王泽鑫和司昊田，在科研道路上给予的无私帮助和经验分享。无论是技术难题的讨论还是实验方案的优化，师兄们总是不吝赐教，令我受益匪浅。

感谢中国科学院大学计算机网络信息中心提供的优越科研环境和学术资源，感谢联通数据产品团队的各位同事在联培期间的关心与指导，良好的工作氛围和团队协作让我的研究工作得以顺利推进。

感谢我的父母，感谢你们一直以来的付出和无条件的支持，你们的爱是我求学路上最坚实的后盾。

最后，感谢自己在这段旅程中始终心怀希望，愿未来继续对生活保有热忱，不负所学，砥砺前行。

2026年6月

作者简历及攻读学位期间发表的学术论文与其他相关学术成果

作者简历：

2018 年 9 月——2023 年 6 月，在北京大学信息科学技术学院获学士学位。

2023 年 9 月——2026 年 6 月，在中国科学院大学计算机网络信息中心攻读硕士学位。

参加的研究项目及获奖情况：

联通大数据集群异常检测及预测算法研发项目

联通大数据集群”健康诊断”系统项目

